



ISTQB Certified Tester

AI Testing (CT-AI) Syllabus

AI 테스트팅(CT-AI) 실러버스

Version 1.0

Korean 버전 1.0.3

(사) KSTQB

Korean Software Testing Qualifications Board

International Software Testing Qualifications Board

Copyright Notice

Copyright Notice© International Software Testing Qualifications Board (hereinafter called ISTQB®)

ISTQB® is a registered trademark of the International Software Testing Qualifications Board.

Copyright © 2021, the authors Klaudia Dussa-Zieger (chair), Werner Henschelchen, Vipul Kocher, Qin Liu, Stuart Reid, Kyle Siemens, and Adam Leon Smith.

All rights reserved. The authors hereby transfer the copyright to the ISTQB®. The authors (as current copyright holders) and ISTQB® (as the future copyright holder) have agreed to the following conditions of use:

출처를 명기하는 조건으로, 비상업적인 사용을 위해 본 문서의 발체가 가능합니다. ISTQB 인증 교육 기관(제공사)은 해당 교육 과정을 위해 ISTQB® 회원국(기관)에서 인증한 교재를 확보한 후 실러버스 저자들 및 ISTQB®가 교재의 출처 및 저작권 소유자임을 밝히는 조건으로 교육과정의 기반으로 이 실러버스를 사용할 수 있습니다.

저자들과 ISTQB®가 이 실러버스의 출처 및 저작권 소유자임을 밝히는 경우, 개인이나 그룹은 이 실러버스를 기사나 책에 활용할 수 있습니다. ISTQB®의 서면 동의 없이 해당 실러버스를 다른 용도로 사용하는 것은 금지됩니다.

ISTQB®가 인정한 모든 회원국(기관)은 전술한 저작권 고지를 번역본에 기입하는 조건으로 해당 실러버스 번역이 가능합니다.

본 ISTQB AI Testing (CT-AI) 한글 실러버스의 저작권은 ISTQB에 있습니다.

- Korean Software Testing Qualifications Board

목차

Copyright Notice	2
Acknowledgements	8
0. 서문 Introduction	9
0.1 실러버스의 목적	9
0.2 AI 테스트 인증 테스터 (CT-AI: Certified Tester AI Testing)	9
0.3 출제 가능 학습 목표와 지식 인지 수준	10
0.4 역량(Competencies)의 실습 수준	10
0.5 The Certified Tester AI Testing 시험	11
0.6 인증 (Accreditation)	11
0.7 상세 수준	11
0.8 실러버스 구성 방법	12
제1장 인공지능 소개	13
1.1 인공지능의 정의와 인공지능 효과	15
1.2 약인공지능, 강인공지능, 초인공지능	15
1.3 인공지능기반 시스템과 전통적 시스템	16
1.4 인공지능 기술	16
1.5 인공지능 개발 프레임워크	17
1.6 인공지능기반 시스템을 위한 하드웨어	17
1.7 서비스형 인공지능	19
1.7.1 서비스형 인공지능 계약	19
1.7.2 서비스형 인공지능 사례	20
1.8 사전 훈련 모델	20
1.8.1 사전 훈련 모델	20
1.8.2 전이 학습	20
1.8.3 사전 훈련 모델과 전이 학습 사용 리스크	21
1.9 표준, 규정, 인공지능	22

제2장 인공지능기반 시스템 품질 특성	23
2.1 유연성과 적응성	25
2.2 자율성	25
2.3 진화	26
2.4 편향성	26
2.5 윤리	27
2.6 부작용과 보상 해킹	28
2.7 투명성, 해석 가능성, 설명 가능성	28
2.8 안전과 인공지능	29
제3장 기계학습(ML) - 개요	31
3.1 기계학습 종류	32
3.1.1 지도학습	32
3.1.2 비지도학습	32
3.1.3 강화 학습	33
3.2 기계학습 워크플로	33
3.3 기계학습 종류 선택	37
3.4 기계학습 알고리즘 선택과 관련된 요소	37
3.5 과적합과 과소적합	38
3.5.1 과적합	38
3.5.2 과소적합	38
3.5.3 실습 예제: 과적합과 과소적합 시연	38
제4장 기계학습 - 데이터	39
4.1 기계학습 워크플로에서의 데이터 준비	40
4.1.1 데이터 준비 과정의 어려움	41
4.1.2 실습: 기계학습을 위한 데이터 준비	42
4.2 기계학습 구현 단계에서의 훈련, 검증, 테스트 데이터	42
4.2.1 실습: 학습과 테스트 데이터 식별 및 기계학습 모델 생성	43
4.3 데이터 세트 품질 문제	44
4.4 데이터 품질과 데이터 품질이 기계학습 모델에 미치는 영향	45

4.5 지도학습을 위한 데이터 라벨링	45
4.5.1 데이터 라벨링 접근법	46
4.5.2 데이터 세트에서 잘못 라벨링 된 데이터	46
제5장 기계학습 기능적 성능 측정지표	47
5.1 혼동 행렬	48
5.2 분류, 회귀, 군집화를 위한 추가 기계학습 기능적 성능 측정지표	49
5.3 기계학습 기능적 성능 측정지표의 한계	50
5.4 기계학습 기능적 성능 지표 선택	50
5.4.1 실습: 생성된 기계학습 모델 평가	51
5.5 기계학습 벤치마크 스위트	51
제6장 기계학습 - 신경망과 테스트	53
6.1 신경망	54
6.1.1 실습: 단층 퍼셉트론 구현	55
6.2 신경망 커버리지 측정지표	56
제7장 인공지능기반 시스템 테스트 개요	58
7.1 인공지능기반 시스템 명세	60
7.2 인공지능기반 시스템의 테스트 레벨	60
7.2.1 입력 데이터 테스트	61
7.2.2 기계학습 모델 테스트	61
7.2.3 컴포넌트 테스트	62
7.2.4 컴포넌트 통합 테스트	62
7.2.5 시스템 테스트	62
7.2.6 인수 테스트	62
7.3 인공지능기반 시스템 테스트를 위한 테스트 데이터	63
7.4 인공지능기반 시스템의 자동화 편향성에 대한 테스트	63
7.5 인공지능 컴포넌트의 문서화	64
7.6 컨셉 드리프트(Concept Drift) 테스트	65
7.7 기계학습 시스템을 위한 테스트 접근법 선택	66

제8장 인공지능 특유 품질 특성 테스트	68
8.1 자가학습 시스템 테스트의 어려움	70
8.2 자율 인공지능기반 시스템 테스트	71
8.3 알고리즘, 샘플, 부적절한 편향성 테스트	72
8.4 확률적 및 비 결정적 인공지능기반 시스템 테스트의 어려움	72
8.5 복잡한 인공지능기반 시스템 테스트의 어려움	73
8.6 인공지능기반 시스템의 투명성, 해석 가능성, 설명 가능성 테스트	74
8.6.1 실습: 모델 설명 가능성	74
8.7 인공지능기반 시스템의 테스트 오라클	75
8.8 테스트 목적과 인수 기준	76
제9장 인공지능기반 시스템 테스트 방법 및 기법	78
9.1 적대적 공격 및 데이터 오염	80
9.1.1 적대적 공격	80
9.1.2 데이터 오염	80
9.2 페어와이즈 테스트	81
9.2.1 실습: 페어와이즈 테스트	82
9.3 백투백 테스트	82
9.4 A/B 테스트	83
9.5 변성 테스트	83
9.5.1 실습: 변성 테스트	85
9.6 인공지능기반 시스템의 경험기반 테스트	85
9.6.1 실습: 탐색적 테스트와 탐색적 데이터 분석(EDA)	87
9.7 인공지능기반 시스템을 위한 테스트 기법 선택	88
제10장 인공지능기반 시스템의 테스트 환경	89
10.1 인공지능기반의 시스템 테스트 환경	90
10.2 인공지능기반 시스템 테스트를 위한 가상 테스트 환경	90
제11장 테스트에 인공지능 활용하기	92
11.1 테스트를 위한 인공지능 기술	93

11.1.1 실습: 테스트에서 인공지능 활용하기	93
11.2 인공지능을 활용한 보고된 결함 분석	94
11.3 인공지능을 활용한 테스트 케이스 작성	94
11.4 인공지능을 활용한 리그레션 테스트 스위트 최적화	95
11.5 인공지능을 활용한 결함 예측	95
11.5.1 실습: 결함 예측 시스템 구현	96
11.6 인공지능을 활용한 사용자 인터페이스 테스트	96
11.6.1 인공지능을 활용한 그래픽 사용자 인터페이스(GUI)를 통한 테스트	96
11.6.2 인공지능을 활용한 GUI 테스트	97
제12장 참고문헌	98
제13장 부록 A - 약어	104
제14장 부록 B - Abbreviations	105

Acknowledgements

This document was formally released by the General Assembly of the ISTQB® on October 1st, 2021. It was produced by a team from the International Software Testing Qualifications Board: Klaudia Dussa-Zieger (chair), Werner Henschelchen, Vipul Kocher, Qin Liu, Stuart Reid, Kyle Siemens, and Adam Leon Smith. The team thanks the authors of the three contributing syllabi;

- A4Q: Rex Black, Bruno Legeard, Jeremias Röbler, Adam Leon Smith, Stephan Goericke, Werner Henschelchen
- AiU: Main authors Vipul Kocher, Saurabh Bansal, Srinivas Padmanabhuni and Sonika Bengani and co-authors Rik Marselis, José M. Diaz Delgado
- CSTQB/KSTQB: Qin Liu, Stuart Reid

The team thanks the Exam, Glossary and Marketing Working Groups for their support throughout the development of the syllabus, Graham Bath for his technical editing and the Member Boards for their suggestions and input.

The following persons participated in the reviewing and commenting of this syllabus:

Laura Albert, Reto Armuzzi, Árpád Beszédes, Armin Born, Géza Bujdosó, Renzo Cerquozzi, Sudeep Chatterjee, Seunghee Choi, Young-jae Choi, Piet de Roo, Myriam Christener, Jean-Baptiste Crouigneau, Guofu Ding, Erwin Engelsma, Hongfei Fan, Péter Földházi Jr., Tamás Gergely, Ferdinand Gramsamer, Attila Gyúri, Matthias Hamburg, Tobias Horn, Jarosław Hryszko, Beata Karpinska, Joan Killeen, Rik Kochuyt, Thomas Letzkus, Chunhui Li, Haiying Liu, Gary Mogyorodi, Rik Marselis, Imre Mészáros, Tetsu Nagata, Ingvar Nordström, Gábor Péterffy, Tal Pe'er, Ralph Pichler, Nishan Portoyan, Meile Posthuma, Adam Roman, Gerhard Runze, Andrew Rutz, Klaus Skafté, Mike Smith, Payal Sobti, Péter Sótér, Michael Stahl, Chris van Bael, Stephanie van Dijck, Robert Werkhoven, Paul Weymouth, Dong Xin, Ester Zabar, Claude Zhang.

0. 서문 Introduction

0.1 실러버스의 목적

이 실러버스는 ISTQB® Certified Tester AI 테스트(CT-AI)의 기초를 구성합니다. 이 실러버스는 아래와 같은 목적으로 활용될 수 있습니다:

1. 회원국 위원회(Member Boards)는 해당 지역 언어로 번역하고 교육 제공자를 인증한다. 회원국 위원회는 실러버스를 자국의 언어 요구에 맞게 수정하고 지역 출판에 적절하도록 참고자료를 수정할 수 있다.
2. 자격증 기관(Certification Bodies)은 본 교육 과정의 학습 목표에 맞게 현지 언어로 시험 문제를 출제한다.
3. 교육 제공자는 교육 자료를 제작하고 적절한 교육 방법을 결정한다.
4. 시험 응시자는 인증 시험을 준비한다(교육 과정 참가 또는 자체 준비 가능).
5. 국제 소프트웨어 및 시스템 엔지니어링 커뮤니티는 소프트웨어 및 시스템 테스트의 위상 제고를 위해, 또는 관련 도서 및 기사의 기초로 사용할 수 있다.

0.2 AI 테스트 인증 테스터 (CT-AI: Certified Tester AI Testing)

Certified Tester AI 테스트(CT-AI) 학습 및 자격증은 AI 기반 시스템이나 AI를 테스트하는 데 관련된 모든 사람을 대상으로 합니다. 여기에는 테스터, 테스트 분석가, 데이터 분석가, 테스트 엔지니어, 테스트 컨설턴트, 테스트 관리자, 사용자 인수 테스터 및 소프트웨어 개발자 등의 역할을 담당하는 사람들이 포함되며, 프로젝트 관리자, 품질 관리자, 소프트웨어 개발 관리자, 비즈니스 분석가, 운영 팀 구성원, IT 책임자 및 경영 컨설턴트 등 AI 기반 시스템이나 AI에 대한 기본적인 이해가 필요한 모든 사람에게 적합합니다.

The Certified Tester AI Testing Overview [I03]는 다음 정보를 포함하는 별도의 문서입니다:

- 실러버스 내용과 비즈니스 결과 (Business outcomes for the syllabus)
- 비즈니스 결과 매트릭스 및 학습 목표와의 연결 (Matrix of business outcomes and connection with learning objectives)
- 실러버스 요약 (Summary of the syllabus)
- 다른 실러버스들과의 관계 (Relationships among the syllabi)

0.3 출제 가능 학습 목표와 지식 인지 수준

학습 목표는 비즈니스 결과를 지원하며 Certified Tester AI 테스트(CT-AI) 시험 문제 출제에 사용됩니다. 문제를 풀기 위해 시험 응시자들은 열한 개의 장(Chapter) 중에 나와 있는 키워드나 개념을 기억하고 상기하는 것이 필요합니다.

특정 학습 목표 수준은 각 장의 시작 부분에 나와 있으며 다음과 같이 분류됩니다:

- K1: 기억하기 (Remember)
- K2: 이해하기 (Understand)
- K3: 적용하기 (Apply)
- K4: 분석하기 (Analyze)

각 장의 도입 부분에 키워드로 나열된 모든 용어는 학습 목표에 명시적으로 언급하지 않았더라도 기억(K1)해야 합니다.

0.4 역량(Competencies)의 실습 수준

Certified Tester AI 테스트는 실기와 역량에 중점을 둔 실습 목표를 포함합니다.

실습 목표는 다음과 같은 수준이 적용됩니다:

- H0: 녹화된 영상이나 실습의 라이브 데모를 실습한다.
- H1: 실습 가이드 - 학생들은 강사가 수행한 일련의 단계를 따라한다.
- H2: 힌트가 주어진 실습 - 힌트가 들어있는 실습을 제공해 학생들이 주어진 시간 내에 풀 수 있도록 하거나, 학생들이 토론에 참여한다.

다음 목록에 나와 있는 실기 실습을 수행함으로써 역량을 달성합니다:

- 과적합과 과소적합을 시연한다 (H0).
- 기계학습 모델 생성에 필요한 데이터를 준비한다 (H2).
- 훈련 및 테스트 데이터 세트를 식별하고 기계학습 모델을 생성한다 (H2).
- 선택된 기계학습 기능적 성능 측정지표를 사용해 만든 기계학습 모델을 평가한다 (H2).
- 퍼셉트론(perceptron) 구현을 경험한다 (H1).
- 테스터가 설명 가능성(explainability)을 어떻게 활용할 수 있는지 보여주는 도구를 사용한다 (H2).
- 페어와이즈 테스트를 적용해 인공지능기반 시스템의 테스트 케이스를 도출하고 실행한다 (H2).

- 변성 테스트링(metamorphic testing)을 적용해 주어진 시나리오에 대한 테스트 케이스를 도출하고 실행한다 (H2).
- 인공지능기반 시스템에 탐색적 테스트를 적용한다 (H2).
- 인공지능 활용 가능성이 낮은 테스트 활동을 예제를 들어가며 논의한다 (H2).
- 간단한 인공지능기반 결함 예측 시스템을 구현한다 (H2).

0.5 The Certified Tester AI Testing 시험

Certified Tester AI 테스트 시험은 이 실러버스를 기반으로 합니다. 시험 문제의 답은 본 실러버스 중 하나 이상의 섹션에 기초한 자료를 사용해야 할 수 있습니다. 서문과 부록을 제외한 실러버스의 모든 섹션에서 문제가 출제될 수 있습니다. 표준과 도서는 참고문헌으로 포함되어 있지만, 이런 표준과 도서에서 참고해 실러버스에 요약되어 있는 내용 외에는 출제되지 않습니다.

응시자격 요구사항(응시 전제 조건): AI 테스트 시험에 응시하기 전에 ISTQB® 파운데이션 레벨(CTFL) 자격증을 취득해야 합니다. 시험 신청시 CTFL 자격증 제출이 필요합니다.

0.6 인증 (Accreditation)

ISTQB® 회원국 위원회는 해당 실러버스의 교육 제공자를 인증할 수 있습니다. 교육 제공자는 이 실러버스를 준수해 제작한 교재를 먼저 인증받아야 하며, 인증을 수행하는 회원국 위원회로부터 인증 가이드라인을 받아야 합니다. 따라서 인증 교육기관의 교육 과정은 본 실러버스를 준수하는 것으로 인정되며, 과정의 일부로 ISTQB® 시험을 치를 수 있습니다.

이 실러버스에 대한 인증 가이드라인은 프로세스 관리와 컴플라이언스 워킹그룹에서 발행한 일반 인증 가이드라인을 따릅니다.

0.7 상세 수준

이 실러버스의 상세 수준은 국제적으로 일관된 교육과 시험을 가능하게 합니다. 이 목표 달성을 위해 실러버스는 다음과 같이 구성됩니다:

- Certified Tester AI 테스트의 의도를 설명하는 일반적인 지침 목표
- 학생들이 기억해야 하는 용어 목록

- 달성해야 할 학습 결과를 설명하는 각 지식 영역에 대한 학습 및 실습 목표
- 승인된 문헌이나 표준 등의 출처에 대한 참조를 포함하는 주요 개념에 대한 설명

실러버스 내용은 AI 기반 시스템 테스트를 위한 전체 지식 영역을 설명하는 것이 아니며, Certified Tester AI 테스트 인증 교육 과정에서 다루어야 할 세부 사항 수준을 반영하고 있다. 특히 인공지능(AI)과 머신러닝의 기본 개념을 소개하고, 이런 기술을 기반으로 하는 시스템의 테스트 방법을 중점적으로 다루고 있습니다.

0.8 실러버스 구성 방법

11개의 장(Chapter)에서 시험문제가 출제될 수 있습니다. 각 장의 상위 표제 오른쪽에 각 장에 지정된 시간이 나타나 있고, 이후 하위 표제에선 제공하지 않습니다. 인증 교육 과정의 경우, 실러버스는 아래 11개의 장으로 구분되어 최소 25.1 시간의 교육이 요구됩니다:

- Chapter 1: 105 분 AI 소개
- Chapter 2: 105 분 인공지능기반 시스템 품질 특성
- Chapter 3: 145 분 기계학습 (ML) - 개요
- Chapter 4: 230 분 기계학습 - 데이터
- Chapter 5: 120 분 기계학습 기능적 성능 측정지표
- Chapter 6: 65 분 기계학습 - 신경망과 테스트
- Chapter 7: 115 분 인공지능기반 시스템 테스트 개요
- Chapter 8: 150 분 인공지능 특유 품질 특성 테스트
- Chapter 9: 245 분 인공지능기반 시스템 테스트 방법과 기법
- Chapter 10: 30 분 인공지능기반 시스템의 테스트 환경
- Chapter 11: 195 분 테스트에 인공지능 활용하기

제1장 인공지능 소개 Introduction to AI

105분

Testing Keywords

None

AI-Specific Keywords

AI as a Service (AlaaS), AI development framework, AI effect, AI-based system, artificial intelligence (AI), neural network, deep learning (DL), deep neural network, general AI, General Data Protection Regulation (GDPR), machine learning (ML), narrow AI, pre-trained model, super AI, technological singularity, transfer learning

학습 목표:

1.1 인공지능의 정의와 인공지능 효과

AI-1.1.1 (K2) 인공지능 효과와 그것이 인공지능 정의에 미치는 영향을 설명할 수 있다.

1.2 약인공지능, 강인공지능, 초인공지능

AI-1.2.1 (K2) 약인공지능, 강인공지능, 초인공지능을 구별할 수 있다.

1.3 인공지능기반 시스템과 전통적 시스템

AI-1.3.1 (K2) 인공지능기반 시스템과 전통적 시스템을 구분할 수 있다.

1.4 인공지능 기술

AI-1.4.1 (K1) 인공지능 구현에 사용하는 다양한 기술을 식별할 수 있다.

1.5 개발 프레임워크

AI-1.5.1 (K1) 대중적인 인공지능 개발 프레임워크를 식별할 수 있다.

1.6 인공지능기반 시스템을 위한 하드웨어

AI-1.6.1 (K2) 인공지능기반 시스템 구현을 위해 선택 가능한 하드웨어를 비교할 수 있다.

1.7 서비스형 인공지능 (AlaaS, AI as a Service)

AI-1.7.1 (K2) 서비스형 인공지능 개념을 설명할 수 있다.

1.8 사전 훈련 모델

AI-1.8.1 (K2) 사전 훈련 인공지능 모델의 사용과 그와 관련된 리스크를 설명할 수 있다.

1.9 표준, 규정, 인공지능

AI-1.9.1 (K2) 인공지능기반 시스템에 표준이 어떻게 적용되는지 설명할 수 있다.

1.1 인공지능의 정의와 인공지능 효과 Definition of AI and AI Effect

인공지능(AI)이라는 용어는 1950년대 처음 사용되었으며 인간을 모방할 수 있는 “지능형” 기계를 만들고 프로그래밍하는 것을 목적으로 했다. 많이 변화된 현재의 정의는 다음과 같다[S01]:

엔지니어링 된 시스템이 지식과 기술을 습득, 처리, 적용할 수 있는 능력

사람들의 인공지능에 대한 이해는 시대에 따라 다르다. 1970년대에는 체스에서 인간을 이길 수 있는 컴퓨터 시스템은 먼 미래의 얘기였고 사람들은 이런 시스템을 인공지능이라고 생각했다. 컴퓨터 기반 시스템인 딥 블루(Deep Blue)가 세계 체스 챔피언 가리 카스파로프(Garry Kasparov)를 이긴 지 20년이 지난 지금, 대부분의 사람들은 해당 시스템이 사용한 “무작위 대입(brute force)” 접근법을 진정한 인공지능으로 생각하지 않는다 (시스템이 데이터로 학습되지 않았고 스스로 학습할 수 없기 때문임). 유사하게, 1970년대와 1980년대의 전문가 시스템은 전문가의 관여 없이도 반복적으로 실행될 수 있는 규칙을 기반으로 인간의 전문지식을 구현했다. 이런 시스템을 당시에는 인공지능으로 여겼지만 지금은 그렇지 않다.

어떤 요소를 가져야 인공지능인가에 대한 인식의 변화를 “인공지능 효과”[R01]라고 한다. 사회에서 인공지능에 대한 인식이 변화함에 따라 그 정의도 변화하고 있다. 결과적으로, 오늘날 정의는 향후 바뀔 가능성이 있으며 과거의 정의와 일치하지 않을 수도 있다.

1.2 약인공지능, 강인공지능, 초인공지능 Narrow, General and Super AI

인공지능은 크게 세 가지 범주로 나눌 수 있다.

- 약인공지능 시스템은 제한된 환경에서 특정 작업을 수행하도록 프로그래밍 되어있다. 현재 이런 형태의 인공지능이 널리 사용되고 있다. 예를 들어, 게임 플레이 시스템, 스팸 필터, 테스트 케이스 생성기, 음성 도우미 등이 여기에 해당한다.
- 강인공지능 시스템은 인간과 유사한 일반적인(광범위한) 인지 능력을 갖추고 있다. 이런 인공지능 기반 시스템은 인간처럼 환경을 추론하고 이해할 수 있으며 그에 따라 행동할 수 있다. 2021년 말 현재 강인공지능 시스템은 아직 구현되지 않았다.
- 초인공지능 시스템은 인간의 인지(강인공지능)를 복제할 수 있고, 방대한 처리 능력, 사실상 무제한의 기억 능력 그리고 모든 인간의 지식을 사용할 수 있다(예: 웹에 접속해서). 초인공지능 시스템은 빠른 시일 내에 인간보다 현명해질 것으로 여겨진다. 인공지능 기반 시스템이 강인공지능에서 초인공지능으로 전환되는 시점을 일반적으로 기술적 특이점(technological singularity)[B01]이라고 한다.

1.3 인공지능기반 시스템과 전통적 시스템 AI-Based and Conventional Systems

일반적인 전통 컴퓨터 시스템에서 소프트웨어는 사람이 if-then-else 및 루프(loop)와 같은 구문을 포함한 명령형 언어를 사용해 프로그래밍하게 된다. 시스템이 입력을 출력으로 변환하는 방법을 사람이 비교적 쉽게 이해할 수 있다. 기계학습(ML)을 사용하는 인공지능기반 시스템은 데이터의 패턴을 사용해 미래의 새로운 데이터에 어떻게 반응할지 결정하게 된다.(기계학습에 대한 자세한 설명은 3장 참조). 예를 들어, 고양이의 이미지를 식별하도록 설계한 인공지능기반 이미지 처리 시스템은 고양이가 포함된 것으로 알려진 일련의 이미지로 훈련을 한다. 그런 다음 이런 패턴과 규칙을 새로운 이미지에 적용해 여기에 고양이가 있는지 인공지능 스스로 판단한다. 대부분 인공지능기반 시스템의 이런 예측 절차는 인간이 이해하기 쉽지 않다(2.7절 참조).

실제로 인공지능기반 시스템은 다양한 기술로 구현할 수 있으며(1.4절 참조), “인공지능 효과”(1.1절 참조)로 오늘날 인공지능기반 시스템으로 간주하는 것과 전통적 시스템으로 간주하는 것의 구분이 가능하다.

1.4 인공지능 기술 AI Technologies

인공지능은 다음과 같은 다양한 기술을 사용해 구현할 수 있다(자세한 내용은 [B02] 참조).

- 퍼지 논리 (Fuzzy logic)
- 검색 알고리즘 (Search algorithms)
- 추론 기법 (Reasoning techniques)
 - 규칙 엔진 (Rule engines)
 - 연역적 분류기 (Deductive classifiers)
 - 사례 기반 추론 (Case-based reasoning)
 - 절차적 추론 (Procedural reasoning)
- 기계학습 (Machine learning techniques)
 - 신경망 (Neural networks)
 - 베이저안 모델 (Bayesian models)
 - 결정 트리 (Decision trees)
 - 랜덤 포레스트 (Random forest)
 - 선형 회귀 (Linear regression)
 - 로지스틱 회귀 (Logistic regression)

- 클러스터링 알고리즘 (Clustering algorithms)
- 유전 알고리즘 (Genetic algorithms)
- 서포트 벡터 머신 (SVM, Support vector machine)

인공지능기반 시스템은 일반적으로 이런 기술 중 하나 이상의 기술로 구현한다.

1.5 인공지능 개발 프레임워크 AI Development Frameworks

사용 가능한 다양한 인공지능 개발 프레임워크가 있으며, 그중 일부는 특정 도메인에 초점을 맞추고 있다. 이런 프레임워크는 중앙 처리 장치(CPU), 그래픽 처리 장치(GPU) 또는 클라우드 텐서 처리 장치(TPU)와 같은 다양한 프로세서에서 실행된다. 또한, 데이터 준비, 알고리즘 선택, 모델 컴파일과 같은 다양한 활동을 지원한다. 프레임워크의 선택은 구현에 사용되는 프로그래밍 언어, 사용 용이성 등에 따라 달라질 수 있다. 다음은 현재 가장 널리 사용되고 있는 프레임워크 중 일부이다(2021년 4월 기준).

- Apache MxNet : 아마존 웹 서비스(AWS)를 위해 아마존에서 사용하는 딥러닝 오픈소스 프레임워크 [R02]
- CNTK : 오픈소스 딥러닝 툴킷인 Microsoft Cognitive Toolkit [R03]
- IBM Watson Studio : 인공지능 솔루션 개발을 지원하는 도구 모음 [R04]
- Keras : 파이썬 언어로 작성된 오픈소스 상위 수준 API로 TensorFlow와 CNTK와 함께 사용 가능 [R06]
- PyTorch : Facebook에서 운영하는 오픈소스 기계학습 라이브러리로 이미지 처리 및 자연어 처리(NLP)를 적용하는 애플리케이션에 사용. 파이썬 및 C++ 인터페이스 모두 지원 [R07]
- Scikit-learn : 파이썬 프로그래밍 언어를 위한 오픈소스 기계학습 라이브러리 [R08]
- TensorFlow : 구글에서 제공하는 확장 가능 데이터 흐름 그래프 기반 오픈소스 기계학습 프레임워크 [R05]

이런 개발 프레임워크는 끊임없이 발전하고 있으며 서로 결합되거나 새로운 프레임워크로 대체되기도 한다.

1.6 인공지능기반 시스템을 위한 하드웨어 Hardware for AI-Based Systems

기계학습 모델 훈련(3장 참조)과 모델 구현에는 다양한 하드웨어가 사용된다. 예를 들어, 음성 인식을 수행하는 모델의 실행은 보급형 스마트폰에서 가능하지만, 훈련을 위해서는 클라우드 컴퓨팅을 이용할 수 있다. 호스

트 장치가 인터넷에 연결되어 있지 않을 때 사용하는 일반적인 방법은 클라우드에서 모델을 훈련한 다음 호스트 장치에 배포하는 것이다.

기계학습에는 일반적으로 다음과 같은 특성을 지원하는 하드웨어가 필요하다:

- 낮은 정밀도 산술(Low-precision arithmetic)이 가능한 하드웨어: 이런 하드웨어는 계산에 더 적은 비트 (예: 32비트가 아닌 8비트. 일반적으로 ML은 8비트만 있으면 된다)를 사용한다.
- 대규모 데이터 구조(예: 행렬 곱셈 지원) 처리 능력이 있는 하드웨어
- 대규모 병렬(동시) 처리가 가능한 하드웨어

범용 CPU는 일반적으로 기계학습 애플리케이션에 불필요한 복잡한 작업을 지원하며 제한된 몇 개의 코어만 가지고 있다. 결과적으로, 범용 CPU의 아키텍처는 GPU에 비해 기계학습 모델을 훈련하고 실행하는데 효율적이지 못하다. GPU는 보통 수천 개의 코어를 가지며 이미지 처리와 같은 대규모 병렬 처리가 필요하지만, 상대적으로 간단한 작업을 하도록 설계된다. 결국 CPU의 클럭(clock)이 더 빠르지만 기계학습 애플리케이션에는 GPU가 상대적으로 우수한 성능을 보이게 된다. 보통 소규모 기계학습 작업에서는 GPU가 최상의 옵션이 된다.

ASIC(Application-Specific integrated Circuit), SoC(System on a Chip) 등과 같이 인공지능에 사용하도록 설계된 하드웨어도 있다. 이런 인공지능 전용 솔루션은 다수의 코어, 특별한 데이터 관리, 인-메모리(in-memory) 처리 등의 기능을 갖추고 있다. 이런 솔루션은 에지 컴퓨팅(edge computing)에 가장 적합하며 기계학습 모델 훈련은 클라우드에서 수행된다.

현재 개발되고 있는 특정 기능을 위한 인공지능 하드웨어도 있다(2021년 4월 기준). 예를 들어, 전통적인 폰 노이만(von Neumann) 아키텍처를 사용하지 않고, 뇌신경을 비슷하게 모방한 아키텍처를 사용하는 신경망 프로세서[B03]도 개발되고 있다.

대표적인 인공지능 하드웨어 제공업체와 프로세서에는 다음과 같다(2021년 4월 기준):

- NVIDIA : 다양한 GPU와 Volta[R09]와 같은 인공지능 전용 프로세서를 제공한다.
- Google : 훈련 및 추론을 위한 애플리케이션 별 집적 회로를 개발한다. Google TPUs (Cloud Tensor Processing Units) [R10]는 구글 클라우드에서 사용자가 사용할 수 있는 반면, Edge TPU[R11]는 개별 기기에서 인공지능이 실행되도록 설계된 전용 ASIC이다.
- Intel : 딥러닝(훈련 및 추론 모두)을 위한 Nervana 신경망 프로세서[R12]와 컴퓨터 비전 및 신경망 애플리케이션의 추론을 위한 Movidius Myriad 비전 처리 장치를 제공한다.
- Mobileye : 복잡하고 연산 집약적인 비전 처리를 지원하는 EyeQ SoC 장치 제품군[R13]을 생산한다. 이들은 차량에 사용하기 위해 전력 소비량이 낮다.

- Apple : 아이폰에 탑재되는 온-디바이스(on-device) 인공지능 바이오닉 칩을 생산한다[B04].
- Huawei : 스마트폰용 Kirin 970 칩에는 인공지능 신경망 처리가 내장되어 있다[B05].

1.7 서비스형 인공지능 AI as a Service (AlaaS)

기계학습 모델과 같은 인공지능 컴포넌트는 조직 내에서 개발하거나, 외부에서 다운로드하거나, 웹 서비스 (AlaaS)로 사용할 수 있다. 인공지능 기능의 일부는 시스템 내에서 제공하고, 일부는 서비스로 사용하는 하이브리드 접근 방식도 가능하다.

기계학습을 서비스로 사용하는 경우, 웹을 통해 기계학습 모델에 접근할 수 있으며 데이터 준비와 저장, 모델 훈련, 검증, 튜닝(tuning), 테스트, 배포에 대한 지원도 받을 수 있다.

제3자 공급자(예: AWS, 마이크로소프트)는 얼굴 및 음성 인식과 같은 인공지능 서비스를 제공한다. 이를 통해 개인과 조직은 자체 인공지능 서비스를 구축할 자원과 전문성이 부족한 경우에도 클라우드 기반 서비스를 사용해 인공지능을 구현할 수 있다. 또한, 제3자 서비스로 제공되는 기계학습 모델은 대부분 이해관계자가 쉽게 구할 수 있는 것보다 더 많고 더 다양한 훈련 데이터 세트로 훈련됐을 가능성이 높다. 비교적 최근에 인공지능 분야에 진출한 기업에게도 이런 점이 적용될 수 있다.

1.7.1 서비스형 인공지능 계약

이런 인공지능 서비스는 일반적으로 비 인공지능 클라우드 기반 서비스형 소프트웨어(SaaS)와 유사한 계약을 통해 제공된다. 서비스형 인공지능 계약은 일반적으로 가용성과 보안 책무를 정의하는 서비스 수준 계약(SLA)을 포함한다. 이런 SLA는 보통 서비스 가동 시간(예: 99.99% 가동 시간)과 결함 수정을 위한 응답 시간을 포함하지만, 기계학습의 기능적 성능 메트릭(예: 정확도)을 정의하는 경우는 거의 없다(5장 참조). 서비스형 인공지능은 구독 형태로 요금을 지불하는 경우가 많으며, 계약된 가용성이나 응답 시간이 충족되지 않으면 서비스 제공업체는 보통 향후에 사용 가능한 서비스 크레딧(credit)을 제공하게 된다. 이런 크레딧 외에는 대부분의 서비스형 인공지능 계약은 제한된 책임(지불된 구독료 제외)을 제공한다. 즉, 서비스형 인공지능에 의존하는 인공지능 기반 시스템은 일반적으로 서비스 손실이 크지 않은 상대적으로 리스크가 낮은 애플리케이션으로 제한된다.

대다수의 서비스는 인수 기간 대신 무료 평가 기간을 제공한다. 이 기간 동안 서비스형 인공지능의 소비자는 제공된 서비스가 필요한 기능 및 성능(예: 정확도) 측면에서 요구사항을 충족하는지 테스트해야 한다. 이런 과정은 제공 서비스에 대한 설명이 부족한 부분을 해결하는 데 필요하다 (7.5 참조).

1.7.2 서비스형 인공지능 사례

다음은 서비스형 인공지능 사례들이다(2021년 4월 기준).

- IBM Watson Assistant : 월간 활성 이용자(active user) 수에 따라 가격이 책정되는 인공지능 챗봇.
- Google Cloud 인공지능과 ML 제품 : 양식 분석과 문서 OCR을 포함한 문서 기반 인공지능 제공. 가격은 처리한 페이지 수를 기준으로 책정.
- Amazon Code Guru : 개발자에게 코드 품질 개선을 위한 권장 사항을 제공하는 ML JAVA 코드 리뷰 제공. 가격은 분석된 소스 코드 라인 수를 기준으로 결정.
- Microsoft Azure Cognitive Search : 인공지능 클라우드 검색 제공. 검색 단위(사용 처리량과 스토리지로 정의)를 기준으로 가격 결정.

1.8 사전 훈련 모델 Pre-Trained Models

1.8.1 사전 훈련 모델

기계학습 모델을 훈련하는 데 많은 비용이 소요될 수 있다(3장 참조). 먼저 데이터를 준비하고 모델을 훈련해야 한다. 전자는 많은 인적 자원을 소비할 수 있고 후자는 많은 컴퓨팅 자원을 소비할 수 있으며, 많은 조직이 이런 자원을 가지고 있지 않다.

더 저렴하고 때에 따라 더 효과적이기도 한 대안은 사전 훈련된 모델을 사용하는 것이다. 사전 훈련 모델의 기능을 확장 또는 집중하는 새로운 모델을 생성하기 위한 기초로 필요한 모델과 유사한 기능을 제공하는 사전 훈련 모델을 사용한다. 이런 모델은 신경망과 랜덤 포레스트 등 몇 가지 기법에 대해서만 존재한다.

이미지 분류기가 필요한 경우, 1000개 이상의 카테고리로 분류되고 1,400만 개 이상의 이미지를 가지고 있는 공개 데이터 세트인 ImageNet 데이터 세트로 훈련할 수 있다. 이렇게 하면 상당한 자원을 소모하거나 실패하는 리스크를 줄일 수 있다. 또는 이 데이터 세트로 이미 훈련된 모델을 재사용할 수도 있다. 사전에 훈련된 모델을 사용함으로써 훈련 비용을 줄이고 실패에 대한 리스크를 크게 줄일 수 있다.

사전 훈련된 모델을 수정 없이 사용할 경우, 간단하게 인공지능기반 시스템에 통합하거나 서비스 형태로 사용할 수도 있다(1.7 참조).

1.8.2 전이 학습

사전 훈련 모델을 가져와 다른 요구사항을 수행하도록 수정할 수 있다. 이것을 전이 학습이라 한다. 딥러닝 신경망은 초기 층(early layers, 6장 참조)이 일반적으로 매우 기본적인 작업(예: 이미지 분류기에서 직선과 곡선

의 차이 식별)을 수행하는 반면, 후기 층(later layers)은 보다 구체적인 작업(예: 건축물 유형 구분)을 수행한다. 이런 경우 이미지 분류기의 후기 층을 제외한 모든 층을 재사용할 수 있으므로 초기 층을 다시 훈련할 필요가 없다. 후기 층은 새로운 분류기의 고유한 요구사항을 처리하도록 다시 훈련시킨다. 실무에서는 사전 훈련된 모델을 새로운 문제 관련 데이터를 가지고 추가 훈련하는 과정을 통해 튜닝하게 된다.

이런 접근법의 효과는 원래 모델이 수행하던 기능과 새로운 모델이 요구하는 기능 간의 유사성 정도에 크게 좌우된다. 예를 들어, 고양이 품종을 식별하는 이미지 분류기를 개의 품종을 식별하도록 수정하는 것이, 이 분류기를 사람들의 억양 식별에 사용하도록 수정하는 것보다 훨씬 효과적일 것이다.

사전 훈련된 모델은 많이 있다. 특히, 학계에서 배포하는 것들이 많다. 예를 들어, 이미지 분류를 위한 Inception, VGG, AlexNet, MobileNet과 같은 ImageNet 모델[R14]과 Google BERT[R15]와 같은 사전 훈련된 NLP 모델이 있다.

1.8.3 사전 훈련 모델과 전이 학습 사용 리스크

사전 훈련 모델과 전이 학습을 사용하는 것은 모두 인공지능기반 시스템을 구축하는 일반적인 접근법이지만, 여기에는 다음과 같은 몇 가지 리스크가 있다:

- 사전 훈련 모델은 내부적으로 제작한 모델에 비해 투명성이 부족할 수 있다.
- 사전 훈련 모델이 수행하는 기능과 필요한 기능 간의 유사도가 충분하지 않을 수 있다. 또한, 데이터 과학자가 그 차이를 이해하지 못할 수도 있다.
- 사전 훈련 모델의 초기 개발 시 사용한 데이터 준비 절차(4.1 참조)와 이 모델을 새로운 시스템에 적용할 때 사용한 데이터 준비 절차의 차이가 기능적 성능에 영향을 미칠 수 있다.
- 사전 훈련 모델의 단점은 그것을 사용하는 사람들에게 승계될 가능성이 높으며 문서화되지 않았을 수 있다. 예를 들어, 모델 훈련에 사용한 데이터에 대한 기록이 부족하면 상속된 편향성(2.4 참조)이 명확하지 않을 수 있다. 또한, 사전 훈련 모델이 널리 사용되지 않았을 경우, 알려지지 않은 (또는 문서화되지 않은) 결함이 더 많을 수 있으며, 이런 리스크를 완화하기 위해 엄격한 테스트가 필요할 수 있다.
- 전이 학습을 통해 만들어진 모델은 그 기반이 되는 사전 훈련 모델과 동일한 취약성(예, 9.1.1에서 설명하는 적대적 공격)에 민감할 가능성이 높다. 또한, 인공지능기반 시스템이 특정 사전 훈련 모델을 포함한다고 알려진 경우 (또는 특정 사전 훈련 모델을 기반으로 하는 경우), 잠재적 공격자가 이미 관련 취약성을 알고 있을 수 있다.
- 위의 리스크 중 다수는 사전 훈련 모델에 대한 상세 문서의 제공을 통해 쉽게 완화할 수 있다는 점을 인지할 필요가 있다(7.5 참조).

1.9 표준, 규정, 인공지능 Standards, Regulations and AI

IEC와 ISO의 정보 기술 분야 공동 기술 위원회(ISO/IEC JTC 1)는 인공지능에 필요한 국제 표준을 개발하고 있다. 예를 들어, 인공지능 소위원회(ISO/IEC JTC 1/SC42)가 2017년에 설치되었으며, 소프트웨어 및 시스템 엔지니어링을 다루는 ISO/IEC JTC 1/SC7에서는 "인공지능기반 시스템 테스트"[S01]에 대한 기술 보고서를 발간했다.

인공지능에 대한 표준은 지역 수준(예: 유럽 표준)과 국가 수준에서도 발표되고 있다.

EU 전역에 적용되는 일반 개인정보 보호법(GDPR, General Data Protection Regulation)은 2018년 5월에 발효되었으며 데이터 취급자가 개인정보 및 자동화된 의사결정에 대해 지켜야 하는 의무를 정의하고 있다[B06]. 여기에는 잠재적 차별 완화를 포함하는 인공지능 시스템의 기능적 성능을 평가 및 개선하고, 개인의 권리가 자동화된 의사결정의 대상이 되지 않도록 보장하기 위한 요건이 포함되어 있다. 테스트 관점에서 GDPR이 가지는 가장 중요한 의미는 개인정보(예측 포함)가 정확해야 한다는 것이다. 이것은 시스템이 하는 모든 예측이 정확해야 한다는 것을 의미하는 것이 아니라 시스템이 사용 목적을 충족하기 위한 정확도를 가져야 한다는 것을 의미한다.

독일 국가 표준 기구(DIN)도 인공지능 품질 메타모델([S02], [S03])을 개발했다.

관련 산업계 기관에서도 인공지능에 대한 표준을 발표하고 있다. 예를 들어, 전기 전자 기술자 협회(IEEE)는 윤리와 인공지능에 대한 다양한 표준(인공지능과 자율 시스템의 윤리 강령에 관한 IEEE 국제 지침)을 개발하고 있다. 이런 표준 중 상당수는 이 실러버스를 작성한 시점에도 여전히 개발 중에 있다.

인공지능이 안전 관련 시스템에 사용되는 경우 관련 규제 표준이 적용된다. 자동차 시스템에 대한 ISO 26262 [S04], ISO/PAS 21448 (SOTIF) [S05] 등이 이런 예가 될 수 있다. 이런 규제 표준은 일반적으로 정부 기관에 의해 의무화되며, 포함된 소프트웨어가 ISO 26262를 준수하지 않을 경우 일부 국가에서 해당 소프트웨어가 포함된 자동차를 판매하는 것은 불법으로 간주된다. 표준은 그 자체로는 자발적인 문서이며, 그 사용의 의무화는 법률이나 계약에 의해 이루어진다. 그러나 많은 사용자는 표준의 저자가 가진 전문지식을 활용하고 높은 품질의 제품을 만들기 위해 표준을 사용하게 된다.

제2장 인공지능기반 시스템 품질 특성

Quality Characteristics for AI-Based Systems

105분

Keywords

None

AI-Specific Keywords

Adaptability, algorithmic bias, autonomy, bias, evolution, explainability, explainable AI (XAI), flexibility, inappropriate bias, interpretability, ML system, machine learning, reward hacking, robustness, sample bias, self-learning system, side effects, transparency

학습 목표

2.1 유연성과 적응성

AI-2.1.1 (K2) 인공지능기반 시스템의 특성으로 유연성과 적응성의 중요성을 설명할 수 있다.

2.2 자율성

AI-2.2.1 (K2) 인공지능기반 시스템과 자율성의 관계를 설명할 수 있다.

2.3 진화

AI-2.3.1 (K2) 인공지능기반 시스템에 대한 진화 관리의 중요성을 설명할 수 있다.

2.4 편향성

AI-2.4.1 (K2) 인공지능기반 시스템에서 나타나는 편향성의 다양한 원인과 유형을 설명할 수 있다.

2.5 윤리

AI-2.5.1 (K2) 인공지능기반 시스템의 개발, 배포, 사용에 있어 존중되어야 하는 윤리 원칙을 논의할 수 있다.

2.6 부작용과 보상 해킹

AI-2.6.1 (K2) 인공지능기반 시스템에서 나타나는 부작용과 보상 해킹을 설명할 수 있다.

2.7 투명성, 해석 가능성, 설명 가능성

AI-2.7.1 (K2) 투명성, 해석 가능성, 설명 가능성이 인공지능기반 시스템에 어떻게 적용되는지 설명할 수 있다.

2.8 안전과 인공지능

AI-2.8.1 (K1) 안전 관련 애플리케이션에 인공지능기반 시스템을 사용하기 어렵게 만드는 특성을 상기할 수 있다.

2.1 유연성과 적응성 Flexibility and Adaptability

유연성과 적응성은 밀접하게 연관된 품질 특성이다. 이 실러버스에서 유연성은 시스템이 본래의 시스템 요구사항의 일부가 아닌 상황에서 얼마만큼 사용될 수 있는가 하는 정도이고, 적응성은 의도와는 다른 하드웨어 나 변화하는 운영 환경 등의 새로운 상황에 맞게 제품을 수정하려고 할 때의 용이성을 말한다.

유연성과 적응성 모두 다음과 같은 상황에 유용하다:

- 시스템 배포 시 운영 환경을 완전히 알 수 없는 경우.
- 시스템이 새로운 운영 환경에 대처할거라고 기대하는 경우.
- 시스템이 새로운 상황에 적응할거라고 기대하는 경우.
- 시스템이 언제 동작을 수정해야 하는지 결정해야 하는 경우.

자가학습을 하는 인공지능기반 시스템은 위의 모든 특성을 보여줄 것이다. 따라서 이들은 적응성이 있어야 하고 유연성을 보여줄 잠재력을 가지고 있어야 한다.

인공지능기반 시스템의 유연성과 적응성 요구사항에는 시스템이 적응해야 할 환경 변화에 대한 세부 사항이 포함되어야 한다. 또한, 이런 요구사항은 시스템이 스스로를 적응시키는 데 사용할 수 있는 시간과 자원에 대한 제약 조건(예: 새로운 유형의 개체 인식에 시스템이 적응하는 데 사용할 수 있는 최대 시간)을 명시해야 한다.

2.2 자율성 Automomy

자율성을 정의할 때, 먼저 완전 자율 시스템은 사람의 감독과 통제로부터 완전히 독립적이라는 것을 먼저 인식하는 것이 중요하다. 실제로, 원하는 것이 완전한 자율성은 아닌 경우가 많다. 예를 들어, 일반적으로 “자율주행”이라고 불리는 완전 자율주행 자동차는 공식적으로 “완전 주행 자동화”로 분류된다[B07].

많은 사람들이 자율 시스템을 “스마트” 또는 “지능적”이라고 생각한다. 결국 이런 시스템에는 일부 기능을 수행하기 위한 인공지능기반 컴포넌트(component)가 포함되어 있을 것으로 생각할 수 있다. 예를 들어, 상황 인식이 필요한 자율주행 차량은 일반적으로 여러 센서 및 이미지 처리 장치를 사용하여 차량의 인접 환경에 대한 정보를 수집한다. 기계학습, 특히 딥러닝(6.1 참조)이 이 기능을 수행하는 데 가장 효과적인 방법으로 밝혀졌다. 자율 시스템에는 의사 결정 및 제어 기능도 포함될 수 있다. 이 두 가지 모두 인공지능기반 컴포넌트를 사용하여 효과적으로 수행할 수 있다.

인공지능기반 시스템 중 일부는 자율 시스템에 해당하지만 모든 인공지능기반 시스템이 자율 시스템인 것은 아니다. 이 실러버스에서 자율성이란 시스템이 사람의 감독과 통제로부터 독립적으로 일정 기간 동작할 수 있는 능력을 말한다. 이것은 명세화되고 테스트 되어야 하는 자율 시스템의 특성을 식별하는 데 도움이 될 수 있다. 예를 들어, 자율 시스템이 사람의 개입 없이 만족스럽게 동작할 것으로 기대되는 시간을 알아야 할 필요가 있다. 자율 시스템이 사람에게 제어권을 돌려줘야 하는 이벤트를 식별하는 것도 중요하다.

2.3 진화 Evolution

이 실러버스에서 진화는 변화하는 외부 제약에 대응해 스스로 개선하는 시스템의 능력을 말한다. 일부 인공지능 시스템은 자가학습 시스템으로 분류되며 성공적인 자가학습 인공지능기반 시스템은 이런 진화가 필요하다.

인공지능기반 시스템은 진화하는 환경에서 동작하는 경우가 많다. 다른 형태의 IT 시스템과 마찬가지로 인공지능기반 시스템도 운영 환경의 변화에 충분히 대응할 수 있도록 유연하고 적응력이 있어야 한다.

자가학습 인공지능기반 시스템은 일반적으로 두 가지 형태의 변화를 관리해야 한다.

- 변화의 한 형태는 시스템이 자신의 결정 및 주변환경과의 상호작용에서 학습하는 경우.
- 변화의 또 다른 형태는 시스템이 시스템 운영 환경에 가해진 변경 사항에서 학습하는 경우.

두 경우 모두 시스템이 자신의 효과성과 효율성을 향상시키기 위해 진화하는 것이 이상적이다. 그러나 이런 진화 과정에서 원하지 않는 시스템 특성이 나타나지 않도록 통제가 필요하다. 모든 진화는 원래의 시스템 요구 사항과 제약 조건을 지속적으로 충족해야 한다. 이런 명세가 부족한 경우라 할지라도 시스템의 모든 진화는 범위를 벗어나지 않고 항상 인간이 가진 가치에 부합하도록 관리되어야 한다. 2.6절에서 부작용과 보상 해킹이 자가학습 인공지능기반 시스템에 미치는 영향 관련 예제를 제공한다.

2.4 편향성 Bias

인공지능기반 시스템에서 편향성이란 시스템이 제공하는 결과와 “공정한 결과”로 간주되는 값 간의 거리차이를 나타내는 통계적 수치이다. 공정한 결과는 특정 그룹에 대한 편애를 나타내지 않는 결과를 뜻한다. 부적절한 편견으로는 성별, 인종, 민족, 성적 지향, 소득 수준, 나이와 같은 속성을 들 수 있다. 인공지능기반 시스템의 부적절한 편향 사례는 예를 들어, 은행 대출 심사 시스템, 채용 시스템, 사법 모니터링 시스템 등에서 보고되었다.

편향은 여러 유형의 인공지능기반 시스템에서 나타날 수 있다. 가령, 전문가 시스템에 적용되는 규칙에 전문가의 편향성이 들어가지 않도록 하는 것은 어렵다. 그러나 기계학습 시스템의 확산으로 인해 편향성과 관련된 대부분의 논의가 이런 시스템에 대해 발생하고 있다.

기계학습 시스템은 수집된 데이터를 사용하는 알고리즘을 가지고 의사 결정과 예측을 하는 데 사용되며, 이 두 가지 컴포넌트는 결과에 편향성을 일으킬 수 있다:

- 학습 알고리즘이 잘못 구성된 경우(예: 일부 데이터를 다른 데이터에 비해 과대평가하는 경우 등) 알고리즘 편향성이 발생할 수 있다. 이렇게 발생한 유형의 편향성은 기계학습 알고리즘의 초매개변수 튜닝으로 관리가 가능하다(3.2절 참조)
- 훈련 데이터가 기계학습이 적용되는 데이터 공간을 충분히 나타내지 못할 때 샘플 편향성이 발생할 수 있다.

부적절한 편향성은 주로 샘플 편향성에 의해 발생하지만 알고리즘 편향성 때문에 나타나는 경우도 있다.

2.5 윤리 Ethics

캠브리지 사전에서 윤리(ethics)를 다음과 같이 정의하고 있다:

행동을 통제하는 수용된 신념 체계, 특히 도덕에 기반한 체계

(a system of accepted beliefs that control behavior, especially such a system based on morals)

고도화된 기능을 갖춘 인공지능기반 시스템은 사람들의 삶에 대체로 긍정적인 영향을 미치고 있다. 이런 시스템이 점점 보편화되면서 그것이 윤리적으로 사용되는지에 대한 우려도 제기되고 있다.

윤리적이라 여겨지는 것은 시간이 지남에 따라 변할 수 있고 지역이나 문화에 따라서도 변할 수 있다. 한 지역에서 다른 지역으로 인공지능기반 시스템을 배포할 때 이해관계자 간의 가치관 차이를 고려해야 한다.

인공지능 윤리에 대한 국가 또는 국제 수준의 정책은 많은 국가와 지역에서 찾아볼 수 있다. 경제협력개발기구(OECD)는 2019년에 인공지능의 책임있는 개발을 위해 최초로 정부 간 합의된 국제 표준인 인공지능 권고안을 발표했다[B08]. 이 권고안은 발표 당시 42개국에서 채택되었으며 유럽연합 집행위원회(European Commission)의 지지를 받았다. 여기에는 “신뢰할 수 있는 인공지능의 책임 있는 관리”를 위한 실질적인 정책 권고와 가치 기반 원칙이 담겨 있으며 다음과 같이 요약된다:

- 인공지능은 포괄적 성장, 지속가능한 개발, 웰빙을 이끌어 인간과 지구에 도움이 돼야 한다.

- 인공지능 시스템은 법치, 인권, 민주적 가치, 다양성의 가치를 존중하고 공정한 사회 보장을 위해 필요한 적절한 보호 장치를 포함해야 한다.
- 결과를 사람이 이해할 수 있고 이의를 제기할 수 있도록 인공지능 전반에 대한 투명성이 있어야 한다.
- 인공지능 시스템은 수명이 다할 때까지 견고하고 보안이 확보된 안전한 방식으로 기능해야 하며 잠재적 위험은 지속적으로 평가되어야 한다.
- 인공지능 시스템을 개발, 배포, 운영하는 조직 및 개인은 이에 대해 책임을 져야 한다.

2.6 부작용과 보상 해킹 Side Effects and Reward Hacking

부작용과 보상 해킹으로 인해 인공지능기반 시스템이 목표 달성을 시도할 때 예상치 못하거나 때로는 해로운 결과가 나타날 수 있다[B09].

부작용은 인공지능기반 시스템의 설계자가 다음과 같은 목표를 설정했을 때 생길 수 있다 - “주어진 환경에서 특정 작업을 수행하는 데만 중점을 두고 (잠재적으로 매우 클 수 있는) 환경의 다른 측면을 고려하지 않아 실제로 변화에 해로운 환경 변수에 대한 무관심을 은연중에 표현”[B09]. 가령, “최대한 연료를 절약하면서 안전한 방법”으로 목적지까지 이동하는 것을 목표로 하는 자율주행 자동차가 목표는 달성하지만 이동 시간이 너무 오래 걸려 탑승자들이 극도로 짜증을 내는 부작용을 초래하는 것을 예로 들 수 있다.

보상 해킹은 인공지능기반 시스템이 “설계자의 의도를 왜곡하는” “영리한” 또는 “쉬운” 해결책을 사용해 정해진 목표를 달성함으로써, 결국 목표가 왜곡되는 결과를 가져오는 것이다. 보상 해킹에 대한 예시로 널리 사용되는 사례는 인공지능기반 시스템이 아케이드 컴퓨터 게임을 스스로 학습하는데 ‘최고 점수’ 달성을 목표로 정했을 때 이를 위해 게임을 하지 않고 저장된 최고 점수 데이터 기록을 해킹하는 경우를 들 수 있다.

2.7 투명성, 해석 가능성, 설명 가능성

Transparency, Interpretability and Explainability

인공지능기반 시스템은 일반적으로 사용자가 시스템을 신뢰해야 하는 영역에 적용된다. 안전상의 이유일 수도 있지만 사생활 보호가 필요하거나 잠재적으로 삶을 변화시키는 예측 및 결정을 제공해야 하는 경우일 수도 있다.

대부분의 사용자는 “블랙박스”로 인공지능기반 시스템을 접하게 되며, 해당 시스템이 결과에 어떻게 도달했는지는 거의 알지 못한다. 어떤 경우에는 시스템을 구축한 데이터 과학자도 마찬가지일 수 있다. 또, 사용자가

인공지능기반 시스템과 상호작용하고 있다는 사실조차 인식하지 못하는 경우도 있다.

인공지능기반 시스템이 가지는 복잡성은 “설명 가능한 인공지능(XAI)”의 출현으로 이어졌다. XAI의 목표는 사용자가 인공지능기반 시스템이 어떻게 결과를 도출하는지 이해할 수 있도록 함으로써 사용자 신뢰를 높이는 것이다.

런던 왕립학회[B10]에 따르면 XAI가 필요한 이유는 다음을 포함해 여러 가지가 있다:

- 사용자에게 시스템에 대한 신뢰 제공
- 편향성으로부터 보호
- 규제 표준 또는 정책 요구사항 충족
- 시스템 설계 개선
- 리스크, 강건성, 취약성 평가
- 시스템 출력의 이해 및 확인
- 자율성, 에이전시(사용자가 권한이 있다고 느끼게 함), 사회적 가치 충족

이는 이해관계자의 관점에서 인공지능기반 시스템에 대해 다음과 같은 세 가지 이상적인 XAI 기본 특성으로 이어진다 (8.6절 참조):

- 투명성: 모델을 생성하는 데 사용된 알고리즘과 훈련 데이터를 얼마나 쉽게 식별할 수 있는지.
- 해석 가능성: 사용자와 다양한 이해관계자가 사용된 인공지능 기술을 얼마나 이해할 수 있는지.
- 설명 가능성: 사용자가 인공지능기반 시스템이 특정 결과를 어떻게 도출했는지 얼마나 쉽게 판단할 수 있는지.

2.8 안전과 인공지능 Safety and AI

이 실러버스에서 안전이란 인공지능기반 시스템이 사람, 재산 또는 환경에 해를 끼치지 않을 것이라는 기대를 말한다. 안전에 영향을 미치는 의사 결정에 인공지능기반 시스템이 사용될 수 있다. 예를 들어, 의료, 제조, 국방, 보안, 운송 분야에서 사용하는 인공지능기반 시스템은 안전에 영향을 미칠 가능성이 있다.

인공지능기반 시스템이 안전한지(예: 사람에게 해를 가하지 않음) 검증하는 것을 더 어렵게 만드는 특성으로 다음과 같은 것들이 있다:

- 복잡성 (complexity)
- 비결정성 (non-determinism)

- 태생적 확률성 (probabilistic nature)
- 자가학습 (self-learning)
- 투명성, 해석 가능성, 설명 가능성 부족 (lack of transparency, interpretability and explainability)
- 강건성 부족 (lack of robustness)

이런 특성에 대한 테스트의 어려움은 8장에서 다루고 있다.

제3장 기계학습 개요 Machine Learning(ML) Overview	145분
--	-------------

Testing Keywords

None

AI-Specific Keywords

Association, classification, clustering, data preparation, ML algorithm, ML framework, ML functional performance criteria, ML model, ML training data, ML workflow, model evaluation, model tuning, outlier, overfitting, regression, reinforcement learning, supervised learning, underfitting, unsupervised learning

학습 목표:

3.1 기계학습 종류

AI-3.1.1 (K2) 지도학습의 종류로 분류와 회귀를 설명할 수 있다.

AI-3.1.2 (K2) 비지도학습의 종류로 군집화와 연관을 설명할 수 있다.

AI-3.1.3 (K2) 강화 학습을 설명할 수 있다.

3.2 기계학습 워크플로

AI-3.2.1 (K2) 기계학습을 구현하는 데 사용하는 워크플로를 요약할 수 있다.

3.3 기계학습 종류 선택

AI-3.3.1 (K3) 프로젝트 시나리오가 주어졌을 때, 적절한 기계학습 종류(분류, 회귀, 군집화, 연관, 강화 학습)를 식별할 수 있다.

3.4 기계학습 알고리즘 선택 관련 요소

AI-3.4.1 (K2) 기계학습 알고리즘 선택과 관련된 요소를 설명할 수 있다.

3.5 과적합과 과소적합 (Overfitting and Underfitting)

AI-3.5.1 (K2) 과적합과 과소적합 개념을 요약할 수 있다.

HO-3.5.1 (H0) 과적합과 과소적합을 시연한다.

3.1 기계학습 종류 Forms of ML

기계학습 알고리즘은 다음과 같이 분류할 수 있다.

- 지도학습
- 비지도학습
- 강화 학습

3.1.1 지도학습

지도학습 알고리즘은 훈련 단계에서 라벨링이 된 데이터를 가지고 기계학습 모델을 만든다. 라벨링 된 데이터는 일반적으로 한 쌍(페어)의 입력값으로 구성되며(예: 개 이미지와 라벨 "개") 훈련 중 알고리즘을 입력 데이터(예: 개 이미지)와 출력 라벨(예: "개" 또는 "고양이") 사이의 관계를 추론하는 데 사용한다. 기계학습 모델 테스트 단계에서는 학습에 사용하지 않은 새로운 데이터 세트를 훈련된 모델에 적용해 결과를 예측하게 된다. 출력 정확도 수준이 만족스러우면 모델을 배포한다.

지도학습으로 해결되는 문제는 다음 두 가지로 분류한다:

- 분류: 입력을 미리 정해진 몇 개의 클래스 중 하나로 분류해야 하는 문제에 사용된다. 이미지에서 얼굴을 인식하거나 물체를 감지하는 것이 예이다.
- 회귀: 기계학습 모델이 회귀 분석을 사용해 어떤 숫자를 예측해야 하는 문제에 사용된다. 습관에 대한 입력 데이터를 기반으로 사람의 나이를 예측하거나 주식의 미래 가격을 예측하는 것은 회귀를 사용하는 문제의 예이다.

기계학습의 문제 맥락에서 사용하는 '회귀'라는 용어는 [I01]에서와 같이 다른 ISTQB 실러버스에서 수정관련 결함을 초래하는 소프트웨어 변경 문제 설명에서 사용하는 '회귀'와는 다른 용어이다.

3.1.2 비지도학습

이런 유형의 학습에서 알고리즘은 훈련 단계에서 라벨링이 되지 않은 데이터로 기계학습 모델을 만든다. 라벨링되지 않은 데이터는 알고리즘이 훈련 중에 입력 데이터에 들어있는 패턴(pattern)을 추론하고 입력 데이터가 가진 공통 특성을 기반으로 데이터를 서로 다른 클래스에 할당하는 데 사용된다. 테스트 단계에서 학습하지 않은 새로운 데이터 세트를 훈련된 모델에 적용해 입력 데이터가 할당돼야 하는 클래스를 예측한다. 출력 정확도 수준이 만족스러우면 모델을 배포한다.

비지도학습으로 해결되는 문제는 다음 두 가지로 분류된다:

- 군집화: 입력 데이터의 유사성을 식별해 공통적인 특성이나 속성을 기반으로 그룹화할 수 있는 문제에 사용한다. 예를 들어, 군집화는 마케팅 목적으로 다양한 유형의 고객을 분류하는 데 사용한다.
- 연관: 데이터 속성 간 흥미로운 관계 또는 종속성을 식별해야 하는 문제에 사용한다. 예를 들어, 제품 추천 시스템은 고객의 쇼핑 습관을 기반으로 연관성을 식별할 수 있다.

3.1.3 강화 학습

강화 학습은 시스템(지능형 에이전트)이 환경과 계속 상호작용하면서 점진적으로 배우는, 결국 경험을 통해 학습하는 접근법이다. 강화 학습은 훈련 데이터를 사용하지 않는다. 에이전트는 올바른 결정을 내리면 보상을 받고 잘못된 결정을 내리면 처벌을 받는다.

환경 설정, 에이전트가 원하는 목표를 달성할 수 있는 올바른 전략 선택, 보상 기능 설계 등이 강화 학습을 구현할 때 핵심 과제이다. 로봇 공학, 자율주행차, 챗봇 등은 강화 학습을 사용하는 애플리케이션의 예이다.

3.2 기계학습 워크플로 ML Workflow

기계학습 워크플로의 활동은 다음과 같다:

목표 이해

- 비즈니스 우선순위와 부합하도록 배포할 기계학습 모델의 목적에 대한 이해와 이해 관계자 협의가 이루어져야 한다. 개발하는 모델에 대한 인수 기준(기계학습 기능적 성능 측정지표 포함 - 5장 참조)을 정의해야 한다.

프레임워크 선택

- 목표, 인수 기준, 비즈니스 우선순위를 기준으로 적합한 인공지능 개발 프레임워크를 선택해야 한다 (1.5 참조).

알고리즘 선택과 구현

- 기계학습 알고리즘은 목표, 인수 기준, 사용 가능한 데이터를 포함한 다양한 요소를 기반으로 선택해야 한다(3.4 참조). 알고리즘은 직접 코딩할 수도 있지만, 미리 작성된 코드 라이브러리에서 가져오는 경우가 많다. 그런 다음 모델 훈련을 위해 필요하다면 알고리즘을 컴파일한다.

데이터 준비와 테스트

- 데이터 준비(4.1 참조)는 데이터 수집, 데이터 전처리 및 특성 엔지니어링(feature engineering)으로 구성된다. 이런 활동과 병행해 탐색적 데이터 분석(EDA)을 수행할 수 있다.
- 알고리즘과 모델이 사용하는 데이터는 목표에 맞춰 준비하며 그림 1의 “모델 생성 및 테스트(Model generation and test)” 영역의 모든 활동에 사용된다. 예를 들어, 시스템이 실시간 거래 시스템이라면 데이터는 거래 시장에서 가져오게 된다.
- 모델을 훈련, 튜닝, 테스트하는 데 사용되는 데이터는 모델이 사용할 운영 데이터를 대변해야 한다. 모델의 초기 훈련을 위해 사전 수집된 데이터 세트 사용이 가능한 경우도 있다(예: Kaggle 데이터 세트 [R16] 참조). 그런 경우가 아니라면 원시 데이터는 일반적으로 전처리 및 특성 엔지니어링이 필요하다.
- 데이터 테스트와 자동화된 데이터 준비 절차를 수행한다. 입력 데이터 테스트에 대한 자세한 내용은 7.2.1절을 참조할 수 있다.

모델 훈련

- 선택한 기계학습 알고리즘으로 훈련 데이터를 사용해 모델을 훈련한다.
- 신경망을 생성하는 알고리즘과 같은 일부 알고리즘은 훈련 데이터 세트를 여러 번 읽어 들인다. 훈련 데이터 세트에 대한 한 번의 훈련 반복주기를 에포크(epoch)라고 한다.
- 모델 구조를 정의하는 매개변수(예: 신경망의 층(layer) 수 또는 의사 결정 트리의 깊이)는 알고리즘에 전달된다. 이런 매개변수를 모델의 초매개변수라고 한다.

모델 검증

- 검증 데이터 세트를 사용해 합의된 기계학습 기능적 성능 측정지표에 따라 모델을 검증한 다음 그 결과를 모델을 개선(튜닝)하는 데 사용한다. 모델 검증 및 튜닝은 명확한 문서화와 함께 통제된 조건에서 신중하게 수행해야 하는 과학 실험과 유사하게 진행된다. 실무에서는 보통 다양한 알고리즘(예: 랜덤 포레스트, SVM, 신경망)을 사용해 여러 모델을 만들고 훈련해서 검증 및 튜닝 결과에 따라 최상의 모델을 선택한다.

모델 튜닝

- 합의된 기계학습 기능적 성능 측정지표를 사용한 모델 검증 결과는 데이터에 맞게 모델 설정을 조정(튜닝)해 성능을 향상시키는 데 사용한다. 모델의 튜닝은 훈련 활동을 조정(예: 훈련 단계의 수 또는 훈련에 사용되는 데이터의 양 조정)하거나 모델의 속성(예: 신경망 뉴런 수 또는 의사 결정 트리의 깊이)을 업데이트하는 초매개변수 튜닝으로 이루어진다.
- 훈련, 검증, 튜닝의 세 가지 활동은 그림 1에서 보여주고 있듯이 모델 생성 구간(Model generation)을

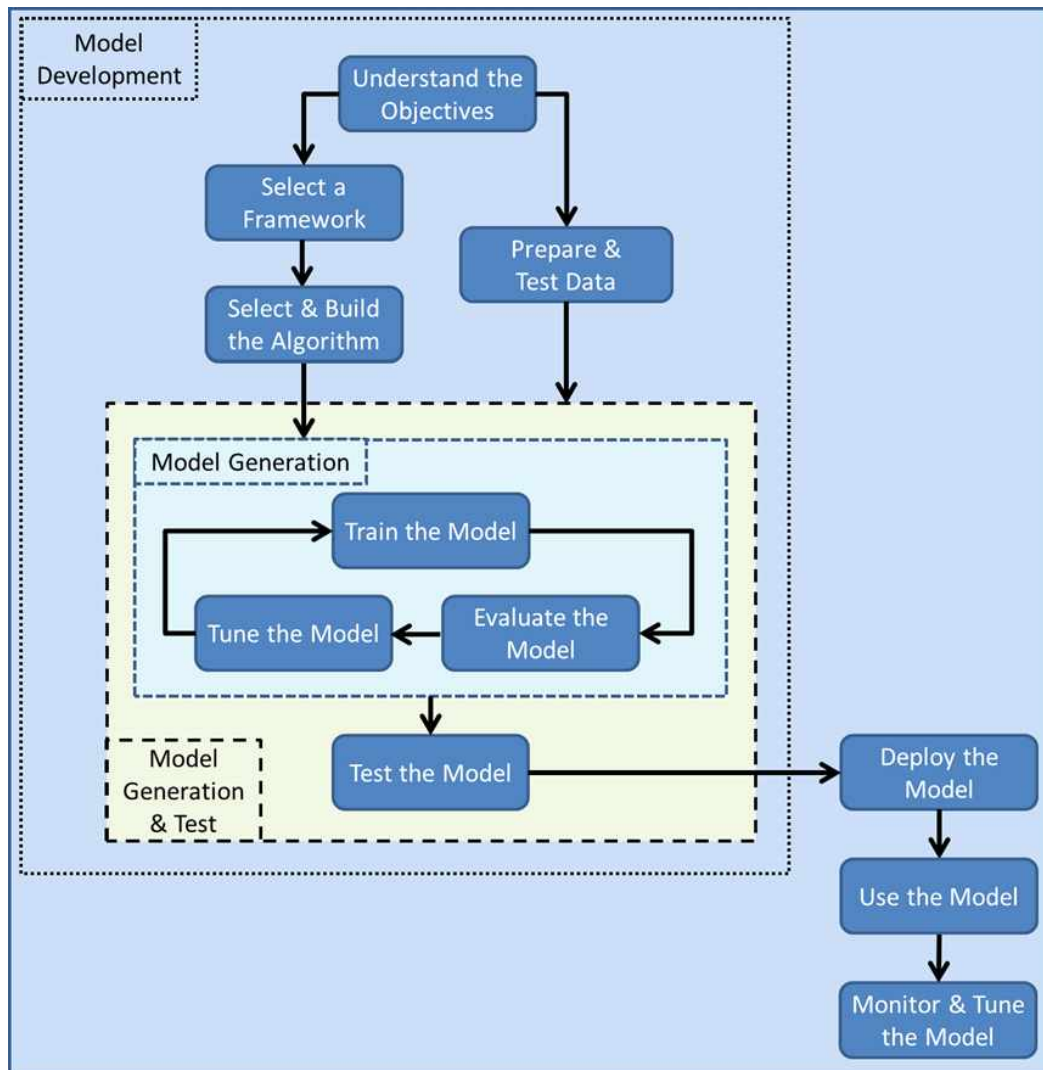


그림 1. 기계학습 워크플로

구성하게 된다.

모델 테스트

- 모델이 생성되면 (즉, 훈련, 검증, 튜닝됨) 합의된 기계학습 기능적 성능 기준을 충족하는지 확인하기 위해 독립적인 테스트 데이터 세트로 테스트해야 한다(7.2.2절 참조). 테스트 단계에서 얻은 기능적 성능 측정 결과는 검증 단계 결과와도 비교된다. 독립적 테스트 데이터로 측정한 모델 성능이 현저히 낮은 경우 다른 모델을 선택해야 할 수 있다.
- 기능적 성능 테스트 외에도 모델을 훈련하는 시간, 예측에 걸리는 시간, 자원 사용량 등 비기능적 테스트도 수행해야 한다. 일반적으로 이런 테스트는 데이터 엔지니어/과학자가 수행하지만 도메인에 대한 충분한 지식과 필요 자원을 가용할 수 있는 테스터가 수행하기도 한다.

모델 배포

- 그림 1과 같이 모델 개발(Model Development)이 완료되면 튜닝된 모델은 보통 관련 자원과 함께 배포를 위해 재가공 된다. 이때 연관된 데이터 파이프라인도 포함된다. 이런 과정은 일반적으로 프레임워크를 통해 진행된다. 목표 환경으로 임베디드 시스템과 클라우드 등이 있을 수 있다. 환경이 클라우드인 경우 웹 API를 통해 모델에 접근하게 된다.

모델 사용

- 일단 배포되면 모델은 일반적으로 더 큰 인공지능기반 시스템의 일부가 되며 운영에 사용될 수 있다. 모델은 정해진 시간 간격으로 이미 계획된 과정을 통해 예측을 수행하거나 요청에 따라 실시간으로 실행할 수도 있다.

모델 모니터링과 튜닝

- 모델을 사용하는 동안 사용 환경의 진화로 모델이 의도한 성능에 못 미칠 수 있다(2.3절과 7.6절 참조). 성능 드리프트(drift)를 식별하고 관리하려면 운영 모델을 인수 기준에 따라 정기적으로 검증해야 한다.
- 드리프트 문제를 해결하기 위해 모델 설정을 업데이트하거나 더 강건하고 정확한 모델을 만들기 위해 새로운 데이터로 다시 훈련해야 할 수 있다. 이 경우 개선된 훈련 데이터로 새 모델을 만들고 훈련할 수도 있다. 그런 다음 A/B 테스트의 한 가지 유형을 가지고 새로운 모델을 기존 모델과 비교할 수 있다(9.4절 참조).

그림 1의 기계학습 워크플로는 논리적 절차이다. 실제로 워크플로는 각 단계가 반복적으로 수행되는 방식으로 적용된다(예: 모델을 검증하고 나면 훈련 단계로 돌아가야 하는 경우가 많고 때로는 데이터 준비 단계로 돌아가야 하는 경우도 있다). 그림 1에 표시된 단계에는 전체 시스템에서 기계학습 모델과 나머지 영역과의 통합을 다루는 단계가 포함되어 있지 않다. 일반적으로 기계학습 모델은 단독으로 배포할 수 없고 기계학습이 아닌 부분과 통합되어야 한다. 예를 들어, 비전(vision) 애플리케이션에는 데이터를 기계학습 모델에 제공하기 전에 데이터를 정제하고 수정하는 데이터 파이프라인이 있다. 이런 경우 7.2절에서 설명하고 있듯이 통합, 시스템, 인수 테스트 단계를 수행할 수 있다.

3.3 기계학습 종류 선택 Selecting a Form of ML

적절한 기계학습 접근법을 선택할 때 다음과 같은 지침을 적용한다:

- 선택한 기계학습 접근법에 충분한 훈련과 테스트 데이터가 있어야 한다.
- 지도학습의 경우 제대로 라벨링된 데이터가 필요하다.
- 출력 라벨이 있으면 지도학습일 수 있다.
- 출력이 이산형(discrete)이면서 범주형(categorical)이면, 분류일 수 있다.
- 출력이 태생적으로 숫자 형이고 연속형이면, 회귀일 수 있다.
- 주어진 데이터 세트에 출력값이 없으면, 비지도학습일 수 있다.
- 문제가 유사한 데이터를 그룹으로 나누는 것과 관련된 경우, 군집화일 수 있다.
- 문제가 동시발생 데이터 항목을 찾는 것과 관련된 경우, 연관일 수 있다.
- 강화 학습은 환경과 상호작용이 있는 상황에 적합하다.
- 문제가 여러 상태의 개념과 관련돼 있고, 각 상태에서 어떤 결정이 이루어지는 경우, 강화 학습을 적용할 수 있다.

3.4 기계학습 알고리즘 선택과 관련된 요소

Factors Involved in ML Algorithm Selection

최적의 기계학습 알고리즘, 기계학습 모델 설정, 기계학습 모델 초매개변수를 선택하는 확실한 방법은 없다. 실제로 이런 사항은 다음과 같은 요소의 조합으로 선택하게 된다:

- 필요한 기능 (예: 기능이 분류인지 이산 값의 예측인지의 여부)
- 요구되는 품질 특성: 예를 들어,
 - 정확도 (예: 어떤 모델은 보다 정확하지만 더 느릴 수 있음)
 - 사용 가능한 메모리의 제약 (예: 임베디드 시스템의 경우)
 - 모델 훈련 (그리고 재훈련) 속도
 - 예측 속도 (예: 실시간 시스템의 경우)
 - 투명성, 해석 가능성, 설명 가능성 요구사항

- 모델 훈련에 사용할 수 있는 데이터 유형 (예: 일부 모델은 이미지 데이터만 취급할 수 있음)
- 모델 훈련과 테스트에 사용할 수 있는 데이터의 양 (예: 일부 모델은 다른 모델보다 데이터 양이 제한적일 때 더 과적합 되는 경향이 있을 수 있음)
- 모델에서 사용할 것으로 예상되는 입력 데이터의 특성 수 (예: 속도, 정확도와 같은 다른 요소가 이 특성 수에 직접적인 영향을 받을 가능성이 있음)
- 군집화 예상 클래스 수 (예: 일부 모델은 둘 이상의 클래스가 있는 문제에 적합하지 않을 수 있음)
- 이전 경험
- 시행 착오

3.5 과적합과 과소적합 Overfitting and Underfitting

3.5.1 과적합

과적합은 모델이 데이터 포인트 세트에 너무 최적화되어 다른 경우에 대해 적절한 일반화가 어려울 때 발생한다. 이런 모델은 훈련에 사용한 데이터를 대상으로는 매우 잘 동작하지만 새로운 데이터에 대해 정확한 예측을 제공하는 것은 어려울 수 있다. 과적합은 모델이 노이즈 또는 이상값으로 볼 수 있는 데이터 포인트를 포함한 모든 데이터 포인트에 피팅(fitting)하려고 할 때 발생할 수 있다. 훈련 데이터 세트에 데이터가 충분하지 않은 경우에도 발생할 수 있다.

3.5.2 과소적합

과소적합은 모델이 훈련 데이터의 패턴에 정확하게 피팅될 수 있을 만큼 정교하지 못할 때 발생한다. 과소적합 모델은 너무 단순한 경향이 있으며, 새로운 데이터나 훈련 데이터와 유사한 데이터 양쪽 모두에 대해 정확한 예측을 제공하는 것이 어려울 수 있다. 과소적합의 하나의 원인은 입력과 출력 간의 주요 상관관계를 반영하는 특징이 들어있지 않은 훈련 데이터 세트일 수 있다. 알고리즘이 데이터에 맞지 않을 때도 발생할 수 있다 (예: 비선형 데이터에 대한 선형 모델 생성).

3.5.3 실습 예제: 과적합과 과소적합 시연

모델의 과적합과 과소적합의 개념을 시연해보자. 데이터가 극히 적은 데이터 세트(과적합)와 특징 상관관계가 낮은 데이터 세트(과소적합)를 사용해 시연할 수 있다.

제4장 기계학습 - 데이터 ML - Data

230분

Keywords

None

AI-Specific Keywords

Annotation, augmentation, classification model, data labelling, data preparation, ML training data, supervised learning, test dataset, validation dataset

학습 목표

4.1 기계학습 워크플로에서의 데이터 준비

AI-4.1.1 (K2) 데이터 준비와 관련된 활동과 어려움을 설명할 수 있다.

HO-4.1.1 (H2) 기계학습 모델 생성에 필요한 데이터를 준비한다.

4.2 기계학습 워크플로에서의 훈련, 검증, 테스트 데이터

AI-4.2.1 (K2) 기계학습 모델 개발에서 훈련, 검증, 테스트 데이터 세트의 사용을 대조할 수 있다.

HO-4.2.1 (H2) 훈련 및 테스트 데이터 세트를 식별하고 기계학습 모델을 생성한다.

4.3 데이터 세트 품질 문제

AI-4.3.1 (K2) 일반적인 데이터 세트 품질 문제를 설명할 수 있다.

4.4 데이터 품질과 데이터 품질이 기계학습 모델에 미치는 영향

AI-4.4.1 (K2) 낮은 데이터 품질이 기계학습 모델에 문제를 일으킬 수 있음을 인식할 수 있다.

4.5 지도학습을 위한 데이터 라벨링

AI-4.5.1 (K1) 지도학습을 위한 데이터 세트의 다양한 라벨링 방식을 상기할 수 있다.

AI-4.5.2 (K2) 데이터 세트의 데이터 라벨링이 잘못되는 이유를 상기할 수 있다.

4.1 기계학습 워크플로에서의 데이터 준비

Data Preparation as Part of the ML Workflow

데이터 준비에는 기계학습 구현 워크플로 전체 노력의 평균 43%가 들어가며, 기계학습 워크플로에서 가장 자원 집약적인 활동이라 할 수 있다. 이에 비해 모델 선택과 구현에는 전체의 17%만 들어간다 [R17]. 데이터 준비 단계는 데이터 파이프라인에 속한다. 데이터 파이프라인은 원본 데이터를 입력받아 기계학습 모델을 훈련하거나 훈련된 기계학습 모델이 데이터를 예측에 사용 가능한 형식으로 출력한다.

데이터 준비는 다음과 같은 활동으로 구성된다:

데이터 수집 (Data acquisition)

- 식별 (Identification): 훈련과 예측에 사용할 데이터 유형을 식별한다. 예를 들어, 자율주행 자동차의 경우 레이더, 비디오, 레이저 이미징, 탐지, 그리고 범위 지정(LiDAR 레이저 거리측량기) 데이터의 필요성 식별을 포함할 수 있다.
- 수집 (Gathering): 데이터의 출처를 식별하고 데이터 수집 방법을 결정한다. 가령, 금융 데이터의 출처로 국제통화기금(IMF)을 식별하고 데이터를 인공지능 기반 시스템에 넣는 데 사용할 채널을 파악하는 것을 예로 들 수 있다.
- 라벨링: 4.5절 참조

수집한 데이터는 다양한 형식일 수 있다(예: 숫자, 범주, 이미지, 표, 텍스트, 시계열, 센서, 지리적 공간, 비디오, 오디오 등).

데이터 전처리(Data pre-processing)

- 정제 (Cleaning): 잘못된 데이터, 중복 데이터 또는 이상값이 식별된 경우 제거하거나 수정한다. 또한, 데이터 대체(imputation)를 사용해 누락된 데이터 값을 추정하거나 추정된 값(예: 평균, 중앙값 그리고 모드값 사용)으로 대체할 수 있다. 개인 정보의 제거 또는 익명화도 수행할 수 있다.
- 변환 (Transformation): 주어진 데이터의 형식을 변경한다(예: 문자열로 저장된 주소를 컴포넌트로 분할, 임의 식별자가 포함된 필드 삭제, 범주형 데이터를 숫자 데이터로 변환, 이미지 형식 변경 등). 숫자 데이터 변환에는 동일한 범위가 적용되도록 크기 조정이 포함된다. 예를 들어, 표준화는 데이터의 평균이 0이고 표준편차가 1이 되도록 데이터의 크기를 조정한다. 이렇게 정규화하면 데이터는 0과 1 사이의 범위를 갖게 된다.

- 증강 (Augmentation): 데이터 세트의 샘플 수를 늘리는 데 사용한다. 증강은 적대적 공격에 대한 강건성을 갖기 위해 훈련 데이터에 적대적 예제를 포함시키는 데에도 사용할 수 있다(9.1절 참조).
- 샘플링: 샘플링은 전체 데이터 세트의 패턴 식별을 위해 사용 가능한 전체 데이터 세트의 일부를 선택하는 작업을 포함한다. 이는 일반적으로 기계학습 모델을 만드는 데 필요한 비용과 시간을 줄이기 위해 수행한다.

모든 전처리에는 유용한 유효 데이터가 변경되거나 비 유효 데이터가 추가될 수 있는 위험이 있다.

특성 엔지니어링

- 특성 선택: 특성은 데이터에 반영된 속성/특징이다. 특성 선택에는 모델 훈련과 예측에 가장 많이 기여할 수 있는 특성을 선택하는 작업이 포함된다. 실제로는 결과 모델에 영향을 주지 않을 (또는 주기를 원하지 않는) 특성을 제거하는 것이 포함되는 경우가 많다. 관련 없는 정보(노이즈)를 제거하는 특성 선택을 통해 전체 훈련 시간을 줄이고, 과적합을 방지하고(3.5.1절 참조), 정확도를 높이고, 모델을 보다 일반화할 수 있다.
- 특성 추출: 기존 특성에서 유용한 특성과 중복되지 않는 특성을 도출하는 과정을 포함한다. 결과 데이터 세트는 일반적으로 더 작고 동일한 정확도를 가진 기계학습 모델을 더 저렴하고 빠르게 생성하는 데 사용할 수 있다.

이런 데이터 준비 활동과 병행해 일반적으로 전체 데이터 준비 작업을 지원하기 위해 탐색적 데이터 분석(EDA, exploratory data analysis)을 수행한다. 여기에는 데이터 분석을 수행해 데이터가 가지고 있는 트렌드(trend)를 식별하고 이 트렌드를 도표로 표시해 데이터를 시각적 형태로 표현하는 데이터 시각화도 포함된다.

위에서는 데이터 준비 활동과 관련 하위 활동을 논리적인 순서로 명시하고 있지만 프로젝트에 따라 순서를 변경하거나 그 중 일부만 사용할 수도 있다. 데이터 출처 식별과 같은 일부 단계는 한 번만 실행되고 또 수동으로 실행되기도 한다. 다른 단계들은 운영 데이터 파이프라인에 포함돼서 주로 라이브(live) 데이터를 가지고 실행되기도 하며, 이런 작업은 자동화되어야 한다.

4.1.1 데이터 준비 과정의 어려움

데이터 준비와 관련된 어려움은 다음과 같다:

- 다음 사항에 관한 지식 필요:
 - 애플리케이션 도메인
 - 데이터와 그것의 속성

- 데이터 준비 관련 다양한 기법
- 여러 출처에서 높은 품질의 데이터를 얻어야 하는 어려움
- 데이터 파이프라인을 자동화하고, 운용 데이터 파이프라인이 확장 가능하고 합리적인 성능 효율성(예: 데이터 항목 처리 완료에 필요한 시간)을 갖도록 해야하는 어려움
- 데이터 준비 관련 비용
- 데이터 준비 중 데이터 파이프라인에 유입된 결함 확인에 충분한 우선순위가 부여되지 않음
- 표본 편향성 유입(2.4절 참조)

4.1.2 실습: 기계학습을 위한 데이터 준비

주어진 원본 데이터 세트에 대해 4.1에서의 설명처럼 데이터 준비단계를 수행해 지도학습 활용 분류 모델 제작에 사용할 데이터 세트를 만들어 본다.

이 활동은 향후 실습에 사용할 기계학습 모델을 만드는 첫 번째 단계이다.

이 활동 수행을 위해 교육생에게는 다음과 같은 적절한 자료가 주어져야 한다:

- 라이브러리
- 기계학습 프레임워크
- 도구
- 개발 환경

4.2 기계학습 구현 단계에서의 훈련, 검증, 테스트 데이터

Training, Validation and Test Datasets in the ML Workflow

논리적으로 봤을 때 기계학습 모델을 개발하려면 세 가지 동등한 데이터 세트(즉, 하나의 초기 데이터 세트에서 무작위로 선택된)가 필요하다:

- 모델 훈련에 사용되는 훈련 데이터 세트
- 모델을 검증하고 이후 튜닝하는 데 사용되는 검증 데이터 세트
- 튜닝된 모델을 테스트하는 데 사용되는 테스트 데이터 세트 (홀드 아웃 데이터 세트(holdout dataset)라고도 함)

적합한 데이터가 무제한으로 가용한 경우 기계학습 워크플로에서 훈련, 검증, 테스트에 사용하는 데이터의

양은 일반적으로 다음 요소에 따라 달라진다:

- 모델 훈련에 사용되는 알고리즘
- 메모리, 디스크 공간, 컴퓨팅 성능, 네트워크 대역폭, 주어진 시간과 같은 자원의 가용성

실제로는 적절한 데이터를 충분히 확보하기 위한 어려움으로 인해 훈련과 검증 데이터 세트는 종종 하나의 데이터 세트에서 도출하게 된다. 테스트 데이터 세트는 별도로 남겨 놓으며 훈련 중에는 사용하지 않는다. 이는 개발한 모델이 테스트 데이터의 영향을 받지 않게 하고 테스트 결과가 모델의 품질을 제대로 보여주게 하기 위함이다.

하나의 데이터 세트를 3개의 데이터 세트로 분할하기 위한 최적의 비율은 없지만, 일반적으로 사용하는(훈련:검증:테스트) 60:20:20에서 80:10:10까지의 비율을 참고할 수 있다. 데이터를 이런 데이터 세트로 분할하는 과정은 보통 무작위로 이루어지며 데이터 세트가 작거나 결과 데이터 세트가 예상 운영 데이터를 대표하지 못할 위험이 있는 경우에는 다른 방법을 사용한다.

데이터가 제한된 경우 가용한 데이터를 3개의 데이터 세트로 분할하면 효과적인 훈련에 필요한 데이터가 부족해질 수 있다. 이 문제를 극복하기 위해 훈련 데이터 세트와 검증 데이터 세트를 결합한 다음 (테스트 데이터 세트는 별도 유지) 이것으로 여러 분할 조합(예: 훈련 80% / 검증 20%)을 생성할 수 있다. 훈련 및 검증 데이터 세트에 데이터는 무작위로 할당한다. 이런 여러 분할 조합으로 훈련, 검증, 튜닝을 수행해 튜닝된 모델을 여러 개 생성하고 전체 모델 성능은 모든 실행의 평균으로 계산할 수 있다. 분할 조합을 만드는 방법에는 분할-테스트, 부트스트랩(bootstrap), k-폴드 교차 검증, 리브-원-아웃 교차 검증(leave-one-out cross validation) 등이 있다(자세한 내용은 [B02] 참조).

4.2.1 실습: 학습과 테스트 데이터 식별 및 기계학습 모델 생성

이전에 준비한 데이터(4.1.2절 참조)를 훈련, 검증, 테스트 데이터 세트로 분할해본다.

이 데이터 세트로 지도학습을 사용해 분류 모델을 훈련 및 테스트해본다.

검증과 테스트 데이터 세트로 달성한 정확도를 비교해 검증/튜닝과 테스트 간의 차이점을 설명해본다.

4.3 데이터 세트 품질 문제 Dataset Quality Issues

데이터 세트의 데이터와 관련한 품질 문제는 다음과 같이 매우 다양하게 나타날 수 있다:

품질 문제	설명
잘못된 데이터	수집한 데이터가 부정확하거나(예: 오류가 있는 센서에서 수집) 잘못 입력됨(예: 복사-붙여넣기 오류)
불완전한 데이터	데이터 값이 누락될 수 있음(예: 레코드의 특정 필드가 비어 있거나 특정 시간의 데이터가 누락되었을 수 있음). 불완전한 데이터의 원인에는 보안 문제, 하드웨어 문제, 인적 오류 등 다양한 원인이 있을 수 있음
잘못 라벨링된 데이터	다양한 원인으로 데이터 라벨링이 잘못될 수 있음(4.5.2절 참조)
데이터 부족	사용 중인 학습 알고리즘에서 패턴을 인식하기에 데이터가 부족함(알고리즘에 따라 필요한 최소 데이터 품질이 달라질 수 있음)
전처리 되지 않은 데이터	데이터는 깨끗하고 일관된 형식을 가지며 원하지 않는 이상값이 포함되지 않도록 전처리되어야 함(4.1절 참조)
오래된 데이터	학습과 예측 모두에 사용되는 데이터는 가능한 최신 데이터이어야 함(예: 몇 년 전의 재무 데이터를 사용함으로써 부정확한 결과가 나올 수 있음)
불균형 데이터	불균형 데이터는 부적절한 편향성(예: 인종, 성별, 민족에 따른 차별), 센서의 잘못된 배치(예: 천장 높이에 설치된 얼굴 인식 카메라), 데이터 세트 가용성의 변동, 데이터 공급자의 다양한 의도 등으로 발생할 수 있음
불공정한 데이터	공정성은 주관적인 품질 특성이지만 정의될 수 있는 경우가 많음. 연령, 다양성 또는 성별 균형을 위해 선택된 데이터는 소수자 또는 취약 계층에 대해 긍정적으로 편향될 수 있음(이런 데이터는 공정한 것으로 간주될 수 있지만 불균형적일 수 있음)
중복 데이터	반복되는 데이터 레코드는 기계학습 모델 결과에 과도하게 영향을 미칠 수 있음
관련성 없는 데이터	처리 중인 문제와 관련이 없는 데이터는 결과에 부정적인 영향을 미칠 수 있고 자원 낭비로 이어질 수 있음
개인 정보 보호 문제	모든 데이터 사용은 관련 데이터 개인 정보 보호법(예: 유럽연합 개인정보법 GDPR)을 준수해야 함
보안 문제	훈련 데이터에 의도적으로 주입된 허위 또는 오해의 소지가 있는 데이터는 훈련된 모델의 정확성에 부정적인 영향을 줄 수 있음

4.4 데이터 품질과 데이터 품질이 기계학습 모델에 미치는 영향

Data Quality and its Effect on the ML Model

기계학습 모델의 품질은 모델을 만들 때 사용한 데이터 세트 품질에 크게 의존한다. 품질이 낮은 데이터는 결함있는 모델과 결함있는 예측 모두를 초래할 수 있다.

데이터 품질 문제로 인해 다음과 같은 유형의 결함이 발생한다:

- 정확도 감소: 이런 결함은 잘못된 데이터, 불완전한 데이터, 잘못 라벨링된 데이터, 부족한 데이터, 오래된 데이터, 관련없는 데이터 그리고 전처리 되지않은 데이터 등으로 인해 발생한다. 예를 들어, 주택 가격 예상 모델을 구축하기 위해 사용할 훈련 데이터가 온실이 딸린 단독주택에 대한 자료를 거의 혹은 전혀 포함하지 않았다면, 이 특정 주택 유형에 대한 가격 예측은 부정확할 것이다.
- 편향된 모델: 이런 결함은 불완전한 데이터, 불균형한 데이터, 불공정한 데이터, 다양성이 부족한 데이터 또는 중복 데이터로 인해 발생한다. 예를 들어, 특정 특성의 데이터가 누락된 경우(예: 질병 예측을 위한 모든 의료 데이터가 특정 성별의 것만 수집됨), 이는 결과 모델에 부정적인 영향을 미칠 가능성이 높다(운영상 모델이 해당 성별만 대상으로 예측하는 경우 제외).
- 손상된 모델: 이런 결함은 데이터 개인정보 보호와 보안 제한으로 인해 발생한다. 예를 들어, 데이터 보호 문제는 보안 취약점으로 이어질 수 있으며, 이는 공격자가 모델의 정보를 역 설계해 개인정보를 유출하는 결과를 가져올 수 있다.

4.5 지도학습을 위한 데이터 라벨링 Data Labelling for Supervised Learning

데이터 라벨링은 라벨을 추가해 라벨이 없는 (또는 라벨이 제대로 지정되지 않은) 데이터를 지도학습에 사용하기 적합하도록 보강하는 것이다. 데이터 라벨링은 기계학습 프로젝트에서 전체 시간의 평균 25%를 사용하는 것으로 보고된 자원 집약적인 활동이다 [B11].

가장 단순한 형태의 데이터 라벨링은 클래스에 맞춰 이미지나 텍스트 파일을 적당한 폴더에 넣는 것으로 구성될 수 있다. 예를 들어, 텍스트 파일에서 긍정적인 제품 리뷰는 모두 한 폴더에 넣고, 부정적인 리뷰는 다른 폴더에 넣는 방식이다. 또 다른 일반적인 라벨링 기법은 이미지 주위에 직사각형을 그려서 이미지에 라벨을 지정하는 것으로 흔히 주석(annotation, 어노테이션)으로 알려진 방법이다. 3D 개체에 라벨을 지정하거나 불규칙한 개체 주위에 경계 상자를 그리려면 더 복잡한 주석이 필요할 수 있다. 데이터 라벨링과 주석은 보통 도구가 사용된다.

4.5.1 데이터 라벨링 접근법

다음과 같은 다양한 방법으로 라벨링을 수행할 수 있다:

- 내부(Internal): 라벨링을 개발자, 테스터 또는 라벨링을 위해 조직에서 구성된 팀이 수행한다.
- 아웃소싱: 라벨링을 외부 전문 기관에서 수행한다.
- 클라우드소싱: 라벨링을 다수의 개인이 수행한다. 라벨링의 품질 관리가 어렵기 때문에 여러 참여자에게 동일한 데이터에 라벨링을 요청하고 이후 사용할 라벨에 대한 결정을 내릴 수 있다.
- 인공지능 지원: 데이터를 인식하고 주석을 달거나 유사한 데이터를 군집화하는 데 인공지능기반 도구를 사용할 수 있다. 이 후 결과는 2단계 프로세스의 일부로써 사람이 결과를 확인하거나 보완할 수 있다 (예: 경계 상자 수정).
- 하이브리드: 위의 라벨링 접근법을 조합해 사용할 수 있다. 예를 들어, 클라우드소싱 라벨링을 일반적으로 전문 인공지능기반 클라우드 관리 도구 사용이 가능한 외부 조직이 관리한다.

이미 라벨링된 데이터 세트를 재사용할 수 있는 경우, 데이터 라벨링 자체가 필요하지 않을 수 있다. 이런 데이터 세트를 캐글[R16] 등에서 많이 공개해 사용 가능하다.

4.5.2 데이터 세트에서 잘못 라벨링된 데이터

지도학습은 데이터 주석자가 데이터를 올바르게 라벨링 했다고 가정한다. 그러나 실제로 데이터 세트의 모든 항목이 올바르게 라벨링 되는 경우는 드물다. 다음과 같은 이유로 라벨이 잘못 지정될 수 있다:

- 주석자에 의해 임의의 오류가 발생할 수 있다 (예: 버튼을 잘못 누름).
- 시스템 오류가 발생할 수 있다 (예: 라벨 담당자가 잘못된 지침을 받거나 훈련을 제대로 받지 못한 경우).
- 악의적인 데이터 주석자에 의해 의도적인 오류가 발생할 수 있다.
- 번역 오류 때문에 한 언어로 맞게 라벨링된 데이터가 다른 언어에서는 라벨을 잘못 지정할 수 있다.
- 해석에 따라 선택이 달라질 수 있는 경우, 데이터 주석자의 주관적 판단으로 인해 서로 다른 주석자의 데이터 라벨이 충돌할 수 있다.
- 필요한 도메인 지식의 부족으로 라벨을 잘못 지정할 수 있다.
- 복잡한 분류 작업으로 인해 더 많은 오류가 발생할 수 있다.
- 데이터 라벨링에 사용한 도구에 결함이 있어 라벨을 잘못 지정할 수 있다.
- 라벨링에 대한 기계학습 기반 접근법이 확률적이므로 일부 라벨을 잘못 지정할 수 있다.

제5장 기계학습 기능적 성능 측정지표 ML Functional Performance Metrics

120분

Keywords

None

AI-Specific Keywords

Accuracy, area under curve (AUC), confusion matrix, F1-score, inter-cluster metrics, intra-cluster metrics, mean square error (MSE), ML benchmark suites, ML functional performance metrics, precision, recall, receiver operating characteristic (ROC) curve, regression model, R-squared, silhouette coefficient

학습 목표

5.1 혼동 행렬(Confusion Matrix)

AI-5.1.1 (K3) 주어진 혼동 행렬 데이터 세트에서 기계학습 기능적 성능 측정지표를 계산할 수 있다.

5.2 분류, 회귀, 군집화를 위한 추가 기계학습 기능적 성능 측정지표

AI-5.2.1 (K2) 분류, 회귀, 군집화 기법에 대한 기계학습 기능적 성능 측정지표의 개념을 대조하고 비교할 수 있다.

5.3 기계학습 기능적 성능 측정지표의 한계

AI-5.3.1 (K2) 기계학습 시스템의 품질을 결정하기 위해 기계학습 기능적 성능 측정지표를 사용하는 것의 한계를 요약할 수 있다.

5.4 기계학습 기능적 성능 지표 선택

AI-5.4.1 (K4) 주어진 기계학습 모델 및 시나리오에 적합한 기계학습 기능적 성능 측정지표 및 값을 선택할 수 있다.

HO-5.4.1 (H2) 선택된 기계학습 기능적 성능 측정지표를 사용해 만든 기계학습 모델을 평가한다.

5.5 기계학습 벤치마크 스위트

AI-5.5.1 (K2) 기계학습에서 벤치마크 스위트의 사용을 설명할 수 있다.

5.1 혼동 행렬 Confusion Matrix

분류 문제에서 모델은 항상 결과를 정확하게 예측하기 어렵다. 이런 문제에 대해 다음과 같은 영역을 가진 혼동 행렬을 만들 수 있다:

		실제값	
		Positive	Negative
예측값	Positive	True Positive (TP)	False Positive (FP)
	Negative	False Negative (FN)	True Negative (TN)

그림 2. 혼동 행렬

그림 2에 표시된 혼동 행렬은 달리 표현될 수 있으나 항상 True Positive (TP), False Positive (FP), False Negative (FN), True Negative (TN)의 네 가지 가능한 상황에 대한 값을 가진다.

혼동 행렬을 기반으로 다음과 같은 측정지표가 정의된다:

- 정확도 (Accuracy)
 - 정확도 = $(TP + TN) / (TP + TN + FP + FN) \times 100\%$
 - 정확도는 모든 올바른 분류의 백분율을 측정한다.
- 정밀도 (Precision)
 - 정밀도 = $TP / (TP + FP) \times 100\%$
 - 정밀도는 올바르게 예측된 positive의 비율을 측정한다. Positive 예측에 대해 얼마나 확신을 가질 수 있는지를 나타내는 척도이다.
- 재현율 (Recall)
 - 재현율 = $TP / (TP + FN) \times 100\%$
 - 재현율(민감도라고도 함)은 실제 positive 중 올바르게 예측된 비율을 측정한다. Positive를 놓치지 않는 것에 대해 얼마나 확신을 가질 수 있는지를 나타내는 척도이다.
- F1-점수
 - F1-점수 = $2 \times (\text{정밀도} \times \text{재현율}) / (\text{정밀도} + \text{재현율})$
 - F1-점수는 정밀도와 재현율의 조화 평균(harmonic mean)으로 계산된다. 이 점수는 0에서 1 사이의 값을 가진다. 점수가 1에 가까우면 False 데이터가 결과에 거의 영향을 미치지 않았다는 것을 나타낸다. F1 점수가 낮으면 모델이 positive 감지에 부족하다는 것을 나타낸다.

5.2 분류, 회귀, 군집화를 위한 추가 기계학습 기능적 성능 측정지표

Additional ML Functional Performance Metrics for Classification, Regression and Clustering

(5.1절에 설명한 분류관련 지표 외에도) 다양한 유형의 기계학습 문제에 대한 수 많은 측정지표가 있다. 일반적으로 가장 많이 사용하는 측정지표의 일부는 다음과 같다:

지도 분류 측정지표(Supervised Classification Metrics)

- 수신자 조작 특성(ROC, receiver operating characteristic) 곡선은 식별 임계값에 따른 이진 분류기의 능력을 보여주는 그래프이다. 이 방법은 원래 군용 레이더를 위해 개발되었기 때문에 이렇게 명명되었다. ROC 곡선은 거짓 양성 비율(false positive rate, $FPR = FP / (TN + FP)$)에 대한 참 양성 비율(TPR, 재현율이라고도 함)로 표시되며, y축에는 TPR이 x축에는 FPR이 표시된다.
- 곡선 아래 면적(AUC, area under curve)은 ROC 곡선 아래의 면적을 말한다. 분류기의 분류 가능성 정도를 나타내며, 모델이 클래스를 얼마나 잘 구분하는지 보여준다. AUC가 높을수록 모델의 예측 성능이 좋다.

지도 회귀 측정지표(Supervised Regression Metrics)

지도 회귀 모델에서 측정지표는 회귀선이 실제 데이터 포인트에 얼마나 잘 맞는지를 나타낸다.

- 평균 제곱근 오차(MSE, Mean Square Error)는 실제값과 예측값 간의 차를 제곱한 값들의 평균이다. MSE 값은 항상 양수이며 0에 가까울수록 더 나은 회귀 모델을 의미한다. 차이를 제곱하여 양의 오류와 음의 오류가 서로 상쇄되지 않도록 한다.
- R-제곱(결정계수라고도 함)은 회귀 모델이 종속 변수에 얼마나 잘 맞는지를 측정한 것이다.

비지도 군집화 측정지표(Unsupervised Clustering Metrics)

비지도 군집화의 경우 다양한 군집 간의 거리와 주어진 군집 내 데이터 포인트의 근접성을 나타내는 몇 가지 측정지표가 있다:

- 군집 내 측정지표(intra-cluster metrics)는 군집 내 데이터 포인트의 유사성을 측정한다.
- 군집 간 측정지표(inter-cluster metrics)는 서로 다른 군집에 있는 데이터 포인트의 유사성을 측정한다.
- 실루엣 계수(실루엣 점수라고도 함)는 평균 군집 내 및 군집 간 거리를 기준으로 하는 측정값(-1에서 +1사이)이다. +1은 군집이 잘 분리되었음을 의미하고, 0은 무작위 군집을 의미하며, -1은 군집이 잘못 할당되었음을 의미한다.

5.3 기계학습 기능적 성능 측정지표의 한계

Limitations of ML Functional Performance Metrics

기계학습 기능적 성능 측정지표는 모델의 기능을 정확도, 정밀도, 재현도, MSE, AUC, 실루엣 계수 등으로 측정하는 것으로 제한된다. ISO 25010[S06]에 정의된 다른 비기능적 품질 특성(예: 수행 효율성) 및 2장에서 설명한 특성(예: 설명 가능성, 유연성, 자율성)은 측정하지 않는다. 이 실러버스에서 "기계학습 기능적 성능 측정지표"라는 용어를 사용하는 것은 이런 기능적 측정지표를 지칭하기 위해 "성능 측정지표"라는 용어가 널리 사용되기 때문이다. "기계학습 기능적"은 이런 측정지표가 기계학습 측면에만 한정되며, 수행 효율성 측정지표와는 관련이 없음을 강조하기 위해 추가됐다.

기계학습 기능적 성능 측정지표는 다음과 같은 다른 요인에 의해 제약을 받는다:

- 지도학습의 경우 기계학습 기능적 성능 측정지표는 라벨링된 데이터를 기반으로 계산되며, 정확한 지표값을 얻기 위해서는 올바른 라벨링이 필요하다 (4.5절 참조)
- 측정에 사용한 데이터는 대표성이 없을 수 있으며(예: 편향될 수 있음), 기계학습 기능적 성능 측정지표는 이 데이터에 따라 달라진다 (4.5절 참조)
- 시스템은 여러 컴포넌트로 구성될 수 있지만, 기계학습 기능적 성능 측정지표는 기계학습 모델에만 적용된다. 예를 들어, 기계학습 기능적 성능 지표는 모델을 평가하기 위해 데이터 파이프라인은 고려하지 않는다.
- 대부분의 기계학습 기능적 성능 지표는 도구를 사용해야 측정 가능하다.

5.4 기계학습 기능적 성능 지표 선택 Selecting ML Functional Performance Metric

일반적으로 혼동 행렬의 모든 기계학습 기능적 성능 측정지표에 대해 최고 점수를 달성하는 기계학습 모델을 구축하는 것은 불가능하다. 대신, 모델의 예상 사용 형태를 기반으로 가장 적절한 기계학습 기능적 성능 측정지표를 인수 기준으로 택한다(예: 거짓양성(False Positives)을 최소화하려면 높은 정밀도 값이 필요하지만 거짓음성(False Negatives)을 최소화하려면 재현율 측정지표가 높아야 한다). 5.1절과 5.2절에 설명한 기계학습 기능적 성능 측정지표를 선택할 때 다음 기준을 사용할 수 있다:

- 정확도: 이 측정지표는 데이터 세트가 대칭인 경우(예: 거짓양성과 거짓음성 개수 및 비용이 유사함)에 적용할 가능성이 높다. 한 데이터 클래스가 다른 클래스보다 우세한 경우에는 좋지 않은 선택이며, 이

경우 F-1 점수를 고려해야 한다.

- 정밀도: 이것은 거짓양성의 비용이 높고 긍정적인 결과에 대한 신뢰도가 높아야 할 때 적합한 측정지표일 수 있다. 스팸 필터는 대부분의 사용자가 실제 스팸이 아닌 이메일을 너무 많이 스팸 폴더에 넣는 것을 허용하지 않기 때문에 높은 정밀도가 필요한 예이다(이메일을 스팸으로 분류하는 것이 긍정적인 것으로 간주되는 경우). 긍정 클래스 비율이 매우 높은 경우에 사용하는 분류기에 정밀도만을 사용하는 것은 좋은 선택이 아닐 것이다.
- 재현율: 긍정 클래스를 놓치지 않는 것이 치명적일 경우에 높은 재현율이 중요하다. 예를 들어, 암 검진에서 실제 양성 결과를 누락해(즉, 암이 검출되지 않음) 음성으로 표시하는 것은 허용되지 않을 것이다.
- F-1 점수: F-1 점수는 예상 클래스 간에 불균형이 있고 정밀도와 재현도의 중요성이 비슷한 경우에 가장 유용하다.

위 측정지표 외에도 5.2절에 여러 측정지표가 설명되어 있다. 이는 다음과 같은 기계학습 문제에 적용할 수 있다:

- ROC 곡선의 AUC는 지도 분류 문제에 사용할 수 있다.
- MSE와 R-제곱은 지도 회귀 문제에 사용할 수 있다.
- 비지도 군집화 문제에는 군집 간 측정지표, 군집 내 측정지표, 실루엣 계수를 사용할 수 있다.

5.4.1 실습: 생성된 기계학습 모델 평가

이런 실습에서 훈련한 분류 모델을 사용해 정확도, 정밀도, 재현도, F-1 점수값을 계산하고 표시해본다. 개발 프레임워크에서 제공하는 라이브러리 함수가 있는 경우 그것을 사용해 계산을 수행한다.

5.5 기계학습 벤치마크 스위트 Benchmark Suites for ML

신규 데이터 세트, 알고리즘, 모델, 하드웨어 등을 포함한 새로운 인공지능 기술이 정기적으로 나오고 있어 각각의 새로운 기술이 가지는 상대적인 효능을 판단하기 어려울 수 있다.

이런 서로 다른 기술을 객관적으로 비교하기 위해 업계 표준 기계학습 벤치마크 스위트(suite)를 사용할 수 있다. 여기에는 광범위한 애플리케이션 영역이 포함되며, 인공지능과 기계학습 성능을 위한 하드웨어 플랫폼, 소프트웨어 프레임워크, 클라우드 플랫폼을 평가하기 위한 도구가 제공된다.

기계학습 벤치마크 스위트는 훈련 시간(예: 프레임워크가 정의된 학습 데이터 세트를 사용해 기계학습 모델

이 정확성 75%와 같은 특정 목표 품질 측정지표 값을 달성하도록 훈련하기 위한 시간)과 예측 시간(예: 훈련된 기계학습 모델이 예측을 수행하는 시간)을 포함한 다양한 측정지표를 제공할 수 있다.

기계학습 벤치마크 제품군은 다음과 같은 여러 조직에서 제공하고 있다:

- MLCommons [R18]: 2020년에 설립된 비영리 단체로 이전에는 ML Perf로 불렸으며 소프트웨어 프레임워크, 인공지능 전용 프로세서, 기계학습 클라우드 플랫폼에 대한 벤치마크를 제공하고 있다.
- DOWNBench [R19]: 스탠포드 대학에서 제공하는 기계학습 벤치마크 스위트이다.
- MLMark [R20]: 임베디드 마이크로프로세서 벤치마크 컨소시엄에서 제공하는 임베디드 시스템의 예측 성능과 정확도를 측정하도록 설계된 기계학습 벤치마크 스위트이다.

제6장 기계학습 - 신경망과 테스트 ML - Neural Networks and Testing

65분

Keywords

None

AI-Specific Keywords

Activation value, deep neural network (DNN), ML training data, multi-layer perceptron, neural network, neuron coverage, perceptron, sign change coverage, sign-sign coverage, supervised learning, threshold coverage, training data, value change coverage

학습 목표

6.1 신경망 (Neural Network)

AI-6.1.1 (K2) 딥러닝 네트워크(DNN, Deep Neural Network)를 포함한 신경망의 구조와 기능을 설명할 수 있다.

HO-6.1.1 (H1) 퍼셉트론(perceptron) 구현을 경험한다.

6.2 신경망 커버리지 측정지표

AI-6.2.1 (K2) 신경망에 대한 다양한 커버리지 측정지표를 설명할 수 있다.

6.1 신경망 Neural Networks

초기 인공 신경망은 생물학적 뉴런의 연결로 볼 수 있는 인간 뇌의 기능을 모방하기 위해 고안되었다. 단층 퍼셉트론은 인공 신경망 구현의 초기 사례 중 하나이며, 하나의 층(즉, 단일 뉴런)으로 구성된 신경망을 가지고 있고 입력값이 특정 클래스에 속하는지 여부를 결정하는 분류기의 지도학습에 사용할 수 있다.

현재 대부분의 신경망은 여러 개의 층으로 구성되기 때문에 심층(Deep) 신경망으로 간주하며 또한, 다층 (multi-layer) 퍼셉트론으로 간주할 수 있다.

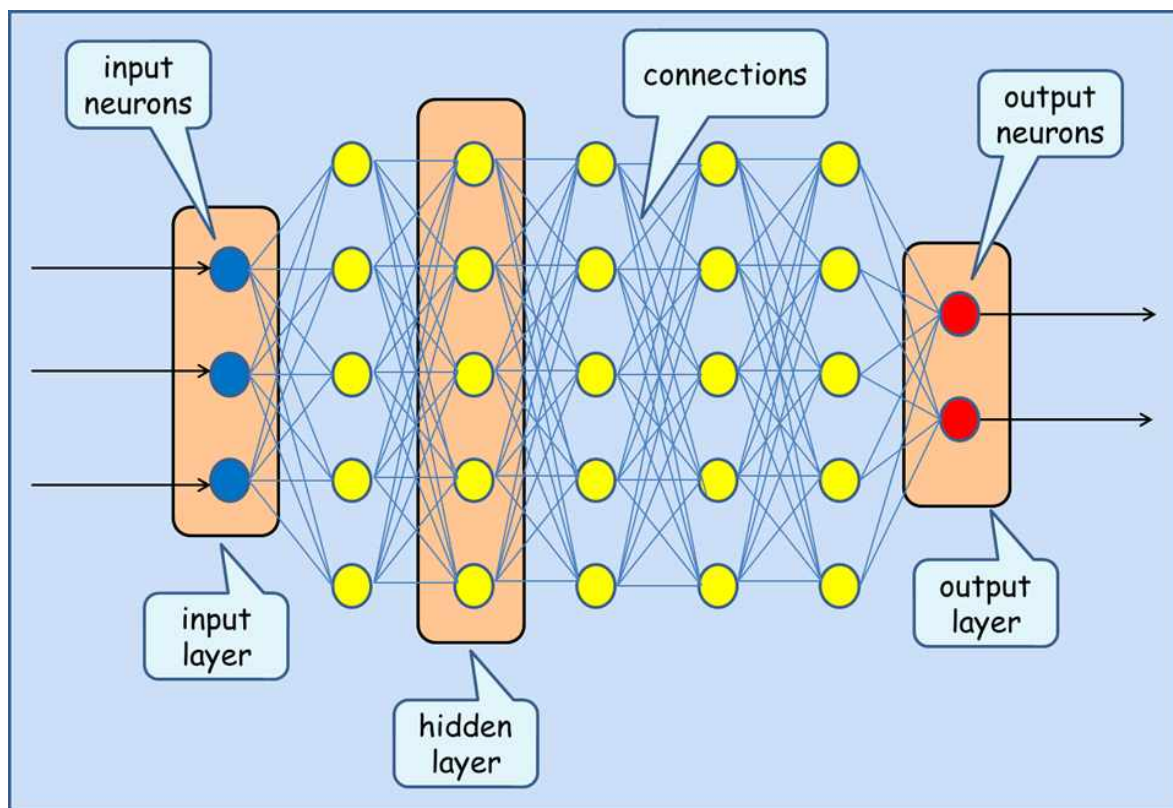


그림 3. 심층(Deep) 신경망의 구조

심층(Deep) 신경망은 세 가지 유형의 층으로 구성된다. 입력층은 카메라로부터 얻는 픽셀값과 같은 입력을 받는다. 출력층은 외부에 결과를 제공한다. 예를 들어, 입력 이미지가 고양이일 가능성을 나타내는 값일 수 있다. 입력층과 출력층 사이에는 노드라고 불리는 인공 뉴런으로 구성된 은닉층(hidden layer)이 있다. 한 층에 있는 뉴런들은 다음 층에 있는 뉴런들과 연결되어 있고 각각 연속되는 층의 뉴런 수는 다를 수 있다. 뉴런은 계산을 수행하고 입력 뉴런에서 출력 뉴런까지 네트워크를 통해 정보가 전달된다.

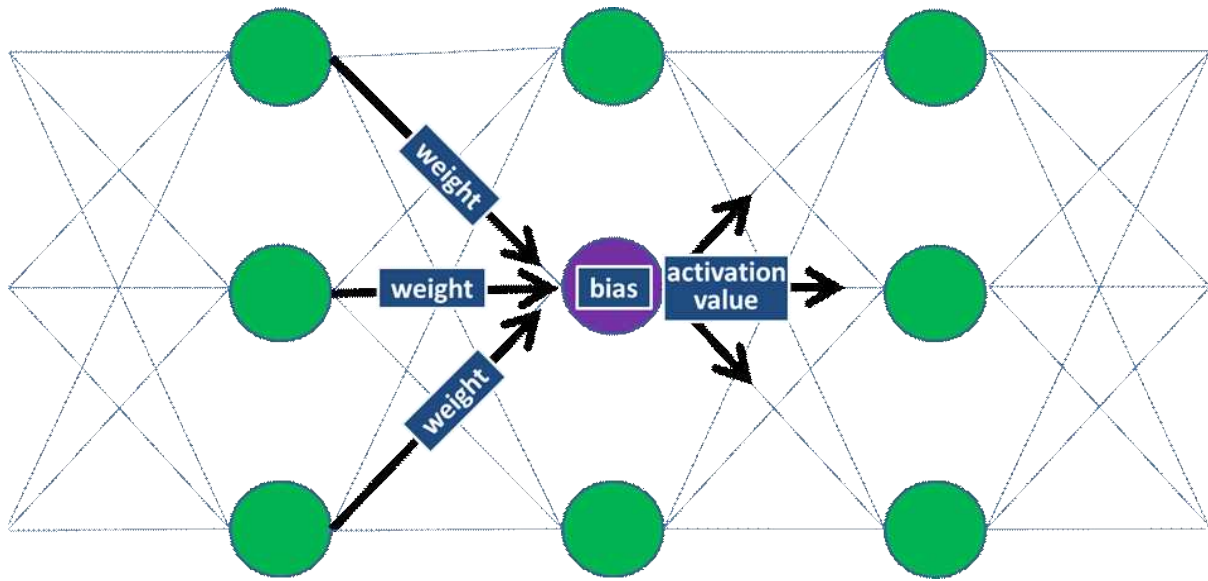


그림 4. 각 뉴런이 수행하는 계산

그림 4에서 볼 수 있듯이, 각 뉴런(입력층 뉴런 제외)이 수행하는 계산은 활성화값을 생성한다. 이 값은 이전 층의 모든 뉴런이 내보낸 활성화값, 뉴런 간의 각 연결에 할당된 가중치(이 가중치는 망이 학습함에 따라 변경됨), 그리고 각 뉴런이 가지는 편향성 값을 입력으로 받는 어떤 공식(활성화 함수)을 통해 계산된다. 여기서 얘기하는 편향성은 사전 설정된 상수로 2.4절에서 언급된 편향성과는 관련이 없다. 다른 활성화 함수를 실행하면 다른 활성화값이 나올 수 있다. 이 값들은 일반적으로 0을 중심으로 하며, -1 (뉴런이 "무관심하다"라는 의미)에서 +1 (뉴런이 "관심이 많다"라는 의미) 사이의 범위를 갖는다.

신경망을 훈련할 때는 각 뉴런에 편향성 값이 사전 설정되고 훈련 데이터는 네트워크를 통해 전달되며 각 뉴런이 활성화 함수를 실행해 최종적으로 어떤 출력이 만들어진다. 그런 다음 생성된 출력을 알고 있는 올바른 결과와 비교한다 (지도학습에 관한 이번 예 같은 경우 라벨링된 데이터를 사용). 실제 출력과 알고 있는 올바른 결과 간의 차이는 네트워크를 통해 역전파 되고, 이 차이를 최소화하기 위해 뉴런 간 연결에 부여된 가중치 값이 수정된다. 네트워크에 더 많은 훈련 데이터가 공급되면 가중치는 망이 학습함에 따라 점진적으로 조정된다. 이런 과정을 통해 결국 생성된 출력이 훈련을 종료해도 괜찮다고 판단되는 시점이 온다.

6.1.1 실습: 단층 퍼셉트론 구현

AND 함수와 같은 간단한 함수를 학습하는 퍼셉트론을 시연하는 실습을 진행한다.

실습은 퍼셉트론이 어떻게 오류가 0이 될 때까지 여러 에포크를 걸쳐 가중치를 수정하고 학습하는지의 과정을 다루어야 한다. 실습에는 다양한 도구가 사용될 수 있다(예: 스프레드시트, 시뮬레이션).

6.2 신경망 커버리지 측정지표 Coverage Measures for Neural Networks

기존의 명령형 소스 코드를 사용할 때 화이트박스 테스트 커버리지 기준(예: 구문, 결정, 변형 조건/결정 커버리지(MC/DC)) [I01]을 달성하는 것은 일부 안전 관련 표준[S07]을 준수하는 데 필수적이며, 다른 중요 애플리케이션 관련 많은 테스트 실무자들이 권장하고 있는 사항이다. 커버리지 모니터링과 개선은 새로운 테스트 케이스 설계를 지원하여 테스트 대상에 대한 신뢰도를 높인다.

신경망이 실행될 때마다 동일한 코드가 실행되는 경향이 있으므로 신경망에 대한 커버리지 측정에 이런 지표를 사용하는 것은 거의 쓸모가 없다. 대신 신경망 자체의 구조, 보다 구체적으로는 그 안의 뉴런에 대한 커버리지를 기반으로 하는 측정지표가 제안되었다. 이런 커버리지 측정지표의 대부분은 뉴런의 활성화값을 기반으로 한다.

신경망에 대한 커버리지는 새로운 연구 영역이다. 2017년 이후에야 학술논문들이 발표되었으며, 제안된 방안들이 효과적이라는 객관적인 증거(예: 중복된 연구 결과)는 아주 적다. 그러나 구문과 결정 커버리지가 50년 이상 사용되었고 의료기기와 항공 전자 시스템과 같은 안전 관련 애플리케이션에서 소프트웨어의 커버리지를 측정하도록 의무화했음에도 불구하고 상대적 효과에 대한 객관적 증거가 거의 없다는 점을 유의해서 생각해 볼 필요가 있다.

신경망에 대한 커버리지 기준이 다음과 같이 제안됐으며 연구가들에 의해 다양한 응용 분야에 적용되었다:

- 뉴런 커버리지: 뉴런 커버리지 달성을 위해서는 (커버리지 100%) 신경망의 각 뉴런은 0보다 큰 활성화값을 달성해야 한다 [B12]. 이것은 실제로 달성하기 매우 쉽고 연구에 따르면 다양한 심층 신경망에서 아주 적은 수의 테스트 케이스로 100%에 가까운 커버리지 달성이 가능한 것으로 나타났다. 이 커버리지 측정지표는 달성되지 않을 때 경고 신호로써 가장 유용할 수 있다.
- 임계값 커버리지: 임계값 커버리지 달성을 위해서는 신경망의 각 뉴런이 지정된 임계값보다 큰 활성화값을 달성해야 한다. DeepXplore 프레임워크를 만든 연구가들은 임계값을 초과하는 활성화값을 기반으로 뉴런 커버리지를 측정해야 한다고 제안했다. 임계값은 상황에 따라 변한다. 연구자들은 이 화이트박스 접근법을 사용해 경계 상황에서 수천 개의 잘못된 동작을 효율적으로 발견했을 때 사용한 임계값이 0.75였다고 발표했다. 임계값을 0으로 하는 뉴런 커버리지를 지칭하기 위해 “뉴런 커버리지”라는 용어를 사용하는 연구가들이 있으므로 여기서는 그것과 더 쉽게 구별하기 위해 임계값 커버리지라는 명칭을 사용한다.
- 부호-변경 커버리지 (Sign-Change coverage): 부호변경 커버리지를 달성하려면 테스트 케이스들은 각 뉴런이 양과 음의 활성화값을 모두 달성해야 한다 [B13].

- 값-변경 커버리지 (Value-Change coverage): 값변경 커버리지를 달성하려면 테스트 케이스들은 각 뉴런이 정해진 어떤 값보다 큰 차를 가지는 두 개의 활성화값을 달성해야 한다. [B13].
- 부호-부호 커버리지 (Sign-Sign coverage): 이 커버리지는 인접한 층에 위치한 뉴런의 페어(pair)와 그 뉴런들의 활성화값 부호와 관련이 있다. 한 쌍의 뉴런 페어에 대한 커버리지를 달성하려면, 테스트 케이스는 첫 번째 층에 속한 뉴런의 부호를 바꾸는 것이 두 번째 층에 속한 뉴런의 부호를 바꾸지만, 두 번째 층의 다른 모든 뉴런의 부호는 바뀌지 않는다는 것을 보여줘야 한다 [B13]. 이것은 명령형 소스 코드에 대한 MC/DC 커버리지와 유사한 개념이다.

연구자들은 층을 대상으로 하는 커버리지(부호-부호 커버리지보다 간단한)를 추가로 발표했으며, 인접한 뉴런 세트의 의미있는 변화를 식별하기 위해 최근접 이웃 알고리즘을 사용한 성공적인 접근법이 TensorFuzz 도구에 구현되었다 [B14].

제7장 인공지능기반 시스템 테스트 개요

Testing AI-Based Systems Overview

115분

Keywords

Input data testing, ML model testing

AI-Specific Keywords

AI component, automation bias, big data, concept drift, data pipeline, ML functional performance metrics, training data

학습 목표

7.1 인공지능기반 시스템 명세

AI-7.1.1 (K2) 인공지능기반 시스템의 시스템 명세가 왜 테스트를 어렵게하는지 설명할 수 있다.

7.2 인공지능기반 시스템의 테스트 레벨

AI-7.2.1 (K2) 인공지능기반 시스템을 각 테스트 레벨에서 어떻게 테스트하는지 설명할 수 있다.

7.3 인공지능기반 시스템 테스트를 위한 테스트 데이터

AI-7.3.1 (K1) 인공지능기반 시스템 테스트를 어렵게 만들 수 있는 테스트 데이터 관련 요소를 상기할 수 있다.

7.4 인공지능기반 시스템의 자동화 편향성에 대한 테스트

AI-7.4.1 (K2) 자동화 편향성과 이것이 테스트에 미치는 영향을 설명할 수 있다.

7.5 기계학습 모델 문서화

AI-7.5.1 (K2) 인공지능 컴포넌트에 대한 문서를 설명하고, 이 문서가 어떻게 인공지능기반 시스템의 테스트를 지원하는지 이해할 수 있다.

7.6 컨셉 드리프트(Concept Drift) 테스트

AI-7.6.1 (K2) 컨셉 드리프트 처리를 위해 왜 훈련된 모델을 자주 테스트해야하는지 설명할 수 있다.

7.7 기계학습 시스템을 위한 테스트 접근법 선택

AI-7.7.1 (K4) 주어진 시나리오에 대해 기계학습 시스템을 개발할 때 따라야 할 테스트 접근법을 결정할 수 있다.

7.1 인공지능기반 시스템 명세 Specification of AI-Based Systems

시스템 요구사항과 설계 명세는 인공지능기반 시스템과 기존 시스템 모두에서 똑같이 중요하다. 이런 명세는 테스터가 실제 시스템 동작이 명시된 요구사항과 일치하는지 확인할 수 있는 근거를 제공한다. 그러나 명세가 불완전하고 테스트 용이성이 부족하면 테스트 오라클(oracle) 문제가 발생한다(8.7절 참조).

인공지능기반 시스템의 명세가 특히 어려울 수 있는 몇 가지 이유는 다음과 같다:

- 많은 인공지능기반 시스템 프로젝트에서의 요구사항은 상위 수준의 비즈니스 목표와 필요 예측 정도에만 명시된다. 그 이유 중 하나는 인공지능기반 시스템 개발이 가지는 탐색적 성격 때문이다. 인공지능기반 시스템 프로젝트는 데이터 세트로 시작해서 해당 데이터에서 얻을 수 있는 예측 결정을 목표로 하는 경우가 많다. 이는 시작부터 필요 논리를 명시하는 전통적 프로젝트와 대조된다.
- 인공지능기반 시스템의 정확도는 독립적인 테스트의 테스트 결과가 나올 때까지 알 수 없는 경우가 대부분이다. 탐색적 개발 접근법으로 원하는 인수 기준을 결정할 때 구현이 이미 진행 중이라는 점이 불완전한 명세로 이어지는 경우가 많다.
- 많은 인공지능기반 시스템이 가지는 확률적 특성으로 예측 정확도 등과 같은 일부 기대 품질 요구사항에 대해서는 허용 오차를 지정해야 할 수 있다.
- 시스템 목표가 특정 기능을 제공하기보다 인간 행동을 모방하는 것인 경우, 대체하고자 하는 인간 행동과 같거나 나아야 한다는 기준만으로 작성된 미흡한 수준의 행동 요구사항이 도출되는 경우가 많다. 이는 테스트 오라클을 정의하기 어렵게 만들 수 있으며, 특히 대체하고자 하는 사람들 간의 능력 편차가 클 때 더욱 심하다.
- 자연어 인식, 컴퓨터 비전, 인간과의 물리적 상호작용 같은 사용자 인터페이스 구현을 위해 인공지능을 사용하는 경우, 시스템은 높은 유연성을 보여야 한다. 그러나 이런 유연성은 상호작용이 발생할 수 있는 다양한 방법을 식별하고 문서화하는 데 어려움을 초래할 수도 있다.
- 적응성, 유연성, 진화, 자율성과 같은 인공지능기반 시스템 특화 품질 특성이 요구사항 명세에 고려되고 정의되어야 한다 (2장 참조). 이런 특성은 새로운 것이기 때문에 그것을 정의하고 테스트하기 어려울 수 있다.

7.2 인공지능기반 시스템의 테스트 레벨 Test Levels for AI-Based Systems

인공지능기반 시스템은 일반적으로 인공지능 컴포넌트와 비 인공지능 컴포넌트를 모두 가진다. 비 인공지능

능 컴포넌트는 기존 접근법[01]을 사용해 테스트할 수 있지만, 인공지능 컴포넌트와 인공지능 컴포넌트를 포함한 시스템은 아래에 설명하고 있는 것처럼 일부 측면에서 다르게 테스트해야 할 수 있다. 인공지능 컴포넌트 테스트가 포함된 모든 테스트 레벨에서는 데이터 엔지니어/과학자 및 도메인 전문가의 긴밀한 지원이 중요하다.

전통적 소프트웨어에 사용하는 테스트 레벨과의 주요 차이점은 인공지능기반 시스템에서 사용되는 모델과 입력 데이터의 테스트 처리를 위한 두 개의 새로운 특정 테스트 레벨이 포함된다는 것이다 [B15]. 여기서 다루는 대부분의 내용은 모든 인공지능기반 시스템에 적용할 수 있지만, 일부는 기계학습에 초점을 두고 있다.

7.2.1 입력 데이터 테스트

입력 데이터 테스트의 목적은 훈련과 예측을 위해 시스템이 사용하는 데이터의 품질이 최상인지 확인하는 것이다(4.3절 참조). 여기에는 다음이 포함된다:

- 리뷰
- 통계 기법 (예: 데이터 편향성 테스트)
- 훈련 데이터의 EDA
- 데이터 파이프라인의 정적 및 동적 테스트

데이터 파이프라인은 일반적으로 데이터 준비를 수행하는 여러 컴포넌트(4.1절 참조)로 구성되며, 이런 컴포넌트에 대한 테스트에는 컴포넌트 테스트와 통합 테스트가 모두 포함된다. 훈련을 위한 데이터 파이프라인은 운영 예측에 사용하는 데이터 파이프라인과 상당히 다를 수 있다. 운영에 사용하는 완전히 엔지니어링 및 자동화된 버전과 비교해 훈련을 위한 데이터 파이프라인은 일종의 프로토타입으로 간주할 수 있다. 이 때문에 이 두 가지 버전의 데이터 파이프라인에 대한 테스트는 많이 다를 수 있다. 다만 두 버전의 기능적 동등성을 테스트하는 경우도 고려해 봐야 한다.

7.2.2 기계학습 모델 테스트

기계학습 모델 테스트의 목적은 선택한 모델이 지정한 성능 기준을 충족하는지 확인하는 것이다. 여기에는 다음이 포함된다:

- 기계학습 기능적 성능 기준 (5.1 및 5.2절 참조)
- 훈련 속도, 예측 속도, 사용 컴퓨팅 자원, 적응성, 투명성 등과 같은 모델 자체를 대상으로 한 기계학습 비기능적 인수 기준

또한, 기계학습 모델 테스트는 기계학습 프레임워크, 알고리즘, 모델, 모델 설정, 초매개변수 등의 선택이 가능한 최적에 가까운지 판단하는 것도 목표로 한다. 필요하다면 기계학습 모델 테스트에 화이트박스 커버리지 기준 달성을 위한 테스트를 포함할 수 있다(6.2절 참조). 선정된 모델은 향후 다른 인공지능 또는 비 인공지능 컴포넌트와 통합된다.

7.2.3 컴포넌트 테스트

컴포넌트 테스트는 모델이 아닌 컴포넌트(non-model components, 예: 사용자 인터페이스나 커뮤니케이션 컴포넌트)에 적용할 수 있는 일반적인 테스트 레벨이다.

7.2.4 컴포넌트 통합 테스트

컴포넌트 통합 테스트는 (인공지능과 비 인공지능 모두의) 시스템 컴포넌트가 올바르게 상호작용하는지 확인하기 위해 수행하는 일반적인 테스트 레벨이다. 이 테스트 레벨에서는 데이터 파이프라인에서 오는 입력을 모델이 예상한 대로 수신하고 모델이 생성한 모든 예측이 관련 시스템 컴포넌트(예: 사용자 인터페이스)에 전달되고 올바르게 사용되는지 테스트한다. 인공지능이 서비스로 제공되는 경우 (1.7절 참조), 컴포넌트 통합 테스트의 일부로 제공되는 서비스의 API 테스트를 수행하는 것이 일반적이다.

7.2.5 시스템 테스트

시스템 테스트는 통합된 컴포넌트의 전체 시스템(인공지능과 비 인공지능 모두)이 운영 환경과 유사하게 구현된 테스트 환경에서 기능 및 비기능적 관점에서 예상한 대로 동작하는지 확인하기 위해 수행하는 일반적인 테스트 레벨이다. 시스템에 따라 이 테스트는 예상 운영 환경에서의 필드 테스트나 시뮬레이터 내에서의 테스트(예: 테스트 시나리오가 위험하거나 운영 환경에서 복제하기 어려운 경우) 형식을 취할 수 있다.

시스템 테스트 중에 기계학습 기능적 성능 기준을 다시 테스트함으로써 모델이 전체 시스템 내에 포함될 때 초기 기계학습 모델 테스트의 결과에 부정적인 영향이 있었는지 확인해야 한다. 이 테스트는 인공지능 컴포넌트가 의도적으로 변경된 경우(예: 크기를 줄이기 위해 DNN을 압축하는 경우) 특히 중요하다.

시스템 테스트는 시스템에 대한 많은 비기능적 요구사항 다수를 테스트하는 레벨이기도 하다. 예를 들어, 강건성 테스트를 위해 적대적 테스트를 수행할 수 있고 시스템의 설명 가능성을 테스트할 수도 있다. 필요한 경우 시스템 테스트 중에 하드웨어 컴포넌트와의(예: 센서) 인터페이스를 테스트할 수도 있다.

7.2.6 인수 테스트

인수 테스트는 일반적인 테스트 레벨이며, 전체 시스템이 고객에게 받아들여질 수 있는지의 여부를 결정

다. 인공지능기반 시스템의 경우 인수 기준의 정의가 어려울 수 있다(8.8절 참조). 인공지능이 서비스로 제공되는 경우(1.7절 참조) 의도된 시스템에 대한 서비스의 적합성과 기계학습 기능적 성능과 같은 기준이 충분히 달성되었는지 결정하기 위해 인수 테스트가 필요할 수 있다.

7.3 인공지능기반 시스템 테스트를 위한 테스트 데이터

Test Data for Testing AI-based Systems

정확하고 테스트 대상 시스템(SUT, system under test)에 따라 테스트 데이터 수집이 어려울 수 있다. 인공지능 기반 시스템의 테스트 데이터를 처리하는 데에는 다음과 같은 여러 잠재적 문제가 있다:

- (많은 양의, 속도가 빠르고, 다양성이 높은 데이터인) 빅 데이터는 생성과 관리가 어려울 수 있다. 예를 들어, 많은 양의 이미지와 오디오를 빠른 속도로 소비하는 시스템에 대해 대표성을 가진 테스트 데이터 생성은 상당히 어려울 수 있다.
- 입력 데이터는 시간이 지남에 따라 변경이 필요할 수 있으며, 특히 실제 이벤트를 나타내는 경우에 더욱 그렇다. 예를 들어, 얼굴 인식 시스템 테스트를 위해 기록한 사진은 실제로 몇 년에 걸친 인간의 노화 과정을 나타내기 위해 “노화(aged)”할 필요가 있을 수 있다.
- 개인 데이터 또는 기타 기밀 데이터는 삭제, 암호화, 수정 작업을 위해 특별한 기술이 필요하며, 사용에 대한 법적 승인이 필요할 수 있다.
- 테스터가 데이터 과학자와 동일한 방식으로 데이터 수집과 전처리를 하는 경우, 이런 단계의 결함이 식별되지 않을 수 있다.

7.4 인공지능기반 시스템의 자동화 편향성에 대한 테스트

Testing for Automation Bias in AI-Based Systems

인간의 의사결정에 도움을 주는 인공지능기반 시스템도 있다. 그러나 때때로 인간은 이런 시스템을 지나치게 신뢰하는 경향이 있다. 이런 잘못된 신뢰를 자동화 편향성 또는 안심(complacency) 편향성이라고 하며 다음처럼 두 가지 형태를 취한다:

- 자동화/안심 편향성의 첫 번째 형태는 시스템이 제공하는 권고 사항을 인간이 수용하고, 다른 출처(본인 포함)의 입력을 고려하지 않는 경우이다. 예를 들어, 사람이 양식에 데이터를 입력하는 과정은 기계

학습을 사용해 양식을 미리 채우고 사람이 이 데이터의 유효성을 검사해 개선할 수 있다. 이런 형태의 자동화 편향성은 일반적으로 의사 결정의 품질을 5% 감소시키는 것으로 나타나지만, 이는 시스템 상황에 따라 훨씬 더 클 수 있다[B16]. 마찬가지로 입력된 텍스트(예: 휴대 전화 메시지)의 자동 수정은 오류가 있는 경우가 많고 의미를 변하게 할 수도 있다. 사용자는 이것을 알아차리지 못하고 오류를 수정하지 않는 경우가 많다.

- 자동화/안심 편향성의 두 번째 형태는 인간이 시스템을 적절하게 모니터링하지 않아 시스템 오류를 놓치는 경우이다. 예를 들어, 반자율주행 차량은 점점 더 자율주행에 가까워지고 있지만 사고가 임박했을 때 여전히 사람에게 의존하고 있다. 일반적으로 차량 탑승자는 시스템의 차량 제어 능력을 지나치게 신뢰하게 되고 주의를 덜 기울이기 시작한다. 이는 필요할 때 적절하게 대응할 수 없는 상황으로 이어질 수 있다.

두 시나리오 모두에서 테스터는 인간의 의사 결정이 어떻게 손상될 수 있는지 이해하고, 시스템이 제공하는 권고/권장의 품질과 이에 상응해 대표적인 사용자가 제공하는 입력의 품질 모두를 테스트해야 한다.

7.5 인공지능 컴포넌트의 문서화 Documenting an AI Component

인공지능 컴포넌트의 문서화를 위한 일반적인 내용은 다음과 같다:

- 일반: 식별자, 설명, 개발자 세부정보, 하드웨어 요구사항, 라이선스 세부정보, 버전, 날짜, 담당자
- 설계: 가정 및 기술적 결정
- 운용: 1차 및 2차 유스케이스, 일반적인 사용자, 자가학습 접근법, 알려진 편향성, 윤리적 문제, 안전 문제, 투명성, 의사결정 임계값, 플랫폼, 컨셉 드리프트
- 데이터 세트: 특성, 수집, 가용성, 전처리 요구사항, 사용, 내용, 라벨링, 크기, 개인정보 보호, 보안, 편향/공정성, 제한/제약사항
- 테스트: 테스트 데이터 세트(설명 및 가용성), 테스트의 독립성, 테스트 결과, 강건성 테스트 접근법, 설명 가능성, 컨셉 드리프트, 이식성
- 훈련 및 기계학습 기능적 성능: 기계학습 알고리즘, 가중치, 검증 데이터 세트, 기계학습 기능적 성능 측정지표 선택, 기계학습 기능적 성능 측정지표에 대한 임계값, 실제 기계학습 기능적 성능 측정지표 값

명확한 문서는 인공지능기반 시스템 구현에 대한 투명성을 제공해 테스트를 개선하는 데 도움이 된다. 테스트에서 중요한 주요 문서 내용은 다음과 같다:

- 시스템의 목적, 기능 및 비기능적 요구사항 명세. 이런 유형의 문서는 일반적으로 테스트 베이스의 일부가 된다.
- 서로 다른 인공지능 및 비 인공지능 컴포넌트가 상호작용하는 방식을 설명하는 아키텍처 및 설계 정보. 이는 통합 테스트의 목표 식별을 지원하고 시스템 구조의 화이트박스 테스트를 위한 베이스를 제공할 수 있다.
- 운영 환경 명세. 이는 시스템의 자율성, 유연성, 적응성 테스트를 할 때 필요하다.
- 관련 메타데이터를 포함한 모든 입력 데이터의 출처. 이는 다음 측면을 테스트할 때 명확히 이해해야 한다:
 - 신뢰할 수 없는 입력의 기능적 정확성
 - 명시적 또는 암묵적인 표본 편향성
 - 유연성 (자율-학습 시스템의 저품질 데이터 입력으로 인한 잘못된 학습 등)
- 시스템이 운영 환경에서 변화에 적응할 것으로 예상되는 방법. 이것은 적응성 테스트의 테스트 베이스로 필요하다.
- 예상되는 시스템 사용자의 세부 정보. 이것은 테스트가 대표성을 가지는 데 필요하다.

7.6 컨셉 드리프트 테스트 Testing for Concept Drift

운영 환경은 시간이 지남에 따라 변하는 데 훈련된 모델은 그에 맞춰 변하지 않을 수 있다. 이 현상을 컨셉 드리프트라고 하며 일반적으로 모델 결과의 정확도와 유용성을 점점 더 떨어지게 한다. 예를 들어, 마케팅 캠페인의 영향으로 시간이 지남에 따라 결국 잠재 고객의 행동에 변화가 나타날 수 있다. 이런 변화는 시스템 외부의 문화적, 도덕적, 사회적 변화의 영향에서 기인한 시기적 또는 갑작스런 변화일 수 있다. 이런 급격한 변화의 예로 COVID-19 대유행이 판매 예측 및 주식 시장에 사용되는 모델의 정확도에 미치는 영향을 들 수 있다.

컨셉 드리프트가 발생하기 쉬운 시스템에 대해선 합의된 기계학습 기능적 성능 기준에 맞춰 정기적으로 테스트함으로써 문제를 완화하기 충분한 시점에 컨셉 드리프트의 발생을 감지할 수 있도록 해야 한다. 일반적인 완화 방법으로 시스템의 사용 중지, 시스템 재훈련 등을 들 수 있다. 재훈련의 경우 최신 훈련 데이터로 수행하며 확인 테스트, 리그레션 테스트 그리고 특정 유형의 A/B 테스트(9.4절 참조) 등을 수행할 수 있다. A/B 테스트의 경우 업데이트된 B 시스템이 원래의 A 시스템보다 높은 성능을 보여야 한다.

7.7 기계학습 시스템을 위한 테스트 접근법 선택

Selecting a Test Approach for an ML System

인공지능기반 시스템은 일반적으로 인공지능과 비 인공지능 컴포넌트를 모두 가지고 있다. 테스트 접근법은 이런 시스템에 대한 리스크 분석을 기반으로 선정되며, 전통적 테스트 외에도 인공지능 컴포넌트와 인공지능기반 시스템 특화 요소를 다루기 위해 좀 더 특수한 테스트를 포함하게 된다.

다음 목록은 기계학습 시스템 관련 몇 가지 일반적인 리스크와 그에 상응하는 완화 방법을 나열하고 있다. 이 목록에는 일부 예제만 제공하며, 테스트로 완화해야 하는 기계학습 시스템 관련 리스크는 이 외에도 많은 점을 유념해야 한다.

리스크	설명 및 완화 방법 예시
데이터 품질이 예상보다 낮을 수 있다.	이 리스크는 여러 방식으로 문제가 될 수 있으며 각각 다른 방식으로 예방할 수 있다(4.4절 참조). 일반적인 완화 방법으로는 리뷰, EDA, 동적 테스트 등이 있다.
운용 데이터 파이프라인에 결함이 있을 수 있다.	이 리스크는 개별 파이프라인 컴포넌트에 대한 동적 테스트와 전체 파이프라인의 통합 테스트를 통해 부분적으로 완화할 수 있다.
모델 개발에 사용하는 기계학습 워크플로가 최적이지 아닐 수 있다 (3.2절 참조).	이 리스크는 다음과 같은 이유로 발생할 수 있다: • 선택한 기계학습 워크플로에 대한 사전 협의 부족 • 잘못된 워크플로 선택 • 데이터 엔지니어가 워크플로를 따르지 않음 전문가와의 리뷰를 통해 잘못된 워크플로를 선택할 가능성을 완화할 수 있고, 더 많은 직접 관리 또는 감사를 통해 워크플로에 관한 합의 및 구현 문제를 해결할 수 있다.
기계학습 프레임워크, 알고리즘, 모델, 모델 설정 및 초매개변수의 선택이 최적이지 아닐 수 있다.	이 리스크는 의사결정자의 전문지식 부족이나 기계학습 워크플로의 검증 및 튜닝(또는 테스트) 단계의 잘못된 구현으로 발생할 수 있다. 전문가와의 리뷰를 통해 잘못된 결정을 내릴 가능성을 완화할 수 있으며, 더 철저한 관리를 통해서 워크플로의 검증 및 튜닝 (또는 테스트) 절차를 따르도록 할 수 있다.
원하는 기계학습 기능적 성능 기준을 기계학습 컴포넌트가 독립적으로는 충족하지만, 운영상에서는 기준을 충족하지 않을 수 있다.	이 리스크는 고립되어 모델을 훈련 및 테스트하는 데 사용한 데이터 세트가 운영상 접하게 되는 데이터를 대표하지 않기 때문일 수 있다. 전문가(또는 사용자)에 의해 선정된 데이터 세트 리뷰를 통해 선택한 데이터가 대표성이 없을 가능성을 완화할 수 있다.

리스크	설명 및 완화 방법 예시
원하는 기계학습 기능적 성능 기준을 충족하지만, 사용자가 제공된 결과에 만족하지 않을 수 있다.	이 리스크는 잘못된 성능 기준을 선택했기 때문일 수 있다 (예: 높은 정밀도가 필요한 경우에 재현율을 선택함). 전문가와의 리뷰를 통해 잘못된 기계학습 기능적 성능 측정지표를 선택할 가능성을 완화할 수 있으며, 경험기반 테스트를 통해서도 부적절한 기준을 식별할 수 있다. 이런 리스크는 컨셉 드리프트에 의해 발생할 수 있으며, 이 경우 운영 시스템을 더 자주 테스트함으로써 리스크를 완화할 수 있다.
원하는 기계학습 기능적 성능 기준을 충족하지만, 사용자가 제공된 서비스에 만족하지 않을 수 있다.	이 리스크는 시스템의 비기능적 요구사항을 충분히 고려하지 않았기 때문에 발생할 수 있다. 인공지능기반 시스템에 관한 품질 특성은 ISO/IEC 25010에 열거된 품질 특성 외에도 많다는 점을 유의해야 한다(2장 참조). 리스크 기반 접근법을 통해 품질 특성의 우선순위를 정하고 관련 비기능적 테스트의 수행을 통해 이런 리스크를 완화할 수 있다. 다르게는, 문제가 시스템 테스트 중에 경험기반 테스트를 통해 파악할 수 있는 여러 요인의 조합으로 발생할 수 있다. 8장에서는 이런 특성을 테스트하는 방법에 대한 지침을 제공한다.
자가학습 시스템이 사용자가 기대하는 서비스를 제공하지 못할 수 있다.	이 리스크는 다음과 같은 다양한 이유로 발생할 수 있다: • 시스템이 자가학습에 사용하는 데이터가 부적절 할 수 있다. 이 경우 전문가의 리뷰를 통해 문제가 있는 데이터를 식별할 수 있다. • 자가학습한 새로운 기능의 성능이 부족해 시스템에 장애가 발생할 수 있다. 이는 이전 기능과의 성능 비교 등 자동화된 리그레션 테스트를 통해 완화될 수 있다. • 시스템이 사용자가 예상하지 못한 방식으로 학습할 수 있으며, 이는 경험기반 테스트를 통해 식별할 수 있다.
사용자가 시스템의 의사결정 방법을 이해하지 못해 좌절할 수 있다.	이 리스크는 설명 가능성, 해석 가능성 및 투명성의 부족으로 발생할 수 있다. 이런 특성을 테스트하는 방법에 대한 자세한 내용은 8.6절을 참조하라.
사용자의 데이터가 훈련 데이터와 유사할 때는 모델이 우수한 예측을 제공하지만, 그렇지 않으면 좋지 않은 결과를 제공한다는 것을 발견할 수 있다.	이 리스크는 과적합(3.5.1절 참조)으로 인해 발생할 수 있으며, 이는 훈련 데이터 세트와 완전히 독립적인 데이터 세트를 사용해 모델을 테스트하거나 경험기반 테스트를 수행해 식별할 수 있다.

제8장 인공지능 특유 품질 특성 테스트 Testing AI-Specific Quality Characteristics

150분

Keywords

Test oracle

AI-Specific Keywords

Algorithmic bias, autonomous system, autonomy, expert system, explainability, inappropriate bias, interpretability, LIME method, ML training data, non-deterministic system, probabilistic system, sample bias, self-learning system, transparency

학습 목표

8.1 자가학습 시스템 테스트의 어려움

AI-8.1.1 (K2) 인공지능기반 시스템의 자가학습으로 인한 테스트의 어려움을 설명할 수 있다.

8.2 인공지능기반 자율 시스템 테스트

AI-8.2.1 (K2) 인공지능기반 자율 시스템을 테스트하는 방법을 설명할 수 있다.

8.3 알고리즘, 샘플, 부적절한 편향성 테스트

AI-8.3.1 (K2) 인공지능기반 시스템에서 편향성을 테스트하는 방법을 설명할 수 있다.

8.4 확률적 및 비 결정적 인공지능기반 시스템 테스트의 어려움

AI-8.4.1 (K2) 인공지능기반 시스템의 확률적 및 비 결정적 특성으로 인한 테스트의 어려움을 설명할 수 있다.

8.5 복잡한 인공지능기반 시스템 테스트의 어려움

AI-8.5.1 (K2) 인공지능기반 시스템의 복잡성으로 인한 테스트의 어려움을 설명할 수 있다.

8.6 인공지능기반 시스템의 투명성, 해석 가능성, 설명 가능성 테스트

AI-8.6.1 (K2) 인공지능기반 시스템의 투명성, 해석 가능성, 설명 가능성을 어떻게 테스트할 수 있는지 설명할 수 있다.

HO-8.6.1 (H2) 테스터가 설명 가능성(explainability)을 어떻게 활용할 수 있는지 보여주는 도구를 사용한다.

8.7 인공지능기반 시스템의 테스트 오라클

AI-8.7.1 (K2) 인공지능기반 시스템 특유의 특성으로 인한 테스트 오라클 생성의 어려움을 설명할 수 있다.

8.8 테스트 목적과 인수 기준

AI-8.8.1 (K4) 주어진 인공지능기반 시스템의 인공지능 특유 품질 특성에 대한 적절한 테스트 목적과 인수 기준을 선택할 수 있다.

8.1 자가학습 시스템 테스트의 어려움 Challenges Testing Self-Learning Systems

자가학습 시스템을 테스트할 때 극복해야 할 잠재적인 문제는 다음과 같다(이런 시스템에 대한 자세한 내용은 2장 참조):

- 예기치 않은 변경: 일반적으로 시스템의 초기 동작을 명시한 요구사항과 제약 조건은 있지만, 시스템 자체에서 일어나는 변경에 대해 알려진 정보는 부족하거나 아예 없을 수 있다. 정의된 요구사항과 설계에 (그리고 특정 제약 조건에) 대해 테스트하는 것은 일반적으로 가능하지만, 시스템이 혁신적인 구현 방법을 찾아냈거나 새로운 해결책을 고안한 (해결책의 구현 방법은 알 수 없는) 경우 이 새로운 구현에 적합한 테스트를 설계하는 것은 어려울 수 있다. 또한, 시스템이 스스로 (그리고 연관 출력도) 바뀔 경우, 이전에 합격한 테스트의 결과도 바뀔 수 있다. 이런 점이 테스트 설계를 어렵게 한다. 한 가지 해결 방법은 시스템의 동작 변화에도 관련성을 유지하는 적절한 테스트를 설계해 잠재적인 리그레션 테스트 문제를 방지하는 것이다. 그리고 관찰된 새로운 시스템 동작을 기반으로 새로운 테스트를 설계해야 할 수 있다.
- 복잡한 인수 기준: 시스템이 스스로 학습할 때 개선에 대한 기대치를 정의해야 한다. 예를 들어, 시스템이 자체적으로 변하는 경우 전반적인 기능적 성능이 향상되어야 한다고 가정할 수 있다. 단순히 “개선”되어야 한다는 점 외에 추가적인 항목을 고려하게 되면 급속하게 복잡해질 수 있다. 예를 들어, (단순하게 개선되지만 하면 되는 것이 아니라) 최소한의 정도를 명시하거나 개선이 환경적 요인과 연관되어 일어나도록 할 수도 있다(예: 환경 요인 F가 Y 이상으로 바뀌는 경우 기능 X가 최소 10% 개선되어야 함). 이런 문제는 더 복잡한 인수 기준에 대한 명세와 테스트를 통해, 그리고 시스템 기준 기능적 성능 기록을 지속적으로 유지/관리함으로써 해결할 수 있다.
- 테스트 시간 부족: 다양한 시나리오에서 시스템이 얼마나 빨리 학습하고 적응할 것으로 기대되는지 알아야 한다. 이런 인수 기준을 명시하고 정의하는 것은 어려울 수 있다. 시스템이 빠르게 적응하는 경우 매번 변경사항이 발생한 이후 새로운 테스트를 수동으로 실행할 시간이 부족할 수 있으므로, 시스템이 변경될 때 자동으로 실행될 수 있는 테스트를 작성해야 한다. 이런 문제는 적절한 인수 기준(8.8절 참조) 명세와 자동화된 반복 테스트를 통해 해결할 수 있다.
- 자원 요구사항: 시스템 요구사항은 자가학습 또는 적응을 수행할 때 시스템이 사용해도 되는 자원에 대한 인수 기준을 포함한다. 예를 들어, 여기에는 개선을 위해 사용할 수 있는 프로세싱 시간과 메모리 양이 포함될 수 있다. 또한, 이런 자원 사용이 측정 가능한 기능이나 정확성 개선 정도와 연결되어야 하는지의 여부를 고려해야 한다. 이런 어려움은 인수 기준의 명세화에 영향을 준다.

- 운영 환경의 불충분한 명세: 자가학습 시스템은 환경으로부터 수신하는 입력이 예상 범위를 벗어나거나 훈련 데이터에 반영되지 않았을 경우 바뀔 수 있다. 이런 입력은 데이터 중독의 형태를 가진 공격일 수도 있다(9.1.2절 참조). 사용할 수 있는 운영 환경과 나타날 수 있는 환경 변화 모두를 예측하는 것이 어렵기 때문에 대표성을 가지는 테스트 케이스와 환경 요구사항 세트를 식별하는 것은 어려울 수 있다. 이상적인 경우라면 시스템 사용이 예상되는 운영 환경의 가능한 모든 범위안에서의 변경을 인수 기준으로 정의할 것이다.
- 복잡한 테스트 환경: 모든 잠재적인 고위험 운영 환경 변화를 모방할 수 있도록 테스트 환경을 관리하는 것은 어려운 일이며, 테스트 도구(예: 결함 주입 도구) 사용이 필요할 수 있다. 운영 환경의 특성에 따라 입력 및 센서를 조작하거나 시스템을 테스트할 수 있는 다양한 물리적 환경을 확보해 테스트할 수 있다.
- 바람직하지 않은 동작 수정: 자가학습 시스템은 입력을 기반으로 행동을 수정하며, 테스터가 이런 현상을 방지하는 것은 불가능할 수 있다. 예를 들어, 제3자 시스템을 사용 중이거나 생산 시스템을 테스트하는 경우 발생할 수 있다. 동일한 테스트를 반복하게 되면 자가학습 시스템은 해당 테스트 관련 반응에 익숙해질 수 있으며, 이는 시스템의 장기적인 동작에 영향을 미칠 수 있다. 그러므로 테스터가 자가학습 시스템의 동작에 부정적인 영향을 주는 상황을 방지하는 것이 중요하다. 이것은 테스트 케이스 설계 및 테스트 관리에서도 어려움이 된다.

8.2 자율 인공지능기반 시스템 테스트 Testing Autonomous AI-Based Systems

자율 시스템은 사람의 개입이 필요한 시점과 필요하지 않은 시점을 결정할 수 있어야 한다. 따라서 인공지능 기반 시스템의 자율성을 테스트하려면 시스템이 이런 의사 결정을 수행할 수 있는 조건을 만들어야 한다. 자율성 테스트에서는 다음이 요구될 수 있다:

- 시스템이 제어를 포기해야 하는 특정 시나리오에서 시스템이 사람의 개입을 요청하는지 여부를 테스트한다. 이런 시나리오에는 운영 환경 또는 시스템의 자율성 범위를 초과하는 변화가 포함될 수 있다.
- 시스템이 지정된 시간 후에 제어를 포기할 때 사람의 개입을 요청하는지 테스트한다.
- 시스템이 자율적으로 동작해야 할 때 불필요하게 사람의 개입을 요청하는지 여부를 테스트한다.

이런 테스트에 필요한 조건 생성을 위해 운용 환경에 적용되는 경계값 분석을 사용하는 것이 도움이 될 수 있다. 자율성을 결정하는 변수가 운영 환경에서 어떻게 나타나는지 정의하고 자율성의 특성에 따라 달라지는 테스트 시나리오를 생성하는 것은 어려울 수 있다.

8.3 알고리즘, 샘플, 부적절한 편향성 테스트

Testing for Algorithmic, Sample and Inappropriate Bias

기계학습 시스템은 부적절한 편향성 제거를 위해 취해진 다양한 편향성과 조치를 기준으로 평가되어야 한다. 여기에는 부적절한 편향성에 대응하기 위한 의도적인 긍정적 편향성의 주입이 포함될 수 있다.

독립적인 데이터 세트를 사용한 테스트는 편향성을 식별할 수 있도록 하는 경우가 많다. 그러나 기계학습 알고리즘은 관련성 없어 보이는 기능들의 조합을 통해 원하지 않는 편향성을 만들어 낼 수 있으므로 편향성을 유발하는 모든 데이터를 식별하기는 어려울 수 있다.

인공지능기반 시스템은 알고리즘 편향성, 샘플 편향성, 부적절한 편향성에 대해 테스트되어야 한다(2.4절 참조). 여기에는 다음이 포함될 수 있다:

- 알고리즘 편향성 식별을 위해 모델의 훈련, 검증, 튜닝 활동 중에 분석한다.
- 샘플 편향성 식별을 위해 훈련 데이터 출처 및 획득 프로세스를 리뷰한다.
- 기계학습 워크플로 중 데이터 전처리 과정에서 샘플 편향성을 일으킬 수 있는 방식으로 데이터가 처리됐는지 리뷰한다.
- 시스템 입력의 변화가 많은 상호작용을 거치면서 시스템 출력에 어떤 영향을 미치는지를 측정하고, 시스템이 부적절하게 편향을 가질 수 있는 사람이나 객체의 집단을 기준으로 결과를 조사한다. 이것은 8.6절에서 논의하고 있는 LIME(Local Interpretable Model-Agnostic Descriptions) 방법과 유사하며, 출시 전 테스트 뿐만 아니라 생산 환경에서도 수행할 수 있다.
- 잠재적으로 편향성과 관련이 있을 수 있는 입력 데이터 속성에 대한 추가 정보를 수집하고, 이를 결과와 연관시킨다. 예를 들어, 어떤 집단에 속하는지가 편향성을 평가하는 데 영향을 주지만, 해당 정보가 모델에 입력되지 않는 상황처럼, 특정 집단에 속한 사람에 대한 불합리한 편향성에 대한 테스트에서의 인구통계학 정보와 연관될 수 있다. 이는 편향성이 입력 데이터에 명시되지는 않지만, 알고리즘에 의해 추론되는 “숨겨진” 변수를 기반으로 할 수 있기 때문이다.

8.4 확률적 및 비 결정적 인공지능기반 시스템 테스트의 어려움

Challenges Testing Probabilistic and Non-Deterministic AI-Based Systems

대부분의 확률적 시스템은 비결정적이기도 하므로 이런 속성을 가진 인공지능기반 시스템은 일반적으로 다음과 같은 테스트의 어려움을 가진다:

- 사전 조건과 입력이 동일한 테스트에서 여러 유효한 결과를 얻을 수 있다. 이로 인해 기대 결과의 정의가 쉽지 않고 테스트에 여러 어려움이 발생할 수 있다:
 - 확인 테스트를 위해 테스트를 재사용하는 경우
 - 리그레션 테스트를 위해 테스트를 재사용하는 경우
 - 테스트의 재현성이 중요한 경우
 - 테스트가 자동화된 경우
- 테스터는 테스트 기대 결과로 특정값을 명시하기보단 테스트의 합격 여부에 대한 합리적인 판단이 필요하므로, 요구되는 시스템 동작에 대해 더 깊은 지식이 필요할 수 있다. 예를 들어, 테스터는 전통적인 시스템보다 더 세련된 기대 결과를 정의해야 할 수 있고, 이런 테스트 기대 결과로는 허용 오차가 존재할 수 있다(예: “실제 결과가 최적 해답의 3% 이내인가?”).
- 시스템의 확률적 특성으로 인해 테스트 결과가 하나의 값이 되기는 불가능한 경우, 테스터는 통계적으로 유효한 테스트 결과를 얻기 위해 테스트를 여러 번 실행해야 하는 경우가 많다.

8.5 복잡한 인공지능기반 시스템 테스트의 어려움

Challenges Testing Complex AI-Based Systems

인공지능기반 시스템은 사람이 수행하기에 매우 복잡한 작업을 구현하는 데 자주 사용된다. 이로 인해 테스터가 평상시처럼 기대 결과를 정할 수 없기 때문에 테스트 오라클 문제가 나타날 수 있다(8.7절 참조). 예를 들어, 인공지능기반 시스템은 대용량 데이터에서 패턴을 식별하는 데 자주 사용된다. 이런 시스템은 많은 분석 후에도 사람이 수동으로는 찾을 수 없는 패턴을 찾는 데 사용된다. 이런 시스템이 요구하는 동작을 충분히 이해해서 기대 결과를 도출하는 것은 어려울 수 있다.

인공지능기반 시스템의 내부 구조가 소프트웨어로 생성되어 인간이 이해하기 너무 복잡할 때도 비슷한 문제가 발생한다. 이는 인공지능기반 시스템을 블랙박스로서만 테스트할 수 있는 상황으로 이어진다. 내부 구조를 알 수 있는 경우에도 테스트에 필요한 정보를 추가로 알아내기는 힘들다.

인공지능기반 시스템의 복잡성은 확률적 결과를 제공하고, 본질적으로 비 결정적일 때 증가한다(8.4절 참조).

비 결정적 시스템이기 때문에 발생하는 문제는 인공지능기반 시스템이 각각의 확률적 결과를 제공하는 여러 컴포넌트로 구성될 때 더욱 커진다. 예를 들어, 안면 인식 시스템은 이미지 내의 얼굴 식별을 위해 하나의 모델을 사용하고, 어떤 얼굴이 식별되었는지를 판단하기 위해 또 다른 모델을 사용할 수 있다. 인공지능 컴포넌트 간의 상호작용은 복잡하고 이해하기 어렵기 때문에, 모든 리스크를 파악하고 시스템을 적절하게 검증하기 위한 테스트의 설계를 어렵게 만든다.

8.6 인공지능기반 시스템의 투명성, 해석 가능성, 설명 가능성 테스트 Testing the Transparency, Interpretability and Explainability of AI-Based Systems

시스템 구현 방법에 대한 정보는 시스템 개발자가 제공할 수 있다. 여기에는 훈련 데이터 출처, 라벨링 방법, 시스템 컴포넌트 설계법이 포함될 수 있다. 이 정보가 없는 경우 테스트 설계가 어려울 수 있다. 예를 들어, 훈련 데이터 관련 정보가 없는 경우 데이터의 잠재적인 공백을 식별하고 이런 공백의 영향을 테스트하는 것이 어려워진다. 이런 상황은 블랙박스 와 화이트박스 테스트에 비유할 수 있으며 비슷한 장단점이 있다. 투명성은 데이터와 알고리즘에 대해 문서화된 정보를 실제 구현과 비교하고 이들이 얼마나 일치하는지 판단해 테스트할 수 있다.

기계학습의 경우 전통적 시스템보다 특정 입력과 특정 출력 사이의 연결을 설명하는 것이 더 어려울 수 있다. 이런 낮은 수준의 설명 가능성은 출력을 생성하는 모델 자체가 코드(알고리즘)로 생성되고 사람이 동일한 문제에 대해 생각하는 방식을 반영하지 않는 경우가 많기 때문이다. 다양한 기계학습 모델은 다양한 수준의 설명 가능성을 제공하며, 설명 가능성과 테스트 용이성을 포함한 시스템 요구사항에 따라 선택되어야 한다.

설명 가능성을 이해하기 위한 한 가지 방법은 테스트 데이터에 섭동(perturbations) 적용 시 기계학습 모델을 대상으로 동적 테스트를 수행하는 것이다. 이런 방법으로 설명 가능성을 정량화하고 이에 대한 시각적 설명을 제공하는 기법도 존재한다. 이런 기법 중 일부는 모델에 상관없이 적용 가능하지만, 다른 방법들은 특정 유형의 모델에 대한 액세스 권한이 필요하다. 모델의 입력과 출력 사이의 관계를 더 잘 이해하는 데 탐색적 테스트를 사용할 수도 있다.

LIME 기법은 모델에 상관없이 적용 가능하며 동적으로 주입된 입력 섭동과 출력 분석을 사용해 테스터에게 입력과 출력 간의 관계에 대한 정보를 제공한다. 이는 모델의 설명 가능성을 확보하는 효과적인 방법이 될 수 있다. 그러나 이는 결정적인 이유보다는 출력이 나오게 된 이유를 제공하는 것으로 제한되며, 모든 유형의 알고리즘에 적용되지는 않는다.

인공지능기반 시스템의 해석 가능성은 대상이 누구인지에 따라 크게 달라진다. 근간 기술을 얼마나 잘 파악해야 하는지는 이해관계자마다 요구사항이 다를 수 있다.

이해관계자마다 역량이 다를 수 있고 원하는 것이 다를 수 있으므로, 해석 가능성과 설명 가능성에 대한 이해 수준을 측정하고 테스트하는 것은 어려울 수 있다. 많은 유형의 시스템에서 일반적인 이해관계자의 프로파일일을 식별하는 것 또한 어려울 수 있다. 이런 테스트는 일반적으로 사용자 설문 조사나 설문지 형태로 수행된다.

8.6.1 실습: 모델 설명 가능성

적절한 도구를 사용해 이전에 만든 모델을 대상으로 설명 가능성을 확인한다. 예를 들어, 이미지 분류 모델

이나 텍스트 분류 모델의 경우 LIME과 같은 모델 독립적인 방법이 적합할 수 있다.

도구를 사용해서 모델 결정에 대한 설명, 특히 입력의 특성이 출력에 어떻게 영향을 미치는지에 대한 설명을 도출해본다.

8.7 인공지능기반 시스템의 테스트 오라클 Test Oracles for AI-Based Systems

인공지능기반 시스템 테스트의 주요 문제는 기대 결과의 정의일 수 있다. 테스트 오라클은 테스트의 기대 결과를 정하는데 사용하는 출처이다[101]. 기대 결과를 정하는 데 어려움이 생기면 테스트 오라클 문제가 있다고 말하게 된다.

복잡하거나 비 결정적 또는 확률적 시스템의 경우, “실제 값(ground truth)” (즉, 인공지능기반 시스템이 예측하려고 하는 실제 결과)을 알지 못하면 테스트 오라클을 정의하기 어려울 수 있다. 이 “실제 값”은 테스트 오라클과 구별되는데, 테스트 오라클은 반드시 어떤 기대값을 제공할 필요는 없고, 시스템이 올바르게 동작하는지를 판단할 수 있는 메커니즘만 제공할 수도 있다.

인공지능기반 시스템은 진화할 수도 있고(2.3절 참조), 자가학습 시스템 테스트의 경우도(8.1절 참조) 시스템이 자체적으로 바뀔에 따라 기능적 기대 결과를 자주 업데이트해야 하므로 테스트 오라클 문제로 어려움을 겪을 수 있다.

효과적인 테스트 오라클을 얻기 힘든 또 다른 이유는 소프트웨어 동작의 정확성에 관한 판단이 대부분 주관적이라는 것이다. 서로 다른 사용자의 기대 수준이 다른 경우가 많고 선택한 단어나 말의 명확성에 따라 다른 결과를 얻을 수 있다는 점에서 가상 비서(예: 시리나 알렉사)를 이런 문제의 예로 들 수 있다.

범위나 허용 오차로 기대 결과를 정의할 수 있는 경우도 있다. 예를 들어, 자율주행 자동차의 정지 지점을 특정 지점으로부터 최대 얼마 이내의 거리로 정의할 수 있다. 전문가 시스템의 경우 전문가와의 논의를 통해 기대 결과를 판단할 수 있다 (전문가의 의견이라도 틀릴 수 있다는 점에 유의한다). 이런 상황에서 고려해야 할 몇 가지 중요한 요소는 다음과 같다:

- 인간 전문가는 각자 능력 수준이 다양하다. 참여 전문가는 최소한 시스템이 대체하려는 전문가만큼의 역량이 있어야 한다.
- 전문가들은 같은 정보에 대해서도 서로 동의하지 않을 수 있다.
- 인간 전문가는 그들이 해야 할 판단을 자동화하는 것에 찬성하지 않을 수 있다. 그런 경우 잠재적 산출물에 대한 그들의 평가는 이중 블라인드 처리되어야 한다(즉, 전문가나 산출물 평가자 모두 어떤 등급이 자동화된 것인지 알 수 없어야 한다).

- 인간은 반응에 주의를 기울일 가능성이 더 크다(예를 들어, “잘 모르겠지만...”과 같은 문구). 인공지능 기반 시스템이 이런 종류의 반응을 사용할 수 없는 경우 응답을 비교할 때 이 점을 고려해야 한다.

A/B 테스트(9.4절 참조), 백투백 테스트(9.3절 참조), 변성 테스트(9.5절 참조)과 같은 테스트 오라클 문제를 완화할 수 있는 테스트 기법이 존재한다.

8.8 테스트 목적과 인수 기준 Test Objectives and Acceptance Criteria

시스템에 대한 테스트 목표와 인수 기준은 식별된 제품 리스크를 기반으로 해야 한다. 이런 리스크는 필요한 품질 특성의 분석을 통해 식별할 수 있는 경우가 많다. 인공지능기반 시스템의 품질 특성은 전통적으로 ISO/IEC 25010[S06]에서 얘기하는 특성(즉, 기능적 적합성, 수행 효율성, 호환성, 유용성, 신뢰도, 보안, 유지 가능성, 이식성)을 포함하지만, 다음 사항도 고려해야 한다:

특성 (Aspect)	인수 기준 (Acceptance Criteria)
적응성	• 시스템이 환경 변화에 적응할 때 계속 올바르게 동작하고 비기능적 요구사항을 충족하는지 확인한다. 이것은 자동화된 리그레션 테스트로 구현할 수 있다. • 시스템이 환경 변화에 적응하는 데 걸리는 시간을 확인한다. • 시스템이 환경 변화에 적응할 때 사용되는 자원을 확인한다.
유연성	• 시스템이 초기 명세를 벗어난 상황에서 어떻게 대처하는지 고려한다. 이는 바뀐 운영 환경에서 실행되는 자동화된 리그레션 테스트로 구현될 수 있다. • 시스템이 새로운 상황을 관리하기 위해 자체 변경하는 데 걸리는 시간이나 사용되는 자원을 확인한다.
진화	• 시스템이 자체 경험을 통해 얼마나 잘 학습하는지 확인한다. • 데이터 프로파일이 변경될 때 시스템이 얼마나 잘 대처하는지 확인한다 (즉, 컨셉 드립트).
자율성	• 시스템이 완전히 자율적일 것으로 예상하는 운영 범위를 강제로 벗어났을 때 어떻게 반응하는지 확인한다. • 시스템이 완전히 자율적이어야 할 때 사람의 개입을 요청하도록 “설득”이 가능한지 확인한다.
투명성, 해석 가능성, 설명 가능성	• 알고리즘과 데이터 세트에 대한 접근 용이성을 검토하여 투명성을 확인한다. • 시스템 사용자 또는 실제 시스템 사용자가 없는 경우, 유사한 배경을 가진 사람들에게 질문을 통해 해석 가능성과 설명 가능성을 확인한다.

특성 (Aspect)	인수 기준 (Acceptance Criteria)
부적절한 편향으로부터의 자유	- 시스템이 편향성의 영향을 받을 가능성이 높은 경우, 편향성이 없는 독립적인 테스트 스위트를 사용하거나 전문 검토자를 통해 테스트를 수행할 수 있다. - 추정 변수에 대해 원하지 않는 편향성의 존재를 확인하기 위해 인구 조사 데이터와 같은 외부 데이터를 사용해 테스트 결과를 비교한다 (외부 타당성 테스트).
윤리	"신뢰할 수 있는 인공지능을 위한 윤리 지침[R22]"에서 얘기하는 주요 요구사항을 지원하는 "신뢰할 수 있는 인공지능에 대한 EC 평가 목록[R21]"과 같은 적절한 체크리스트로 시스템을 확인한다.
확률적 시스템 및 비 결정적 시스템	이 항목은 정확한 인수 기준을 가지고 평가할 수 없다. 올바르게 동작하는 경우 시스템은 동일한 테스트에 대해 조금씩 다른 결과를 낼 수 있다.
부작용	잠재적으로 해가 될 수 있는 부작용을 식별하고 시스템에 이런 부작용이 나타날 수 있는 테스트를 생성한다.
보상 해킹	테스트 중인 지능형 에이전트와 다른 방법으로 성공을 측정하는 독립적인 테스트를 통해 보상 해킹을 식별할 수 있다.
안전성	안전성은 신중하게 평가해야 한다. 필요하다면 가상 테스트 환경을 사용할 수 있다 (10.2절 참조). 시스템이 스스로에게 해를 끼치도록 강제하는 시도를 포함할 수 있다.

기계학습 시스템의 경우 기계학습 모델을 위한 기계학습 기능적 성능 측정지표를 지정해야 한다 (5장 참조).

제9장 인공지능기반 시스템 테스트 방법과 기법

245분

Methods and Techniques for the Testing of AI-Based Systems

Keywords

A/B testing, adversarial testing, back-to-back testing, error guessing, experience-based testing, exploratory testing, metamorphic relation (MR), metamorphic testing (MT), pairwise testing, pseudo-oracle, test oracle problem, tours

AI-Specific Keywords

Adversarial attack, adversarial example, data poisoning, ML system, trained model

학습 목표

9.1 적대적 공격과 데이터 오염

AI-9.1.1 (K2) 기계학습 시스템 테스트가 적대적 공격과 데이터 오염을 방지하는 데 어떻게 도움이 되는지 설명할 수 있다.

9.2 페어와이즈 테스트

AI-9.2.1 (K2) 인공지능기반 시스템에서 페어와이즈 테스트를 어떻게 사용하는지 설명할 수 있다.

HO-9.2.1 (H2) 페어와이즈 테스트를 적용해 인공지능기반 시스템의 테스트 케이스를 도출하고 실행한다.

9.3 백투백 테스트

AI-9.3.1 (K2) 인공지능기반 시스템에서 백투백 테스트를 어떻게 사용하는지 설명할 수 있다.

9.4 A/B 테스트

AI-9.4.1 (K2) 인공지능기반 시스템 테스트에 A/B 테스트를 어떻게 적용하는지 설명할 수 있다.

9.5 변성 테스트

AI-9.5.1 (K3) 인공지능기반 시스템 테스트에 변성 테스트를 적용할 수 있다.

HO-9.5.1 (H2) 변성 테스트를 적용해 주어진 시나리오에 대한 테스트 케이스를 도출하고 실행한다.

9.6 인공지능기반 시스템의 경험기반 테스트

AI-9.6.1 (K2) 인공지능기반 시스템 테스트에 경험기반 테스트를 어떻게 적용하는지 설명할 수 있다.

HO-9.6.1 (H2) 인공지능기반 시스템에 탐색적 테스트를 적용한다.

9.7 인공지능기반 시스템을 위한 테스트 기법 선택

AI-9.4.1 (K4) 주어진 시나리오에 대해 인공지능기반 시스템을 테스트할 때 적절한 테스트 기법을 선택할 수 있다.

9.1 적대적 공격 및 데이터 오염 Adversarial Attacks and Data Poisoning

9.1.1 적대적 공격

적대적 공격은 훈련된 모델이 잘못된 예측을 제공하도록 유효한 입력을 미묘하게 교란시키는 것이다. 적대적 샘플로 알려진 이런 교란된 입력은 스팸 이메일을 약간 수정해 스팸 필터가 정상적인 이메일로 인식하도록 한 것에서 시작되었다. 최근에는 이미지 분류기와 더 많이 연관되고 있다. 사람의 눈에 보이지 않는 몇 개의 픽셀을 변경하는 것만으로도 신경망을 설득해 높은 신뢰성을 가지고 이미지를 전혀 다른 물체로 분류하도록 바꿀 수 있다.

일반적으로 적대적 샘플은 전이가 가능하며[B17], 이는 하나의 기계학습 시스템을 속일 수 있는 적대적 샘플은 같은 작업을 수행하도록 훈련된 다른 기계학습 시스템도 속일 수 있다는 것을 의미한다. 두 번째 기계학습 시스템이 다른 데이터로 훈련되고 다른 아키텍처를 기반으로 할 때도 같은 적대적 샘플에 대해 잘못된 예측을 하는 경향이 있다.

화이트박스 적대적 공격은 공격자가 모델 훈련에 사용한 알고리즘, 모델 설정 및 매개변수를 알고 있는 경우이다 (합리적인 수준의 투명성이 있다). 공격자는 이 지식을 가지고 입력에 작은 변화를 주어 무엇이 모델 출력에 큰 영향을 주는지 모니터링해 적대적 샘플을 만들 수 있다.

블랙박스 적대적 공격의 경우 공격자는 모델을 탐색해 기능을 확인한 다음, 유사한 기능을 제공하는 복제 모델을 구축한다. 이 후 공격자는 화이트박스 접근법을 사용해 이 복제 모델에 대한 적대적 샘플을 식별한다. 적대적 샘플은 보통 전이가 가능하므로 같은 적대적 샘플이 원래 모델에도 유효한 경우가 많다.

복제 모델을 만들 수 없는 경우 대량의 자동 테스팅으로 다양한 적대적 샘플을 식별하고 결과를 관찰할 수 있다.

적대적 테스팅은 시스템이 미래에 실패하지 않도록 예방 조치를 위한 취약성 식별 목적으로 적대적 공격을 수행하는 것을 말한다. 식별한 적대적 샘플은 모델이 올바르게 인식할 수 있도록 훈련 데이터에 추가한다.

9.1.2 데이터 오염

데이터 오염 공격은 공격자가 훈련 데이터를 조작해 두 가지 결과 중 하나를 달성하는 공격이다. 공격자는 향후 침입이 용이하도록 백도어나 신경망 트로이 목마를 주입하는 경우도 있지만, 일반적으로 손상된 훈련 데이터(예: 잘못 레이블링된 데이터)를 사용해 훈련한 모델이 잘못된 예측을 제공하도록 유도하기 위해 이런 공격을 하는 경우가 더 많다.

오염 공격은 특정 상황에서 기계학습 시스템이 분류를 잘못하도록 하는 것을 목표로 하며, 서비스 거부 공격

(denial-of-service attack)과 같이 무차별적일 수도 있다. 오염 공격의 잘 알려진 사례는 마이크로소프트 테이 (Microsoft Tay) 챗봇의 손상으로 적은 양의 해로운 트위터 대화 내용이 피드백을 통해 시스템을 훈련한 후 오염된 대화를 하도록 한 것이다. 많이 사용되는 데이터 오염 공격은 스팸 필터 소프트웨어가 오류를 내도록 수백만 개의 스팸 이메일을 스팸이 아닌 것으로 허위 보고하는 방식을 사용한다. 데이터 오염에 대해 가장 우려스런 부분 중 하나는 널리 사용되는 공개 인공지능 데이터 세트의 감염 가능성이다.

오염된 데이터가 이상값으로 나타날 수 있으므로 EDA를 사용해 데이터 오염을 식별하는 검증이 가능하다. 또한, 데이터 수집 정책을 검토해 훈련 데이터의 출처를 검증할 수도 있다. 운영 중인 기계학습 시스템이 오염된 데이터로 공격을 받을 수 있는 경우, A/B 테스트(9.4절 참조)으로 업데이트된 시스템이 이전 버전과 비교해서 크게 벗어나지 않았는지 확인할 수 있다. 또는 신뢰할 수 있는 테스트 스위트로 업데이트된 시스템의 리그레션 테스트를 통해서도 시스템 감염을 확인할 수 있다.

9.2 페어와이즈 테스트 Pairwise Testing

인공지능기반 시스템에서 다루는 변수는 매우 많을 수 있다. 특히 시스템이 빅 데이터를 사용하거나 자율주행 자동차와 같이 외부와 상호작용할 때 더욱 그렇다. 완벽한 테스트를 위해서는 이런 변수의 모든 가능한 값으로 만든 모든 조합을 테스트해야 한다. 그러나 그러기 위해서는 실제로 무한한 수의 테스트가 필요하기 때문에 제한된 시간 내에 실행할 수 있는 하위 집합을 선택하는데 테스트 기법을 사용하게 된다.

각각 다양한 값을 가질 수 있는 여러 매개변수를 조합하는 것이 가능하지만, 조합 테스트를 적용해 테스트 스위트의 결합 식별 능력을 손상시키지 않으면서 필요한 테스트 케이스 수를 크게 줄일 수 있다. 조합 테스트 기법에는 몇 가지가 있다([102] 및 [S08] 참조). 그러나 실제로는 이해하기 쉽고 도구 지원이 풍부한 페어와이즈 테스트가 가장 널리 사용된다. 또한, 연구에 따르면 대부분 결합은 소수의 변수 간 상호작용으로 발생한다 [B33].

실제로는 조합 테스트를 사용하더라도 매우 광범위한 테스트 스위트가 만들어지는 시스템이 많아서 필요한 수의 테스트 수행을 위해서는 자동화 및 가상 테스트 환경(10.2 참조)이 필요하다. 예를 들어 자율주행 자동차의 경우, 시스템 테스트를 위한 상위 수준의 시나리오는 자동차가 운행될 것으로 예상되는 다양한 환경과 여러 차량 기능을 동시에 모두 실행해야 한다. 따라서 변수는 다양한 환경 제약 조건(예: 도로 유형 및 표면, 날씨 및 교통 상황, 가시성)과 다양한 자율주행 기능(예: 적응형 크루즈 컨트롤, 차선 유지 및 차선 변경)이 포함해야 한다. 이런 변수 외에 센서 입력도 다양한 수준으로 고려해야 한다(예: 비디오 카메라로부터의 입력은 주행이 진행되고 카메라가 더러워짐에 따라 품질이 떨어진다).

현재 자율주행 자동차와 같은 안전이 중요한 인공지능기반 시스템에 어느 만큼의 철저한 조합 테스트가 필

요한지에 대한 연구는 부족하다. 페어와이즈 테스트가 충분하지 않을 수 있으나, 이 기법이 결함을 찾는 데 효과적이라는 것은 잘 알려진 사실이다.

9.2.1 실습: 페어와이즈 테스트

최소 5개의 변수와 적어도 500개의 가능한 조합을 가진 인공지능기반 시스템을 대상으로 페어와이즈 테스트 도구를 사용해 선별된 페어와이즈 조합 세트를 식별하고 이런 조합으로 테스트를 수행해본다. 테스트한 페어와이즈 조합 수와 이론적으로 모든 가능한 조합의 테스트를 위해 필요했을 테스트 수를 비교해 본다.

9.3 백투백 테스트 Back-to-Back Testing

인공지능기반 시스템을 테스트할 때 나타나는 테스트 오라클 문제(8.7절 참조)에 대한 잠재적인 해결책 중 하나는 백투백 테스트를 사용하는 것이다. 이를 차등 테스트(differential testing)이라고도 한다. 백투백 테스트에서는 시스템의 대체 버전이 슈도 오라클(pseudo-oracle)로 사용되며, SUT(system under test, 테스트 대상 시스템)의 테스트 결과와 비교된다. 슈도 오라클은 기존 시스템일 수도 있고 다른 팀에 의해 개발된 시스템일 수도 있다. 이때 다른 팀에서 개발한 시스템은 다른 아키텍처와 다른 프로그래밍 언어로 다른 플랫폼에서 개발된 것일 수 있다. 기능적 적합성을 테스트할 때 (비기능적 요구사항을 확인할 때와는 달리) 슈도 오라클로 사용되는 시스템은 SUT와 동일한 비기능적 인수 기준을 달성하지 않아도 된다. 가령, 빠르게 실행할 필요가 없을 수 있고 때문에 빌드 비용이 훨씬 적게 들 수 있다.

기계학습에서는 다른 프레임워크, 알고리즘, 모델 설정을 사용해 기계학습 슈도 오라클을 만들 수 있다. 상황에 따라서는 인공지능이 아닌 전통적 소프트웨어를 사용해 슈도 오라클을 만들 수 있다.

슈도 오라클이 결함 식별에 효과적일 수 있기 위해서는 슈도 오라클과 SUT에 공통 소프트웨어가 없어야 한다. 그렇지 않으면 같은 결함이 양쪽에서 발생해 둘 다 문제가 있음에도 불구하고 테스트 결과가 일치할 수도 있다. 인공지능기반 시스템을 개발하는 데 성숙도가 낮고 재사용이 가능한 오픈소스 인공지능 소프트웨어가 너무 많이 사용되고 있어 슈도 오라클과 SUT 간의 코드 재사용은 슈도 오라클을 손상시킬 수 있다. 재사용 가능한 인공지능 솔루션에 대한 문서화 부족도 이런 문제가 발생하고 있는 것을 테스터가 인식하기 어렵게 만들 수 있다.

9.4 A/B 테스트 A/B Testing

A/B 테스트는 동일한 입력에 대해 프로그램의 서로 다른 변형(A와 B) 간의 반응을 비교해 두 변형 중 나은 쪽을 결정하는 방법이다. 이는 통계적 테스트 접근법으로, 보통 프로그램 간의 차이를 결정하기 위해서는 테스트 실행 결과를 여러 번 비교해야 한다.

이 기법의 간단한 예로 마케팅 대상 집단을 둘로 나눠 서로 다른 프로모션 제안을 이메일로 보내는 것을 생각해 볼 수 있다. 마케팅 대상의 절반은 A, 절반은 B 제안을 받고 각 제안의 반응을 향후 어떤 제안을 사용할지 결정하는 데 사용한다. 많은 전자 상거래와 웹 기반 기업이 운영에 A/B 테스트를 사용하며 특정 기준에 따라 소비자를 분류해 소비자 선호도를 파악하고 있다.

A/B 테스트는 테스트 오라클 문제를 해결하는 방법의 하나로 기존 시스템을 부분 오라클로 활용한다. A/B 테스트는 테스트 케이스는 만들지 않으며, 테스트 설계 방법에 대한 가이드라인을 제공하지 않지만, 운영 중 발생하는 입력이 테스트에 사용되는 경우는 많다.

A/B 테스트는 5장에서 설명한 것과 같이, 기계학습 기능적 성능 측정지표 등의 합의된 인수 기준이 있는 인공지능 기반 시스템의 업데이트를 테스트하는 데 사용할 수 있다. 시스템을 업데이트할 때마다 A/B 테스트는 업데이트된 변형이 이전 변형과 비교해 성능이 유사하거나 더 나은지 확인하는 데 사용한다. 이 접근법은 간단한 분류기에 사용할 수도 있지만, 훨씬 더 복잡한 시스템을 테스트하는 데에도 사용할 수 있다. 예를 들어, 스마트 시티 교통 경로 설정 시스템의 효율성 개선을 위한 업데이트를 A/B 테스트를 사용해 테스트할 수 있다 (예: 두 가지 시스템 변형에 대해 몇 주간 평균 통근 시간 비교).

A/B 테스트는 자가학습 시스템을 테스트하는 데에도 사용 가능하다. 시스템이 변경되면 자동화된 테스트가 실행되고 관련된 시스템 특성이 변경 전의 그것과 비교된다. 시스템이 개선됐다면 변경 사항이 승인되고 그렇지 않으면 시스템은 이전 상태로 복원된다.

A/B 테스트와 백투백 테스트의 주요 차이점은 A/B 테스트는 동일한 시스템의 두 가지 변형을 비교하기 위해 사용하고, 백투백 테스트는 결함을 식별하기 위해 사용한다는 점이다.

9.5 변성 테스트 Metamorphic Testing (MT)

변성 테스트[B18] 기법은 합격한 소스(source) 테스트 케이스를 기반으로 추가 테스트 케이스를 생성하는 것을 목표로 한다. 변성 관계(MR, metamorphic relation)를 기반으로 소스 테스트 케이스를 변경(변성)해 하나 이상의 후속 테스트 케이스를 만든다. 변성 관계는 테스트 대상이 아닌 기능 속성을 기반으로 하며, 테스트 케이

스 입력에 가해진 변경이 같은 테스트 케이스의 기대 결과에 어떻게 반영되는지를 설명한다.

예를 들어, 숫자 집합의 평균을 계산하는 프로그램을 생각해 보자. 일련의 숫자와 기대 평균으로 구성된 테스트 케이스를 생성하고 실행해 합격하는지 확인한다. 이제 프로그램의 평균 기능에 대해 알려진 내용을 기반으로 후속 테스트 케이스를 생성할 수 있다. 우선 평균을 내는데 사용한 숫자들의 순서만 바꿀 수도 있다. 평균 함수를 고려했을 때 기대 결과가 같을 것으로 예상할 수 있다. 따라서 새로운 기대 결과를 계산할 필요 없이 숫자의 순서로 다른 후속 테스트 케이스를 생성할 수 있다. 숫자 집합이 크면 숫자는 같고 순서만 다른 여러 숫자 세트를 만들 수 있으며, 이들을 각각 별도의 후속 테스트 케이스를 만드는 데 사용할 수 있다. 이런 모든 테스트 케이스는 같은 소스 테스트 케이스를 기반으로 하며 같은 기대 결과를 가질 것이다.

기대 결과가 소스 테스트 케이스의 원래 기대 결과와 달라지는 변성 관계와 후속 테스트 케이스도 많다. 예를 들어, 같은 평균 함수를 사용해 입력 세트의 각 수에 2를 곱한 변성 관계를 도출할 수 있다. 이렇게 만든 세트의 기대 결과는 원래 기대 결과에 단순히 2를 곱한 값이 될 것이다. 마찬가지로 이 변성 관계를 가지고 다른 값을 곱해서 잠재적으로 무한한 수의 후속 테스트 케이스를 만들 수 있다.

변성 테스트는 대부분의 테스트 대상에 사용할 수 있으며 기능과 비기능성 테스트 모두에 적용할 수 있다(예: 여러 목표 설정을 다루는 설치성 테스트에서 설치 관련 매개변수의 순서를 달리해 선택한다). 적은 비용으로 테스트 오라클 제작이 어려워 기대 결과 생성에 문제가 있는 경우에 특히 유용하다. 빅 데이터 분석을 기반으로 하거나 테스터가 기계학습 알고리즘의 예측 도출법을 정확하게 이해하기 어려운 인공지능 기반 시스템이 이에 해당할 수 있다. 인공지능 분야에서 변성 테스트는 이미지 인식, 검색 엔진, 경로 최적화, 음성 인식 등을 테스트 하는데 활용되고 있다.

위 설명처럼 변성 테스트는 소스 테스트 케이스를 기반으로 합격을 판단하지만, 소스 테스트 케이스 중 합격한 것이 있는지 확인할 수 없는 경우에도 유용하다. 예를 들어, 일부 인공지능 기반 시스템처럼 시스템이 너무 복잡한 기능을 구현해서 테스터가 해당 기능을 복제해 테스트 오라클로 사용할 수 없는 경우가 여기에 해당된다. 이 경우, 변성 테스트로 하나 이상의 테스트 케이스를 생성하고 실행했을 때 얻게 되는 결과 간의 관계 확인을 통해 유효성을 검증할 수 있다. 이런 형태의 변성 테스트를 사용하면 개별 테스트 결과가 맞는지 알 수는 없더라도, 결과 간의 관계가 올바르다면 프로그램에 대한 신뢰도가 높아질 수 있다. 예를 들어, 대규모 데이터 세트를 기반으로 사망 연령을 예측하는 인공지능 기반 보험심사 프로그램을 생각해 보자. 여기서는 흡연 횟수가 늘어나면 예측 사망 연령이 낮아져야 한다(최소한 그대로 유지돼야 한다).

변성 테스트는 1998년에 처음 제안된 비교적 최신 테스트 기법이다. 후속 테스트 케이스의 기대 결과가 특정한 값이 아니라 소스 테스트 케이스 기대 결과와의 상대적인 관계로 정의된다는 점에서 기존 테스트 기법과 다르다. 쉽게 이해할 수 있는 개념을 기반으로 하며, 해당 기법에 대한 경험이 부족해도 애플리케이션 도메인을 이해하는 테스터라면 적용할 수 있으며, 비용도 전통적 기법과 비슷하게 든다. 또한, 결함을 식별하는데 효과적

이다. 기존의 테스트 오라클[B19]을 기반으로한 기법을 사용해 검출할 수 있는 결함의 90% 이상을 3~6개의 서로 다른 변성 관계로 식별할 수 있다는 연구 결과도 있다[B19]. 잘 작성된 변성 관계와 소스 테스트 케이스에서 후속 테스트 케이스를 자동으로 생성할 수도 있다. 아직은 관련 상용 도구는 없지만 이미 구글은 그래픽퍼즈 (GraphicsFuzz) 도구를 사용해 안드로이드 그래픽 드라이버 테스트를 위해 자동 변성 테스트를 적용하고 있다. 그래픽퍼즈는 오픈소스로 공개돼 있다([R23] 참조).

9.5.1 실습: 변성 테스트

실습을 통해 다음과 같은 실질적인 경험을 해볼 수 있다:

- 주어진 인공지능기반 애플리케이션이나 프로그램에 대한 여러 변성 관계(MR)를 도출한다. 이런 변성 관계에는 소스나 후속 테스트 케이스의 기대 결과가 같은 경우와 서로 다른 경우가 포함돼야 한다.
- 인공지능기반 애플리케이션 또는 프로그램에 대한 소스 테스트 케이스를 작성한다. 이 테스트 케이스가 반드시 합격한다는 것을 보장할 필요는 없지만, “골드 스탠다드”가 없는 변성 테스트의 한계를 학생들에게 상기시켜야 한다.
- 도출한 변성 관계와 작성한 소스 테스트 케이스를 사용해 후속 테스트 케이스를 도출한다.
- 후속 테스트 케이스를 실행한다.

9.6 인공지능기반 시스템의 경험기반 테스트

Experience-Based Testing of AI-Based Systems

경험기반 테스트에는 오류 추정, 탐색적 테스트 그리고 체크리스트기반 테스트가 포함되며[101], 모두 인공지능기반 시스템의 테스트에 적용할 수 있다.

오류 추정은 일반적으로 테스트어의 지식, 보편적인 개발 오류, 유사 시스템(또는 이전 버전)에서의 오류를 기반으로 한다. 인공지능기반 시스템에 적용되는 오류 추정의 예로는 시스템적으로 편향된 훈련 데이터의 사용으로 기계학습 시스템이 과거에 실패한 원인 관련 정보를 사용하는 것을 들 수 있다.

탐색적 테스트에서 테스트는 반복적인 방식으로 설계, 생성, 실행되며 이전 테스트의 테스트 결과를 기반으로 후속 테스트를 도출할 수 있다. 탐색적 테스트는 명세가 충분하지 않거나 테스트 오라클 문제가 있을 때 특히 유용하다. 인공지능기반 시스템은 테스트 오라클 문제를 가지고 있는 경우가 많다. 결과적으로 탐색적 테스트는 이런 맥락에서 자주 사용되며, 변성 테스트(9.5 참조)과 같은 기법을 기반으로 하는 보다 체계적인 테스트를 보완하기 위해 사용된다.

둘러보기(tour)는 테스트가 특정 포커스 중심으로 구성된 탐색적 테스트 수행시 참조할 전략과 목표 세트를 지

칭하는 비유(metaphor) 표현이다[B20]. 인공지능기반 시스템의 탐색적 테스트를 위한 둘러보기는 일반적으로 기계학습 시스템의 편향성, 과소적합, 과적합 개념에 포커스를 맞출 수 있다. 가령, 모델을 테스트하는 데 데이터 둘러보기를 적용할 수 있다. 이 둘러보기에서 테스터는 훈련에 사용되는 데이터의 유형과 분포, 변형, 형식, 범위 등을 식별한 다음, 데이터 유형을 사용해 모델을 테스트할 수 있다.

기계학습 시스템은 훈련 데이터의 품질에 크게 의존하며 이미 존재하는 EDA 분야는 탐색적 테스트 접근법과 밀접한 관련이 있다. EDA는 데이터를 검사해서 패턴, 관계, 추세, 이상값을 찾는 것이다. 이 과정은 데이터에 대한 상호작용적인 가설 기반 탐색을 포함하며, [B21]에서 *“우리는 기대를 가지고 데이터를 탐색한다. 우리는 데이터에서 본 것을 기반으로 기대를 수정한다. 그리고 우리는 이 과정을 반복한다.”*로 설명하고 있다. EDA는 일반적으로 두 가지 영역에서 도구 지원을 필요로 한다. 하나는 분석가가 복잡한 데이터를 더 잘 이해할 수 있도록 하기 위한 데이터와의 상호작용에서, 다른 하나는 분석 결과를 쉽게 표시할 수 있도록 하기 위한 데이터 시각화에서 필요하다. 탐색적 기법 사용은 활용중인 기계학습 알고리즘의 검증, 효율적인 모델이 되도록 변경 사항 식별, 도메인 전문 지식 활용 등에 도움이 될 수 있다. 이런 과정은 주로 데이터 시각화를 통해 이루어진다 [B22].

구글은 선언문 형태로 작성된 28개의 기계학습 테스트를 가지고 있다. 이 테스트들은 데이터, 모델 개발, 인프라, 모니터링 영역을 다루며, 구글 내에서 기계학습 시스템에 대한 테스트 체크리스트로 사용된다[B23]. 구글에서 공개하고 있는 “기계학습 테스트 체크리스트”는 다음과 같다:

기계학습 데이터:

1. 기능 기대치는 스키마(schema)에 스캔되어야 한다.
2. 모든 기능은 유익해야 한다.
3. 비용이 너무 많이 드는 기능은 없어야 한다.
4. 기능은 메타레벨 요구사항을 준수해야 한다.
5. 데이터 파이프라인에는 적절한 개인 정보 보호 기능이 있어야 한다.
6. 새로운 기능을 빠르게 추가할 수 있어야 한다.
7. 모든 입력 특성 코드는 테스트되어야 한다.

모델 개발:

1. 모델 명세를 검토하고 제출해야 한다.
2. 온라인과 오프라인 측정지표는 상관 관계가 있어야 한다.
3. 모든 초매개변수는 튜닝되어야 한다.

4. 오래된 모델의 영향이 알려져 있어야 한다.
5. 더 단순한 모델이 더 나은 것은 아니다.
6. 주요 데이터 조각에 대한 모델 품질이 충분해야 한다.
7. 포함 여부의 고려를 위해 모델을 테스트해야 한다.

기계학습 인프라:

1. 훈련은 재현 가능해야 한다.
2. 모델 사양에 대한 단위 테스트가 수행되어야 한다.
3. 기계학습 파이프라인에 대한 통합 테스트가 수행되어야 한다.
4. 모델 품질은 배포 전에 검증되어야 한다.
5. 모델은 디버깅할 수 있어야 한다.
6. 모델은 배포 전에 카나리아 테스트를 통과해야 한다.^{주)}
7. 배포된 모델은 롤백할 수 있다.

테스트 모니터링:

1. 의존성이 변경되면 알림이 표시되어야 한다.
2. 입력에는 데이터 불변성이 유지되어야 한다.
3. 훈련과 배포는 편향되지 않아야 한다.
4. 모델이 너무 오래되지 않아야 한다.
5. 모델은 수치적으로 안정적이어야 한다.
6. 컴퓨팅 성능이 저하되지 않아야 한다.
7. 예측 품질이 저하되지 않아야 한다.

9.6.1 실습: 탐색적 테스트와 탐색적 데이터 분석(EDA)

선택한 모델과 데이터 세트에 대해 다양한 유형의 데이터와 다양한 매개변수에 대한 분포를 고려해 데이터 둘러보기를 수행한다.

누락된 데이터나 데이터의 잠재적 편향을 식별하기 위해 데이터에 대해 EDA를 수행한다.

주) 전체한테 배포하기 전에 일부 사용자/대상한테 먼저 배포해서 이상 없는지 확인하고 나서 전체한테 배포해야 한다

9.7 인공지능기반 시스템을 위한 테스트 기법 선택

Selecting Test Techniques for AI-Based Systems

인공지능기반 시스템은 일반적으로 인공지능과 비 인공지능 컴포넌트를 모두 포함하고 있다. 비 인공지능 컴포넌트 테스트를 위한 테스트 기법의 선택은 기존의 테스트와 거의 동일하다. 인공지능기반 컴포넌트의 경우 선택에 더 많은 제한이 있을 수 있다. 예를 들어, 테스트 오라클 문제가 예상되는 경우(즉, 기대 결과 생성이 어려운 경우), 식별된 리스크를 기반으로 다음을 사용해 이 문제를 완화할 수 있다:

- 백투백 테스트: 이를 위해서는 테스트 케이스가 있거나 만들 수 있어야 하고, 슈도 오라클 역할을 할 동등한 시스템이 필요하다. 리그레션 테스트의 경우 시스템의 이전 버전이 그 역할을 할 수 있다. 결함의 효과적인 식별을 위해서는 독립적으로 개발된 시스템이 필요할 수 있다.
- A/B 테스트: 운영 환경의 입력을 테스트 케이스로 사용하는 경우가 많으며, 주로 통계 분석을 통해 동일한 시스템의 두 변형을 비교하는 데 사용한다. A/B 테스트는 새로운 변형의 데이터 오염 여부를 확인하거나 자가학습 시스템의 자동화된 리그레션 테스트를 위해 사용할 수 있다.
- 변성 테스트: 경험이 부족한 테스터가 비용 효율적으로 결함을 식별할 때 사용할 수 있지만, 애플리케이션 도메인을 이해할 필요가 있다. 변성 테스트는 기대 결과가 절대적이지 않고, 대신 소스 테스트 케이스에 상대적이기 때문에 확정적인 결과를 제공하는 데는 적합하지는 않다. 상용 도구 지원은 현재 제공되지 않지만 많은 테스트를 수동으로 생성할 수 있다.

적대적 테스트는 일반적으로 적대적 사례를 잘못 처리하면 시스템에 심각한 영향을 미치거나 시스템이 공격받을 수 있는 기계학습 모델에 적합하다. 마찬가지로 데이터 오염 테스트도 시스템이 공격받을 수 있는 기계학습 시스템에 적합할 수 있다.

인공지능기반 시스템이 복잡하고 매개변수가 많으면 페어와이즈 테스트가 적합한 경우가 많다.

경험기반 테스트는 인공지능기반 시스템에 적합한 경우가 많다. 훈련에 사용할 데이터나 운영 데이터를 고려할 때 특히 유용할 수 있다. EDA는 사용 중인 기계학습 알고리즘을 검증하거나 효율성 향상을 위한 개선 사항을 식별하고 도메인 전문 지식을 활용하는 데 사용할 수 있다. 구글은 자신들이 배포하는 기계학습 테스트 체크리스트가 기계학습 시스템에 효과적인 접근법임을 발견했다.

신경망에 한정해서 봤을 때, 신경망 커버리지의 적용이 적합한 미션 크리티컬(mission critical) 시스템이 많으며 일부 커버리지 기준은 다른 것보다 더 엄격한 커버리지를 요구한다.

제10장 인공지능기반 시스템의 테스트 환경 Test Environments for AI-Based Systems	30분
--	------------

Keywords

Virtual test environment

AI-Specific Keywords

AI-specific processor, autonomous system, big data, explainability, multi-agent system, self-learning system

학습 목표

10.1 인공지능기반 시스템의 테스트 환경

AI-10.1.1 (K2) 인공지능기반 시스템의 테스트 환경과 전통적 시스템에 필요한 테스트 환경을 구분하는 주요 요소를 설명할 수 있다.

10.2 인공지능기반 시스템 테스트를 위한 가상 테스트 환경

AI-10.2.1 (K2) 인공지능기반 시스템 테스트에서 가상 테스트 환경이 제공하는 이점을 설명할 수 있다.

10.1 인공지능기반의 시스템 테스트 환경 Test Environments for AI-Based Systems

인공지능기반 시스템은 다양한 운영 환경에서 사용될 수 있으며, 이는 테스트 환경 역시 다양할 수 있다는 것을 의미한다. 테스트 환경을 전통적인 시스템과 다르게 만드는 인공지능기반 시스템의 특징은 다음과 같다:

- **자가학습:** 자가학습 시스템과 일부 자율 시스템은 시스템이 처음 배포되었을 때 완전히 정의되지 않았을 수 있는 운영 환경의 변화에 적응해야 한다(2.1절 참조). 때문에 이렇게 정의되지 않은 환경 변화를 모방하는 테스트 환경의 정의는 본질적으로 어려운 일이며, 테스터의 상상력과 테스트 환경에 내재한 일정 수준의 임의성이 모두 필요할 수 있다.
- **자율성(Autonomy):** 자율 시스템은 인간의 개입없이 환경 변화에 적응해야 하며, 자율성이 운영자에게 다시 양도되는 상황도 인식할 수 있어야 한다(2.2절 참조). 일부 시스템의 경우 자율성 양도 상황 식별 후 모방을 위해 시스템을 극단적인 상황에 몰아부치기 위한 테스트 환경이 필요할 수 있다. 일부 자율 시스템의 경우 이것은 위험한 환경에서 작업하는 것이 목적일 수 있으며, 이런 환경을 대변하는 위험한 테스트 환경을 구축하는 것은 쉬운 일이 아니다.
- **다중 에이전트(Multi-agent):** 다중 에이전트 인공지능기반 시스템이 다른 인공지능기반 시스템과 함께 동작해야 하는 경우 테스트 환경은 SUT(system under test)가 상호작용하는 인공지능기반 시스템의 비결정성(non-determinism)을 모방할 수 있도록 일정 수준의 비결정성을 가져야 한다.
- **설명 가능성(Explainability):** 일부 인공지능기반 시스템의 특성상 시스템이 결정을 어떻게 내렸는지 이해하는 것은 어려울 수 있다(2.7절 참조). 배포 전에 이것을 이해하는 것이 중요한 경우 테스트 환경은 결정법을 설명하기 위한 도구를 가지고 있어야 한다.
- **하드웨어:** 인공지능기반 시스템을 호스팅하는 데 사용하는 일부 하드웨어, 예를 들어 인공지능 전용 프로세서는 해당 목적으로 특별히 설계됐을 수 있다(1.6절 참조). 이런 하드웨어를 테스트 환경에 포함해야 할 필요성이 있는지를 관련 테스트 계획 중에 고려해야 한다.
- **빅 데이터:** 인공지능기반 시스템이 빅 데이터(예: 대량, 초고속, 고다양성 데이터)를 사용할 것으로 예상되는 경우, 이를 테스트 환경의 일부로 구축하기 위한 신중한 계획과 구현이 필요하다 (7.3절 참조).

10.2 인공지능기반 시스템 테스트를 위한 가상 테스트 환경 Virtual Test Environments for Testing AI-Based Systems

인공지능기반 시스템을 테스트할 때 가상 테스트 환경을 사용하면 다음과 같은 이점이 있다:

- 위험한 시나리오: SUT, 상호작용하는 기타 시스템(인간 포함), 운영 환경(예: 나무나 건물 등)을 위험에 빠뜨리지 않고 테스트를 수행할 수 있다.
- 이상 시나리오(Unusual scenarios): 실제로 발생하는 데 너무 많이 시간/비용이 필요한 시나리오를 테스트할 수 있다(예: 개기일식 또는 4대의 버스가 동시에 같은 도로 교차로에 진입하는 상황과 같이 드물게 발생하는 이벤트를 기다리는 경우). 마찬가지로, 현실 세계에서 만들기 어려운 경계 케이스도 가상 테스트 환경에서는 더 쉽게, 더 자주, 그리고 재현 가능하게 구현할 수 있다.
- 극단적인 시나리오: 현실에서 구현하는 것이 비용이 많이 들거나 불가능한 경우도 테스트 가능하다 (예: 핵 재난 또는 먼 우주 탐사).
- 시간 집약적 시나리오: 가상 환경에서는 시간을 단축해서(예: 1초에 여러 번) 테스트할 수 있다. 이런 시나리오를 실시간으로 구축하고 실행하면 몇 시간 또는 며칠이 걸릴 수도 있다. 또 다른 이점은 여러 가상 테스트 환경을 병렬로 실행할 수 있다는 것이다. 이는 일반적으로 클라우드에서 실행하고 실제 시스템 하드웨어를 사용하면 불가능할 수 있는 많은 시나리오를 동시에 실행할 수 있게 해준다.
- 관찰 가능성과 제어 가능성: 가상 테스트 환경은 훨씬 더 높은 제어 가능성을 제공한다. 예를 들어, 비정상적인 금융 거래 조건이 반복적으로 발생하도록 할 수 있다. 또한, 디지털 방식으로 제공되는 환경의 모든 부분을 지속해서 모니터링하고 기록할 수 있기 때문에 훨씬 더 나은 관찰 가능성을 제공한다.
- 가용성: 가상 테스트 환경에 의한 하드웨어 시뮬레이션은 아직 개발되지 않았거나 너무 비용이 많이 드는 하드웨어 컴포넌트가 필요한 시스템 테스트도 (시뮬레이션을 통해) 실행할 수 있도록 한다.

가상 테스트 환경은 특정 시스템만을 위해서 구축하거나, 특정 애플리케이션 도메인을 지원하기 위해 또는 범용적으로 사용하기 위해 개발될 수 있다. 인공지능기반 시스템의 테스트를 지원하는 상용 가상 테스트 환경과 오픈소스 환경이 있으며, 그 예는 다음과 같다:

- Morse : 모듈식 오픈 로봇 시뮬레이션 엔진(Modular Open Robots Simulation Engine)으로 블렌더 게임 엔진에 기반한 하나 이상의 로봇을 위한 범용 모바일 로봇 시뮬레이션을 위한 시뮬레이터이다 [R24].
- AI Habitat : Facebook AI가 만든 시뮬레이션 플랫폼으로 구현된 에이전트(예: 가상 로봇)를 사실적인 3D 환경에서 훈련하기 위해 설계되었다 [R25].
- DRIVE Constellation : NVIDIA의 자율주행 자동차를 위한 확장 가능한 개방형 플랫폼이다. 클라우드 플랫폼을 기반으로 하며 수십 억 마일의 자율주행 차량 테스트를 생성할 수 있다 [R26].
- MATLAB 및 Simulink : 훈련 데이터를 준비하고, 기계학습 모델을 생성하고, 합성 데이터를 사용해 모델을 포함한 인공지능기반 시스템의 실행을 시뮬레이션할 수 있는 기능을 제공한다 [R27].

제11장 테스트에 인공지능 활용하기 Using AI for Testing

195분

Keywords

Visual testing

AI-Specific Keywords

Bayesian techniques, classification, clustering algorithm, defect prediction, graphical user interface (GUI)

학습 목표

11.1 테스트를 위한 인공지능 기술

AI-11.1.1 (K2) 소프트웨어 테스트에 활용하는 인공지능 기술을 분류할 수 있다.

Ho-11.1.1 (H2) 인공지능 활용 가능성이 낮은 테스트 활동을 예제를 들어가며 논의한다.

11.2 인공지능을 활용한 보고된 결함 분석

AI-11.2.1 (K2) 인공지능이 새로운 결함의 분석을 지원하는 방법을 설명할 수 있다.

11.3 인공지능을 활용한 테스트 케이스 작성

AI-11.3.1 (K2) 인공지능이 테스트 케이스 작성을 지원하는 방법을 설명할 수 있다.

11.4 인공지능을 활용한 리그레션 테스트 스위트 최적화

AI-11.4.1 (K2) 인공지능이 리그레션 테스트 스위트의 최적화를 지원하는 방법을 설명할 수 있다.

11.5 인공지능을 활용한 결함 예측

AI-11.5.1 (K2) 인공지능이 결함 예측을 지원하는 방법을 설명할 수 있다.

HO-11.5.1 (H2) 간단한 인공지능기반 결함 예측 시스템을 구현한다.

11.6 인공지능을 활용한 사용자 인터페이스 테스트

AI-11.6.1 (K2) 사용자 인터페이스 테스트에 인공지능을 활용하는 방법을 설명할 수 있다.

11.1 테스트를 위한 인공지능 기술 AI Technologies for Testing

1.4절에는 여러 인공지능 기술이 나열되어 있으며 이런 기술은 소프트웨어 테스트의 특정 측면을 지원하는 데 사용할 수 있다. Harman[B24]에 따르면 소프트웨어 엔지니어링 커뮤니티는 다음과 같은 세 가지 인공지능 기술 범위를 사용한다:

- 퍼지논리(Fuzzy logic) 및 확률적 방법: 여기에는 실제로 확률적인 문제 처리를 위한 인공지능 기술을 활용하는 것이 포함된다. 예를 들어, 인공지능을 사용해 베이지안(Bayesian) 기법으로 발생 가능한 시스템 고장을 분석하고 예측할 수 있다. 이런 기술로 컴포넌트나 기능에 오류가 발생할 가능성을 추정하거나 시스템과 인간 상호작용이 가진 잠재적인 무작위성을 반영할 수 있다.
- 분류, 학습, 예측: 이 기술은 프로젝트 계획의 일환으로 비용을 예측하거나 결함을 예측하는 것과 같은 다양한 유스케이스에 활용할 수 있다. 기계학습으로 구현되며 결함 관리(11.2절 참조), 결함 예측(11.5절 참조), 사용자 인터페이스 테스트(11.6절 참조) 등 많은 소프트웨어 테스트에 활용된다.
- 컴퓨터 기반 검색과 최적화 기법: 이 기술은 잠재적으로 크고 복잡한 검색 공간에 대한 컴퓨터 기반 검색으로 최적화 문제를 해결하는 데 사용할 수 있다(예: 검색 알고리즘을 사용해서). 예로는 테스트 케이스 작성(11.3절 참조), 주어진 커버리지 기준을 달성하기 위한 최소 테스트 케이스 식별, 리그레션 테스트 케이스 최적화(11.4절 참조)를 들 수 있다.

인공지능으로 구현할 수 있는 테스트 작업과 다양한 인공지능 기술로 가능한 작업은 겹치는 부분이 많으므로 위 분류는 광범위한 수밖에 없다. 이것은 하나의 분류법일 뿐이며, 동일하게 유효한 다른 분류법들이 만들어질 수 있다.

11.1.1 실습: 테스트에서 인공지능 활용하기

토론을 통해 현재 인공지능으로 구현하기엔 비현실적인 테스트 활동과 작업을 식별한다. 여기에는 다음이 포함될 수 있다:

- 테스트 오라클을 지정하기.
- 모호성 해결과 누락된 정보의 도출을 위해 이해관계자와 소통하기.
- 사용자 경험의 개선을 제안하기.
- 이해관계자의 가정에 도전하고 불편한 질문 던지기.
- 사용자 필요사항을 이해하기.

일부 제한된 작업에 활용할 수 있는 약인공지능과 아직은 없는 강인공지능은(1.2절 참조) 구분해서 논의해야 한다.

11.2 인공지능을 활용한 보고된 결함 분석

Using AI to Analyze Reported Defects

보고된 결함에 대해 일반적으로 분류, 우선순위 할당, 중복된 항목 식별 등이 이뤄진다. 이 활동을 결함 분류 또는 분석이라고 하며, 결함 해결에 들어가는 시간의 최적화를 목표로 한다. 인공지능은 다음과 같은 다양한 방법으로 이 활동을 지원할 수 있다:

- 분류(Categorization): NLP[B25]를 사용해 결함 보고서의 텍스트를 분석하고 영향을 받는 기능 영역 등의 주제를 추출해 k-최근접 이웃(k-nearest neighbors) 또는 서포트 벡터 머신(support vector machines)과 같은 클러스터링 알고리즘에 다른 메타 데이터와 함께 제공할 수 있다. 이런 알고리즘은 적절한 결함 분류를 파악해 유사하거나 중복된 결함을 찾아낼 수 있다. 인공지능기반 분류는 자동화된 결함 보고 시스템(예: Microsoft Windows와 Firefox를 위한)과 소프트웨어 엔지니어가 많은 대규모 프로젝트에 특히 유용하다.
- 심각도(Criticality): 가장 심각한 결함의 특성으로 훈련된 기계학습 모델을 사용해 식별한 결함 중 높은 확률로 주된 시스템 장애를 일으키는 결함을 식별할 수 있다 [B26].
- 배정(Assignment): 기계학습 모델은 결함 내용과 이전 개발자 배정 기록을 기반으로 특정 결함을 수정하는 데 가장 적합한 개발자를 제안할 수 있다.

11.3 인공지능을 활용한 테스트 케이스 작성 Using AI for Test Case Generation

인공지능을 사용해 테스트를 작성하는 것은 테스트 산출물을 빠르게 작성하고 커버리지(예: 코드 또는 요구 사항 커버리지)를 최대화하는 데 매우 효과적인 기법이 될 수 있다. 이런 테스트 작성을 위한 베이스로는 소스 코드, 사용자 인터페이스, 기계가 읽을 수 있는 테스트 모델 등이 있다. 또한, 시스템의 하위 수준 동작에 대한 관찰이나 계측값 또는 로그 파일을 기반으로 테스트를 도출하는 도구도 있다 [B27].

그러나 필요한 동작을 정의한 테스트 모델이 테스트 베이스로 사용되지 않는 한, 이런 방식의 테스트 작성은 일반적으로 테스트 오라클 문제로 어려움을 겪는다. 이것은 인공지능기반 도구가 주어진 테스트 데이터 세

트에 대한 기대 결과가 무엇인지 알 수 없기 때문이다. 이에 대한 한 가지 해결책은 슈도 오라클(pseudo-oracle)로 사용할 수 있는 적절한 시스템이 있는 경우 백투백 테스트(back-to-back testing)를 수행하는 것이다 (9.3절 참조). 아니면 “애플리케이션 응답 없음”이나 시스템 크래시(crash) 등이 발생하거나, 다른 비슷한 단순 장애가 나타나지 않는다는 기대 결과를 가지고 테스트를 수행하는 것이다.

인공지능기반 테스트 작성 도구를 유사한 비 인공지능 퍼지 테스트 도구와 비교한 연구에 따르면, 인공지능 기반 도구는 동일한 수준의 커버리지를 달성하고 장애를 일으키는 데 필요한 평균 단계를 15,000여 개의 단계에서 약 100여 개의 단계로 줄이면서 더 많은 결함을 찾을 수 있다. 따라서 디버깅이 훨씬 쉬워진다 [B27].

11.4 인공지능을 활용한 리그레션 테스트 스위트 최적화

Using AI for the Optimization of Regression Test Suites

시스템에 변경이 생기면서 새로운 테스트가 작성되고 실행되며, 이 새로운 테스트는 리그레션 테스트 스위트에 추가 대상이 된다. 리그레션 테스트 스위트가 너무 커지는 것을 방지하려면 잦은 최적화를 통해 테스트 케이스를 선택해 우선순위를 정하고 필요하면 보강해서 더욱 효과/효율적인 리그레션 테스트 스위트가 되도록 해야 한다.

예를 들어, 인공지능기반 도구로 이전 테스트 결과에 대한 정보, 관련 결함, 최근 변경 사항 등을 분석해서 리그레션 테스트 스위트의 최적화를 수행할 수 있다. 오류 발생 빈도가 높은 기능, 최근 변경의 영향을 받은 코드 실행 테스트 등을 고려할 수 있다.

연구에 따르면, 리그레션 테스트 스위트의 크기를 50% 줄여도 대부분의 결함을 식별할 수 있으며[B28], 지속적인 통합 프로세스의 테스트도 장애 검출의 현저한 감소없이 테스트 수행 시간을 40% 줄일 수 있다 [B29].

11.5 인공지능을 활용한 결함 예측 Using AI for Defect Prediction

결함 예측은 결함 존재 여부, 존재하는 결함의 수, 결함 발견 가능성을 예측하는 데 활용할 수 있다. 결함 예측 능력은 사용하는 도구의 정교함에 따라 달라진다.

결과는 일반적으로 테스트의 우선순위를 정하는 데 활용된다(예: 더 많은 결함이 예측되는 컴포넌트에 대해 더 많은 테스트 수행).

결함 예측은 일반적으로 소스 코드 측정지표, 프로세스 측정지표, 인력과 조직에 대한 측정지표를 기반으로 이뤄진다. 고려해야 할 잠재 요소가 너무 많기 때문에 이런 요소 간의 관계 설정은 인간의 능력을 넘어선다. 따

라서 기계학습을 사용하는 인공지능기반 접근법을 주로 사용하게 된다. 결함 예측은 유사한 상황(예: 동일한 코드 기반 또는 동일한 개발자)의 이전 경험을 기반으로 할 때 가장 효과적이다.

기계학습을 사용한 결함 예측은 여러 상황(예: [B30] 및 [B31])에서 성공적으로 활용되었다. 예측에 가장 많은 영향을 미치는 변수는 코드 라인이나 순환 복잡도와 같이 널리 사용되는 소스 코드 측정지표보다 인력과 조직에 대한 측정지표인 것으로 밝혀졌다 [B32].

11.5.1 실습: 결함 예측 시스템 구현

적절한 데이터 세트(예: 소스 코드 측정값과 상응하는 결함 데이터를 포함한)를 사용해 간단한 결함 예측 모델을 구축하고, 유사한 코드의 소스 코드 측정값을 사용해 결함 발생 가능성을 예측한다.

모델은 데이터 세트에서 최소 4개의 특성을 사용해야 하며, 선택한 특성에 따라 결과가 어떻게 변경되는지 보기 위해 여러 다른 특성을 사용한 결과를 살펴봐야 한다.

11.6 인공지능을 활용한 사용자 인터페이스 테스트

Using AI for Testing User Interfaces

11.6.1 인공지능 활용 그래픽 사용자 인터페이스(GUI)를 통한 테스트

GUI를 사용한 테스트는 수동 테스트의 일반적인 접근법이며(컴포넌트 테스트 제외), 테스트 자동화를 위한 시작점이 되는 경우가 많다. 이런 테스트는 테스트 대상과 인간의 상호작용을 모방한다. 스크립트를 사용한 테스트 자동화는 사용자 인터페이스 요소의 실제 좌표나 정의된 소프트웨어 객체명/위젯을 사용하는 캡처/재생 접근법을 통해 구현할 수 있다. 그러나 이런 접근법은 객체 식별 관련 몇 가지 단점이 있다. 인터페이스 변경, 코드 변경, 플랫폼 변경에 민감하게 반응하는 점도 여기에 포함된다.

인공지능은 이런 접근법이 가진 취약점을 보완할 수 있다. 인공지능기반 도구는 다양한 기준(예: XPath, 라벨, ID, 클래스, X/Y 좌표)을 기반으로 올바른 객체를 식별하고 과거 데이터로 가장 안정적인 식별 기준을 선택할 수 있다. 예를 들어, 애플리케이션의 특정 영역에 있는 버튼의 ID가 릴리즈마다 바뀌는 경우, 인공지능기반 도구는 시간이 지남에 따라 ID의 중요도를 낮추고 다른 기준에 더 많이 의존할 수 있다. 이 접근법은 사용자 인터페이스의 객체를 테스트와 부합하거나 부합하지 않는 것으로 분류하게 된다.

또한, 시각 정보를 기반으로 하는 테스트는 이미지 인식을 사용해 실제 사용자와 동일한 인터페이스에서 GUI 객체와 상호작용하므로, 코드나 인터페이스 정의에 접근할 필요가 없다. 그로 인해 완전히 비침해적(non-intrusive)이고 근간 기술에서 독립적인 테스트가 된다. 스크립트는 시각적으로 보이는 사용자 인터페이스

스를 통해서만 동작하면 된다. 이 접근법을 통해 테스터는 전체 화면 레이아웃에 영향을 받지 않고 인간과 동일한 방식으로 화면의 이미지, 버튼, 텍스트 필드와 직접 상호작용하는 스크립트를 만들 수 있다. 테스트 자동화에서 이미지 인식 사용은 필요한 컴퓨팅 자원에 의해 제한될 수 있다. 그러나 정교한 이미지 인식을 지원하는 저렴한 인공지능이 가용해짐에 따라 이제 이런 접근법을 주요 방법으로 사용할 수 있게 되었다.

11.6.2 인공지능을 활용한 GUI 테스트

기계학습 모델을 사용해 사용자 인터페이스 화면의 인수 가능 여부도 결정할 수 있다(예: 휴리스틱 및 지도 학습을 사용). 이런 모델을 기반으로 한 도구는 잘못 렌더링된 요소를 식별하고, 접근할 수 없거나 식별하기 어려운 객체가 있는지의 여부를 판단하며, GUI의 시각적 모습 관련 다양한 문제를 식별할 수 있다.

이미지 인식도 컴퓨터 비전 알고리즘의 한 형태지만, 이미지(예: 스크린샷)를 비교해 레이아웃, 크기, 위치, 색상, 글꼴, 기타 가시적 속성에 의도하지 않은 변경이 없었는지 확인하기 위해 다른 형태의 인공지능기반 컴퓨터 비전을 사용할 수 있다. 이런 비교 결과는 테스트 대상에 대한 변경이 사용자 인터페이스에 부정적인 영향을 주지 않았는지를 확인하는 리그레션 테스트를 지원하는 데 사용할 수 있다.

스크린의 인수 가능 여부를 점검하는 기술을 비교 도구와 결합해 사용자 인터페이스 변경 사항을 사용자가 수용할 수 있는지 또는 이런 변경 사항을 사람이 점검해야 하는지의 여부를 조언하는 보다 정교한 인공지능기반 리그레션 테스트 도구를 만들 수 있다. 이런 인공지능기반 도구는 애플리케이션의 사용자 인터페이스가 다양한 브라우저/기기/플랫폼에서 올바르게 동작하는지 확인하기 위한 목적으로 다양한 브라우저, 기기, 플랫폼에서의 호환성 테스트를 지원하는 데에도 사용할 수 있다.

제12장 참고문헌

12.1 Standards

- [S01] ISO/IEC TR 29119-11:2020, Software and systems engineering — Software testing — Part 11 Guidelines on the testing of AI-based systems
- [S02] DIN SPEC 92001-1, Artificial Intelligence - Life Cycle Processes and Quality Requirements - Part 1: Quality Meta Model, <https://www.din.de/en/wdc-beuth: din21:303650673> (accessed May 2021).
- [S03] DIN SPEC 92001-2, Artificial Intelligence - Life Cycle Processes and Quality Requirements - Part 2: Technical and Organizational Requirements, <https://www.din.de/en/innovation-and-research/din-spec-en/projects/wdc-proj: din21:298702628> (accessed May 2021).
- [S04] ISO 26262 - <https://www.iso.org/standard/68383.html> (accessed May 2021)
- [S05] ISO/PAS 21448:2019, Road vehicles – Safety of the intended functionality (SOTIF) - <https://www.iso.org/standard/70939.html> (accessed May 2021)
- [S06] ISO/IEC 25010:2011, Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models, 2011.
- [S07] ISO 26262-6:2018 - Road vehicles - Functional safety - Part 6: Product development at the software level.
- [S08] ISO/IEC/IEEE 29119-4:2015, Software and systems engineering — Software testing — Part 4: Test techniques.

12.2 ISTQB® Documents

- [I01] ISTQB® Certified Tester Foundation Level Syllabus, Version 2018 V3.1
<https://www.istqb.org/downloads/category/2-foundation-level-documents.html>
(accessed May 2021).

- [I02] ISTQB® Certified Tester Advanced Level Test Analyst Syllabus, Version 3.1, section 3.2.6 <https://www.istqb.org/downloads/category/75-advanced-level-test-analyst-v3-1.html>(accessed August 2021).
- [I03] ISTQB® Certified Tester AI Testing, Overview of Syllabus, Version 1.0

12.3 Books and Articles

- [B01] Cadwalladr, Carole (2014). "Are the robots about to rise? Google's new director of engineering thinks so..." The Guardian. Guardian News and Media Limited., <https://www.theguardian.com/technology/2014/feb/22/robots-google-ray-kurzweil-terminator-singularity-artificial-intelligence> (accessed May 2021).
- [B02] Stuart Russell and Peter Norvig, Artificial Intelligence: A Modern Approach, 4th Edition, Pearson, 2020.
- [B03] M. Davies et al., "Advancing Neuromorphic Computing With Loihi: A Survey of Results and Outlook," Proceedings of the IEEE, vol. 109, no. 5, pp. 911–934, May 2021, doi: 10.1109/JPROC.2021.3067593.
- [B04] Chris Wiltz, Can Apple Use Its Latest AI Chip for More Than Photos?, Electronics & Test, Artificial Intelligence, <https://www.designnews.com/electronics-test/can-apple-use-its-latest-ai-chip-more-photos/153617253461497> (accessed May 2021).
- [B05] HUAWEI Reveals the Future of Mobile AI at IFA 2017, Huawei Press Release, <https://consumer.huawei.com/en/press/news/2017/ifa2017-kirin970/> (accessed May 2021).
- [B06] REGULATION (EU) 2016/679 OF THE EUROPEAN PARLIAMENT AND OF THE COUNCIL on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Data Protection Regulation), April 2016, <https://eur-lex.europa.eu/eli/reg/2016/679/oj> (accessed May 2021)
- [B07] Taxonomy and Definitions for Terms Related to Driving Automation Systems for On- Road Motor Vehicles J3016_201806, SAE, https://www.sae.org/standards/content/j3016_201806/ (accessed May 2021).
- [B08] G20 Ministerial Statement on Trade and Digital Economy: Annex. Available from: <https://www.mofa.go.jp/files/000486596.pdf> (accessed May 2021).

- [B09] Concrete Problems in AI Safety, Dario Amodei (Google Brain), Chris Olah (Google Brain), Jacob Steinhardt (Stanford University), Paul Christiano (UC Berkeley), John Schulman (OpenAI), Dan Mané (Google Brain), March 2016.
<https://arxiv.org/pdf/1606.06565> (accessed May 2021).
- [B10] Explainable AI: the basics, Policy briefing, Issued: November 2019 DES6051, ISBN: 978-1-78252-433-5, The Royal Society.
- [B11] The Ultimate Guide to Data Labeling for Machine Learning,
www.cloudfactory.com/data-labeling-guide (accessed May 2021).
- [B12] Pei et al, DeepXplore: Automated Whitebox Testing of Deep Learning Systems, Proceedings of ACM Symposium on Operating Systems Principles (SOSP '17), Jan 2017.
- [B13] Sun et al, Testing Deep Neural Networks, https://www.researchgate.net/publication/323747173_Testing_Deep_Neural_Networks, accessed Nov 2019 (accessed May 2021).
- [B14] A. Odena and I. Goodfellow, TensorFuzz: Debugging Neural Networks with Coverage-Guided Fuzzing, ArXiv e-prints, Jul. 2018, <https://arxiv.org/pdf/1807.10875> (accessed May 2021)
- [B15] Riccio, V. et al, Testing Machine Learning based Systems: A Systematic Mapping. Empirical Software Engineering, <https://link.springer.com/article/10.1007/s10664-020-09881-0> (accessed May 2021)
- [B16] Baudel, Thomas et al, Addressing Cognitive Biases in Augmented Business Decision Systems., <https://arxiv.org/abs/2009.08127> (accessed May 2021)
- [B17] Papernot, N. et al, Transferability in machine learning: from phenomena to black-box attacks using adversarial samples, arXiv preprint arXiv:1605.07277, 2016. <https://arxiv.org/pdf/1605.07277> (accessed May 2021).
- [B18] Chen et al, Metamorphic Testing: A Review of Challenges and Opportunities, ACM Comput. Surv. 51, 1, Article 4, January 2018.
https://www.researchgate.net/publication/322261865_Metamorphic_Testing_A_Review_of_Challenges_and_Opportunities (accessed May 2021).
- [B19] Huai Liu, Fei-Ching Kuo, Dave Towey, and Tsong Yueh Chen., How effectively does metamorphic testing alleviate the oracle problem?, IEEE Transactions on Software Engineering 40, 1, 4–22, 2014
- [B20] James Whittaker, Exploratory Software Testing: Tips, Tricks, Tours and Techniques to Guide Test Design, 1. Edition, Addison-Wesley Professional, 2009.

- [B21] L. Wilkinson, A. Anand, and R. Grossman. High-dimensional visual analytics: Interactive exploration guided by pairwise views of point distributions. Visualization and Computer Graphics, IEEE Transactions on, 12(6):1363–1372, 2006, <https://www.cs.uic.edu/~wilkinson/Publications/sorting.pdf> (accessed May 2021).
- [B22] Ryan Hafen and Terence Critchlow, EDA and ML – A Perfect Pair for Large-Scale Data Analysis, IEEE 27th International Symposium on Parallel and Distributed Processing, 2013, <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=6651091> (accessed May 2021).
- [B23] Breck, Eric, Shanjing Cai, Eric Nielsen, Michael Salib, and D. Sculley., The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction, IEEE International Conference on Big Data (Big Data), 2017, <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=8258038> (accessed May 2021).
- [B24] Harman, The Role of Artificial Intelligence in Software Engineering, In First International Workshop on Realizing AI Synergies in Software Engineering (RAISE), pp. 1-6. IEEE, June 2012, <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=6227961>(accessed May 2021).
- [B25] Nilambri et al, A Survey on Automated Duplicate Detection in a Bug Repository, International Journal of Engineering Research & Technology (IJERT), 2014, <https://www.ijert.org/research/a-survey-on-automated-duplicate-detection-in-a-bug-repository-IJERTV3IS041769.pdf> (accessed May 2021)
- [B26] Kim, D.; Wang, X.; Kim, S.; Zeller, A.; Cheung, S.C.; Park, S. (2011). “Which Crashes Should I Fix First? Predicting Top Crashes at an Early Stage to Prioritize Debugging Efforts,” in the IEEE Transactions on Software Engineering, volume 37, <https://ieeexplore.ieee.org/document/5711013> (accessed May 2021).
- [B27] Mao et al, Sapienz: multi-objective automated testing for Android applications, Proceedings of the 25th International Symposium on Software Testing and Analysis, July 2016, http://www0.cs.ucl.ac.uk/staff/K.Mao/archive/p_issta16_sapienz.pdf(accessed May 2021).
- [B28] Rai et al, Regression Test Case Optimization Using Honey Bee Mating Optimization Algorithm with Fuzzy Rule Base, World Applied Sciences Journal 31 (4): 654-662, 2014, https://www.researchgate.net/publication/336133351_Regression_Test_Case_Optimization_Using_Honey_Bee_Mating_Optimization_Algorithm_with_Fuzzy_Rule_Base (accessed May 2021).
- [B29] Dusica Marijan, Arnaud Gotlieb, Marius Liaaen. A learning algorithm for optimizing continuous integration development and testing practice, Journal of Software : Practice and Experience, Nov 2018.

- [B30] Tosun et al, AI-Based Software Defect Predictors: Applications and Benefits in a Case Study, Proceedings of the Twenty-Second Innovative Applications of Artificial Intelligence Conference (IAAI-10), 2010.
- [B31] Kim et al, Predicting Faults from Cached History, 29th International Conference on Software Engineering (ICSE'07), 2007.
- [B32] Nagappan 2008 Nagappan et al, The Influence of Organizational Structure on Software Quality: An Empirical Case Study, Proceedings of the 30th international conference on Software engineering (ICSE'08), May 2008.
- [B33] Kuhn et al, Software Fault Interactions and Implications for Software Testing, IEEE Transactions on Software Engineering vol. 30, no. 6, (June 2004) pp. 418-421.

12.4 기타 참고문헌

The following references point to information available on the Internet. Even though these references were checked at the time of publication, the ISTQB® cannot be held responsible if the references are no longer available.

- [R01] Wikipedia contributors, "AI effect," Wikipedia, https://en.wikipedia.org/wiki/AI_effect (accessed May 2021).
- [R02] <https://mxnet.apache.org/> (accessed May 2021).
- [R03] <https://docs.microsoft.com/en-us/cognitive-toolkit/> (accessed May 2021).
- [R04] IBM Watson, <https://www.ibm.com/watson/ai-services>
- [R05] <https://www.tensorflow.org/> (accessed May 2021).
- [R06] <https://keras.io/> (accessed May 2021).
- [R07] <https://pytorch.org/> (accessed May 2021).
- [R08] https://scikit-learn.org/stable/whats_new/v0.23.html (accessed May 2021).
- [R09] NVIDIA VOLTA, <https://www.nvidia.com/en-us/data-center/volta-gpu-architecture/> (accessed May 2021).
- [R10] Cloud TPU, <https://cloud.google.com/tpu/> (accessed May 2021).

- [R11] Edge TPU, <https://cloud.google.com/edge-tpu/> (accessed May 2021).
- [R12] Intel® Nervana™ Neural Network processors deliver the scale and efficiency demanded by deep learning model evolution, <https://www.intel.ai/nervana-nnp/> (accessed May 2021).
- [R13] The Evolution of EyeQ, <https://www.mobileye.com/our-technology/evolution-eyeq-chip/> (accessed May 2021).
- [R14] ImageNet - <http://www.image-net.org/> (accessed May 2021).
- [R15] Google's BERT - <https://github.com/google-research/bert> (accessed May 2021).
- [R16] <https://www.kaggle.com/datasets> (accessed May 2021).
- [R17] <https://www.kaggle.com/paultimothymooney/2018-kaggle-machine-learning-data-science-survey> (accessed May 2021).
- [R18] MLCommons - <https://mlcommons.org/> (accessed May 2021).
- [R19] DAWNBench – <https://dawn.cs.stanford.edu/benchmark> (accessed May 2021).
- [R20] MLMark – <https://www.eembc.org/mlmark> (accessed May 2021).
- [R21] <https://digital-strategy.ec.europa.eu/en/library/assessment-list-trustworthy-artificial-intelligence-altai-self-assessment> Shaping Europe's digital future (europa.eu) (accessed August 2021)
- [R22] <https://digital-strategy.ec.europa.eu/en/library/ethics-guidelines-trustworthy-ai> (accessed August 2021)
- [R23] Google GraphicsFuzz, <https://github.com/google/graphicsfuzz> (accessed May 2021).
- [R24] <http://www.openrobots.org/morse/doc/0.2.1/morse.html> (accessed May 2021).
- [R25] <https://ai.facebook.com/blog/open-sourcing-ai-habitat-a-simulation-platform-for-embodied-ai-research/> (accessed May 2021).
- [R26] <https://www.nvidia.com/en-gb/self-driving-cars/drive-constellation/> (accessed May 2021).
- [R27] <https://uk.mathworks.com/discovery/artificial-intelligence.html#ai-with-matlab> (accessed May 2021).

제13장 부록 A – 약어

Abbreviation	Description
AI	Artificial intelligence
AlaaS	AI as a service
API	Application programming interface
AUC	Area under curve
DL	Deep learning
DNN	Deep neural network
EDA	Exploratory data analysis
EU	European Union
FN	False negative
FP	False positive
GDPR	General data protection regulation
GPU	Graphical processing unit
GUI	Graphical user interface
LIME	Local interpretable model-agnostic explanations
MC/DC	Modified condition decision coverage
ML	Machine learning
MR	Metamorphic relation
MSE	Mean square error
MT	Metamorphic testing
NLP	Natural language processing
ROC	Receiver operating characteristic
SUT	System under test
SVM	Support vector machine
TN	True negative
TP	True positive
XAI	Explainable AI

제14장 부록 B – AI Specific and other Terms*

Term Name	Definition
accuracy	The ML functional performance metric used to evaluate a classifier, which measures the proportion of predictions that were correct (After ISO/IEC TR 29119-11)
activation function	The formula associated with a neuron in a neural network that determines the output of the neuron from the inputs to the neuron
activation value	The output of an activation function of a neuron in a neural network
adversarial attack	The deliberate use of adversarial examples to cause an ML model to fail
AI as a Service (AlaaS)	A software licensing and delivery model in which AI and AI development services are centrally hosted
AI component	A component that provides AI functionality
AI effect	The situation when a previously labelled AI system is no longer considered to be AI as technology advances (ISO/IEC TR 29119-11)
AI-based system	A system that integrates one or more AI components
AI-specific processor	A type of specialized hardware designed to accelerate AI applications
algorithmic bias	A type of bias caused by the ML algorithm
annotation	The activity of identifying objects in images with bounding boxes to provide labelled data for classification
area under curve (AUC)	A measure of how well a classifier can distinguish between two classes.
artificial intelligence (AI)	The capability of an engineered system to acquire, process, create and apply knowledge and skills (ISO/IEC TR 29119-11)
association	An unsupervised learning technique that identifies relationships and dependencies between samples
augmentation	The activity of creating new data points based on an existing dataset

* 부록 B의 번역이 필요한 분들은 ISTQB Glossary가 번역중이므로 후에 ISTQB Glossary 번역본을 참조하시기 바랍니다.

Term Name	Definition
automation bias	A type of bias caused by a person favoring the recommendations of an automated decision-making system over other sources Synonym: complacency bias
autonomous system	A system capable of working without human intervention for sustained periods
autonomy	The ability of a system to work for sustained periods without human intervention (ISO/IEC TR 29119-11)
Bayesian model	A statistical model that uses probability to represent the uncertainty of both model inputs and outputs
Bayesian technique	A technique that considers before and after probability distributions as parameters of a statistical model
bias	The systematic difference in treatment of certain objects, people or groups in comparison to others (ISO/IEC DIS 22989)
big data	Extensive datasets whose characteristics in terms of volume, variety, velocity and/or variability require specialized technologies and techniques to process
case-based reasoning	The technique of solving a new problem based on the solutions of similar past problems
chatbot	An application used to conduct a conversation via text or text-to-speech
classification	A type of ML function that predicts the output class for a given input (After ISO/IEC TR 29119-11)
classifier	An ML model used for classification Synonym: classification model
clustering	A type of ML function that groups similar data points together
clustering algorithm	A type of ML algorithm used to group similar objects into clusters
concept drift	A change in the perceived accuracy of an ML model predictions over time caused by changes in user expectations, behavior and the operational environment.
confusion matrix	A technique for summarizing the ML functional performance of a classification algorithm
data acquisition	The activity of acquiring data relevant to the business problem to be solved by an ML model

Term Name	Definition
data labelling	The activity of adding meaningful tags to objects in raw data to support classification in ML
data pipeline	The implementation of data preparation activities to provide input data to support training by an ML algorithm or prediction by an ML model
data point	A set of one or more measurements comprising a single observation used as part of a dataset
data poisoning	The deliberate and malicious manipulation of training or input data to an ML model
data preparation	The activities of data acquisition, data pre-processing and feature engineering in the ML workflow
data pre-processing	The activities of data cleaning, data transformation, data augmentation, and data sampling in the ML workflow
data visualization	A technique for graphically representing data relationships, trends and patterns
dataset	A collection of data used for training, evaluation, testing and prediction in ML
decision threshold	A value that transforms the result of a prediction function into a binary outcome of either above or below the value Synonym: discrimination threshold
decision tree	A tree-like ML model whose nodes represent decisions, and whose branches represent possible outcomes
deductive classifier	A classifier based on the application of inference and logic to input data
deep learning (DL)	ML using neural networks with multiple layers
deep neural network	A neural network comprised of several layers of neurons Synonym: multi-layer perceptron
defect prediction	A technique to predict the areas within the test object in which defects will occur or the quantity of defects that are present
deterministic system	A system which will produce the same set of outputs and final state from a given set of inputs and starting state
edge computing	The part of a distributed architecture in which information processing is performed close to where that information is used.

Term Name	Definition
epoch	An iteration of ML training on the whole training dataset
evolution	The process of continuous change from a lower, simpler, or worse state to a higher, more complex, or better state
expert system	An AI-based system for solving problems in a particular domain or application area by drawing inferences from a knowledge base developed from human expertise
explainability	The level of understanding how the AI-based system came up with a given result (ISO/IEC TR 29119-11)
explainable AI (XAI)	The field of study related to understanding the factors that influence AI system outputs
exploratory data analysis (EDA)	The interactive, hypothesis-driven and visual exploration of data used to support feature engineering
F1-Score	An ML functional performance metric used to evaluate a classifier which provides a balance between recall and precision
false negative (FN)	An ML model prediction in which the model mistakenly predicts the negative class
false positive (FP)	An ML model prediction in which the model mistakenly predicts the positive class
feature	An individual measurable attribute of the input data used for training by an ML algorithm and for prediction by an ML model
feature engineering	The activity in which those attributes in the raw data that best represent the underlying relationships that should appear in the ML model are identified for use in the training data (ISO/IEC TR 29119-11)
flexibility	The ability of a system to work in contexts outside its initial specification (After ISO/IEC TR 29119-11)
fuzzy logic	A type of logic based on the concept of partial truth represented by certainty factors between 0 and 1
general AI	AI that exhibits intelligent behaviour comparable to a human across the full range of cognitive abilities (ISO/IEC TR 29119-11) Synonym: strong AI
General Data Protection Regulation (GDPR)	The European Union (EU) regulation on data protection and privacy that applies to the data of citizens of the EU and the European Economic Area

Term Name	Definition
graphical processing unit (GPU)	An application-specific integrated circuit designed to manipulate and alter memory to accelerate the creation of images in a frame buffer intended for output to a display device
ground truth	The information provided by direct observation and measurement that is known to be real or true
hyperparameter	A parameter used to either control the training of an ML model or to set the configuration of an ML model
hyperparameter tuning	The activity of determining the optimal hyperparameters based on particular goals
inappropriate bias	A type of bias that causes a system to produce results that lead to adverse effects for a particular group
intelligent agent	An autonomous program which directs its activity towards achieving goals using observations and actions
inter-cluster metric	A metric that measures the similarity of data points in different clusters
interpretability	The level of understanding how the underlying AI technology works (ISO/IEC TR 29119-11)
intra-cluster metric	A metric that measures the similarity of data points within a cluster
k-nearest neighbor	An approach to classification that estimates the likelihood of group membership for a data point dependent on the group membership of the data points nearest to it
learning algorithm	A program that produces an ML model based on the properties of the training dataset
LIME method	The Local Interpretable Model-Agnostic Explanations program for explaining the predictions from an ML model
linear regression	A statistical technique that models the relationship between variables by fitting a linear equation to the observed data when the target variable is numeric
logistic regression	A statistical technique that models the relationship between variables when the target variable is categorical rather than numeric
machine learning (ML)	The process using computational techniques to enable systems to learn from data or experience (ISO/IEC TR 29119-11)
mean square error (MSE)	The statistical measure of the average squared difference between the estimated values and the actual value

Term Name	Definition
ML algorithm	An algorithm used to create an ML model from a training dataset
ML benchmark suite	A dataset used to compare ML models and ML algorithms over a range of evaluation metrics
ML framework	A tool or library that supports the creation of an ML model
ML function	Functionality implemented by an ML model, such as classification, regression or clustering
ML model evaluation	The process of comparing achieved ML functional performance metrics with required criteria and those of other ML models
ML model training	The process of applying the ML algorithm to the training dataset to create an ML model
ML model tuning	The process of testing hyperparameters to achieve optimum performance
ML system	A system that integrates one or more ML models
ML workflow	A sequence of activities used to manage the development and deployment of an ML model
multi-agent system	A system that comprises multiple intelligent agents
narrow AI	AI focused on a single well-defined task to address a specific problem (ISO/IEC TR 29119-11) Synonym: weak AI
natural language processing (NLP)	A field of computing that provides the ability to read, understand, and derive meaning from natural languages
neural network	A network of primitive processing elements connected by weighted links with adjustable weights, in which each element produces a value by applying a nonlinear function to its input values, and transmits it to other elements or presents it as an output value (ISO/IEC 2382) Synonym: artificial neural network
neural network trojan	A vulnerability injected into a neural network using a data poisoning attack with the intent of exploiting it later
neuromorphic processor	An integrated circuit designed to mimic the biological neurons of the human brain
neuron	A node in a neural network, usually receiving multiple input values and generating an activation value

Term Name	Definition
noise	A distortion or corruption in data
non-deterministic system	A system which will not always produce the same set of outputs and final state given a particular set of inputs and starting state
outlier	An observation that lies outside the overall pattern of the data distribution
overfitting	The generation of an ML model that corresponds too closely to the training dataset, resulting in a model that finds it difficult to generalize to new data (After ISO/IEC TR 29119-11)
perceptron	A neural network with just one layer and one neuron
precision	An ML functional performance metric used to evaluate a classifier, which measures the proportion of predicted positives that were correct (After ISO/IEC TR 29119-11)
pre-trained model	An ML model already trained when it was obtained
probabilistic system	A system whose behavior is described in terms of probabilities; hence its outputs cannot be perfectly predicted
procedural reasoning	AI technology used for constructing real-time reasoning systems that can perform complex tasks in dynamic environments
random forest	Ensemble ML technology for classification, regression and other tasks that operate by constructing and running many decision trees and then either outputting the mode of the class or the mean prediction of the individual trees
reasoning technique	AI that generates conclusions from available information using logical techniques (After ISO/IEC TR 29119-11)
recall	An ML functional performance metric used to evaluate a classifier, which measures the proportion of actual positives that were predicted correctly (After ISO/IEC TR 29119-11) Synonym: sensitivity
receiver operating characteristic (ROC) curve	A graphical plot that illustrates the ability of a binary classifier as its discrimination threshold is varied
regression	A type of ML function that results in a numerical or continuous output value for a given input (After ISO/IEC TR 29119-11)
regression model	An ML model whose expected output for a given numeric input is a continuous variable (After ISO/IEC DIS 23053)

Term Name	Definition
reinforcement learning	The activity of building an ML model using a process of trial and reward to achieve an objective (After ISO/IEC TR 29119-11)
reward function	A function that defines the success of reinforcement learning
reward hacking	The activity performed by an intelligent agent to maximize its reward function to the detriment of meeting the original objective (After ISO/IEC TR 29119-11)
R-squared	A statistical measure of how close the data points are to the fitted regression line. Synonym: coefficient of determination
rule engine	A set of rules that determine which actions should occur when certain conditions are satisfied
safety	The expectation that a system does not, under defined conditions, lead to a state in which human life, health, property, or the environment is endangered (ISO/IEC/IEEE 12207)
sample bias	A type of bias where the dataset is not fully representative of the data space to which ML is applied
search algorithm	An algorithm that systematically visits a subset of all possible states or structures until the goal state or structure is reached (After ISO/IEC TR 29119-11)
self-learning system	An adaptive system that changes its behavior based on learning through trial and error (After ISO/IEC TR 29119-11)
silhouette coefficient	A clustering measure between -1 and +1 based on the average inter-cluster and intra-cluster differences Synonym: silhouette score
super AI	An artificial intelligence-based system that far exceeds human capabilities
supervised learning	Training an ML model from input data and its corresponding labels
support vector machine (SVM)	An ML technique in which the data points are viewed as vectors in multi-dimensional space separated by a hyperplane
technological singularity	A point in the future when technological advances are no longer controllable by people (After ISO/IEC TR 29119-11)
test oracle problem	The challenge of determining whether a test has passed or failed for a given set of test inputs and state

Term Name	Definition
training dataset	A dataset used to train an ML model
transfer learning	A technique for modifying a pre-trained ML model to perform a different related task
transparency	The level of visibility of the algorithm and data used by the AI-based system (After ISO/IEC TR 29119-11)
true negative (TN)	A prediction in which the model correctly predicts the negative class
true positive (TP)	A prediction in which the model correctly predicts the positive class
underfitting	The generation of an ML model that does not reflect the underlying trend of the training dataset, resulting in a model that finds it difficult to make accurate predictions (ISO/IEC TR 29119-11)
unsupervised learning	Training an ML model from input data using an unlabeled dataset
validation dataset	A dataset used to evaluate a trained ML model with the purpose of tuning the model
von Neumann architecture	A computer architecture which consists of five main components: memory, a central processing unit, a control unit, input and output
weight	An internal variable of a connection between neurons in a neural network that affects how it computes its outputs and that changes as the neural network is trained